

REPUBLIQUE ALGERIENNE DEMOCRATIQUE ET POPULAIRE
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE
SCIENTIFIQUE

UNIVERSITE YAHIA FARES DE MEDEA



Faculté de Technologie

Département du Génie Electrique

Mémoire de fin d'études de Master

Filière : Télécommunications

Spécialité : Systèmes des Télécommunications

**Reconnaissance automatique des plaques d'immatriculation par
réseau CNN**

Présenté par :

Mlle : TARED Ikram

Proposé et Dirigé par :

Dr. MAAZOUZ Mohamed

Dr. TOUBAL Abdelmoughni

REMERCIEMENTS

Ce n'est pas parce que la tradition exige que cette page se trouve dans notre rapport, mais c'est bien parce que les personnes à qui nous nous adressons le méritent vraiment.

En préambule à ce mémoire, je remercie ALLAH qui m'a aidé et donné la patience et le courage durant ces longues années d'étude

Je remercie également ma famille pour toute l'aide qu'ils m'ont apportée pendant mes études.

Je tiens à remercier chaleureusement et respectueusement le promoteur Dr Maazouz Mohamed et de notre co-promoteur M. Toubal Abdelmoughni.

Merci également aux membres du jury qui ont montré de l'intérêt à ce projet en acceptant d'examiner ce travail.

En fin, je remercie également tous ceux qui ont contribué, de près ou de loin à l'élaboration de ce travail.

DEDICACES

Grace à Dieu tout puissant et en signe de reconnaissance à tous les sacrifices consentis pour ma réussite et la volonté pour mener bien ce modeste travail que je le dédie :

À celle qui m'a encouragé et m'a soutenu pour réaliser tous mes succès pendant toute mon parcours de formation, c'est pour vous ma chère mère que Dieu vous protège.

À mon très cher père qui m'a soutenue et encouragé dans tous les domaines et surtout pour réaliser ce mémoire.

À ma sœur Nour elhouda et mon frère Abd elraouf à qui je souhaite un avenir prospère.

À toute la famille Tared et Guellati.

Mon encadreur : Dr. MAZOUZ Mohamed à qui je tiens à adresser mes plus vifs remerciements, pour sa patience, sa disponibilité et surtout ses judicieux conseils, qui ont contribué à alimenter ma réflexion et ses encouragements lors de la réalisation de ce mémoire.

Co-encadreur : Dr. TOUBAL Abdelmoughni

À tous ceux qui par un mot m'ont donné la force de continuer je vous remercie de tout cœur.

ملخص:

أصبح نظام التعرف التلقائي على لوحات الترخيص «RAPI» دقيقا وفعالا وضرورة متزايدة في إدارة العديد من المجالات مثل حركة المرور والسلامة على الطرق، إدارة مواقف السيارات، مكافحة السرقة وملاحقة المجرمين. يتمثل العمل المقدم في هذه المذكرة في تنفيذ برنامج يسمح لنا بالتعرف على لوحات ترقيم السيارات الجزائرية باستخدام الشبكات العصبية الالتفافية (شبكة CNN). يحتوي نظام «RAPI» على تقنيتين رئيسيتين لمعالجة الصور، وهما الكشف عن ملامح لوحة الترقيم في صورة تم الحصول عليها بتدرج الرمادي والتعرف بعد ذلك على أرقامها. لذلك يقترح هذا العمل الحالي تصميم وتنفيذ نظام التعرف الآلي على لوحات الترخيص الجزائرية بناء على تقنيات الكشف والتعرف على الحروف وتختلف من حيث الطرق المعتمدة كالاتماد على لون اللوحات المعروف مسبق لاستخراج الأرقام والتعرف عليها، والذي يشغل على جهاز راسبيري «Raspberry Pi3» وكاميرا «Raspberry». خلال مرحلة التصميم قمنا باستخدام المكتبات التالية: أوبن سيفي «OpenCV» الخاصة بمعالجة الصور و الخوازميات المتعلقة بها، كيراس «Keras» مكتبة التعلم العميق وتنسورفلو «Tensorflow» مكتبة للتعلم الآلي ولغة البرمجة (Python) بايثون. تظهر النتائج التجريبية التي تم الحصول عليها أن الطريقة المقترحة تحقق معدلات تعرف مقبولة على الرغم من عدم الامتثال لمعايير تصميم ألواح الترقيم، مما يصعب التعرف عليها تلقائيا.

كلمات مفتاحية: شبكة CNN، نظام «RAPI»، معالجة الصور، راسبيري، لغة البرمجة بايثون.

Résumé :

Le système de reconnaissance automatique des plaques d'immatriculation RAPI est devenu plus robuste et efficace, et une nécessité dans la gestion de plusieurs domaines comme la circulation et la sécurité routière, la gestion des parkings, la lutte contre les vols et la poursuite des criminels.

Le travail présenté dans ce mémoire consiste à réaliser un programme qui nous permet de faire une reconnaissance sur les plaques d'immatriculation algérienne en utilisant les réseaux de neurones convolutif (réseau CNN). Le système RAPI utilise deux techniques majeures de traitement d'image, qui sont la détection de contours de la plaque d'immatriculation à partir d'une image acquise au niveau de gris et la reconnaissance de ses caractères. Par conséquent ce présent travail propose de concevoir et implémenter le système de reconnaissance automatique des plaques d'immatriculation algérienne basé sur la détection et les techniques de reconnaissance des caractères, le système a été implémenté sur une carte « Raspberry Pi 3 » utilisant une caméra « Raspberry ». Au cours de la mise en œuvre nous avons utilisé les bibliothèques suivantes : OpenCV dédiée au traitement d'images et ses algorithmes, Keras qui est une bibliothèque de Deep Learning, Tensorflow pour le Maching Learning et le langage de programmation Python. Les résultats des expérimentations obtenus montrent que la méthode proposée achève des taux de reconnaissance acceptable malgré le non-respect des normes de

construction des plaques d'immatriculation ce qui rend difficile de les identifier automatiquement.

Mots clés : Réseau CNN, système RAPI, traitement d'image, Raspberry Pi.3, langage de programmation Python.

Abstract:

Automatic license plate recognition “ALPR” system is developed to be robust and more efficient and are increasingly become a necessity in the management of several areas such as traffic and road safety, parking management, the fight against theft and prosecuting criminals.

The work presented in this dissertation is to carry out a software that allows the recognition of Algerian license plates using convolutional neural networks (CNN network). The ALPR system has two major image processing techniques, which are the detection of license plate contours from a gray level image and the recognition of its characters. Therefore the present work proposes to design and implement the Algerian automatic license plate recognition system based on the detection and character recognition techniques, where the system has been implemented on a “Raspberry Pi 3” and the “Raspberry” camera. During the implementation we used the following libraries: OpenCV dedicated to image processing and its algorithms, Keras which is a library for Deep Learning, Tensorflow for Machine Learning and the Python programming language. The results of the experiments obtained show that the proposed method achieves acceptable recognition rates despite non-compliance with construction standards for license plates, which makes it difficult to identify them automatically.

Key words: CNN network, ALPR system, image processing, Raspberry Pi.3, Python programming language.

TABLE DES MATIÈRES

<i>Remerciements</i>	I
<i>Dédicaces</i>	II
Table des matières	V
Liste des figures	X
Liste des tableaux.....	X
Liste des abréviation	- 1 -
Introduction générale	- 2 -
Chapitre I. Les notions de base sur le traitement d'images et les techniques de système de RAPI	- 4 -
I.1 Introduction.....	- 0 -
I.2 Qu'est-ce qu'une image ?	- 0 -
I.3 Représentation d'image	- 0 -
I.3.1 Image matricielle	- 0 -
I.3.2 Image vectorielle	- 1 -
I.4 Caractéristiques d'une image numérique.....	- 1 -
I.4.1 Pixel	- 1 -
I.4.2 Dimension.....	- 2 -
I.4.3 Résolution.....	- 2 -
I.4.4 La taille d'une image	- 2 -
I.4.5 Bruit	- 3 -
I.4.6 Histogramme	- 3 -
I.4.7 Contours et textures	- 3 -

I.4.8	Luminance	- 4 -
I.4.9	Contraste	- 4 -
I.4.10	Images à niveaux de gris	- 4 -
I.4.11	Images en couleurs	- 5 -
I.5	Différents types d'images	- 5 -
I.6	Formats d'images numériques	- 6 -
I.6.1	Format GIF (Graphics Interchange Format).....	- 6 -
I.6.2	Format TIFF (Tagged Image File Format)	- 6 -
I.6.3	Format JPG / JPEG (Joint Photographic Experts Group).....	- 6 -
I.6.4	Format PNG (Portable Network Graphics)	- 7 -
I.7	Traitement d'images	- 7 -
I.8	Principale étapes de traitement d'images.....	- 7 -
I.8.1	Prétraitements des images	- 7 -
I.8.2	Amélioration d'image.....	- 8 -
I.8.3	Analyse d'image	- 8 -
I.9	Système de reconnaissance automatique des plaques d'immatriculation....	- 9 -
I.9.1	Les difficultés rencontrées par RAPI.....	- 9 -
I.9.2	Les techniques de reconnaissance des plaques d'immatriculation.....	- 10 -
I.10	Etapes de système RAPI	- 13 -
I.10.1	Acquisition d'une image.....	- 13 -
I.10.2	Prétraitement.....	- 14 -
I.10.3	Segmentation des plaque d'immatriculation	- 14 -
I.10.4	Reconnaissance de caractères	- 16 -
I.11	Conclusion.....	- 17 -
Chapitre II.	Les concepts fondamentaux de L'intelligence artificielle, Deep Learning, Maching Learning et réseaux de neurone.....	18
II.1	Introduction.....	- 19 -
II.2	L'intelligence artificielle.....	- 19 -

II.3	Qu'est-ce que le Maching Learning ?	- 19 -
II.4	Les problèmes que le Maching Learning peut résoudre	- 21 -
II.5	Les types du Maching Learning.....	- 21 -
II.6	Le Deep Learning (L'apprentissage Profond)	- 24 -
II.7	Pourquoi le Deep Learning est important ?	- 25 -
II.8	Exemples d'application de Deep Learning	- 26 -
II.9	Fonctionnement du Deep Learning.....	- 27 -
II.10	Qu'est-ce qui différencie le Maching Learning du Deep Learning ?.....	- 27 -
II.11	Choisir entre le Maching Learning et le Deep Learning	- 28 -
II.12	Fondations de réseaux de neurones	- 29 -
II.12.1	Définition	- 29 -
II.12.2	Principe de fonctionnement.....	- 29 -
II.12.3	Neurone biologique	- 30 -
II.12.4	Neurone formel	- 31 -
II.12.5	Neurone artificiel.....	- 32 -
II.13	Architecture des réseaux de neurones	- 32 -
II.13.1	Réseaux statiques	- 33 -
II.13.2	Réseaux dynamiques	- 33 -
II.13.3	Réseaux auto-organisés	- 33 -
II.14	Apprentissage des neurones artificiels	- 33 -
II.14.1	Définition	- 33 -
II.14.2	Sur-apprentissage	- 34 -
II.15	Les types d'apprentissage.....	- 34 -
II.15.1	Apprentissage supervisé	- 34 -
II.15.2	Apprentissage non supervisé	- 34 -
II.16	Les types de réseaux de neurones	- 35 -
II.16.1	Les RNA à apprentissage supervisé (non boucles)	- 35 -
II.16.2	Les RNA à apprentissage non supervisé (récurrents ou bouclés)	- 36 -

II.17	Les branches du réseau neuronal.....	- 37 -
II.18	Domaines d'application du réseau de neurones	- 39 -
II.19	Avantages et Inconvénients du réseau de neurone	- 39 -
II.19.1	Avantage.....	- 39 -
II.19.2	Inconvénients	- 40 -
II.20	Réseau de neurones et Classification	- 40 -
II.20.1	Classification binaire.....	- 40 -
II.20.2	Classification Multi-classe	- 42 -
II.21	Réseaux de neurones convolutifs CNN.....	- 46 -
II.21.1	Présentation	- 46 -
II.21.2	Architecture de réseaux de neurone convolutifs	- 46 -
II.21.3	Quelques réseaux convolutifs célèbres	- 47 -
II.21.4	Avantages de CNN.....	- 48 -
II.21.5	Exemples de modèles de CNN.....	- 48 -
II.22	Conclusion.....	- 49 -
Chapitre III.	Implémentation du système RAPI.....	- 0 -
III.1	Introduction	- 1 -
III.2	Le réseau VGG-16	- 1 -
III.2.1	Définition	- 1 -
III.2.2	L'architecture de VGG-16	- 1 -
III.3	Environnement de développement	- 3 -
III.3.1	Aspect matériel	- 3 -
III.3.2	Aspect logiciel	- 5 -
III.4	Résultats d'implémentation.....	- 9 -
III.4.1	Description de la base de données	- 9 -
III.4.2	Détection de la plaque d'immatriculation.....	- 9 -
III.4.3	Implémentation sur Raspberry Pi 3.....	- 20 -
III.4.4	Résultats finaux.....	- 27 -

III.5 Conclusion.....	- 33 -
Conclusion générale.....	- 34 -
Bibliographie	- 35 -

LISTE DES FIGURES

Figure 1 Etapes d'un système RAPI	- 2 -
Figure I-1 Lettre « A » afficher comme un groupe de pixels	- 1 -
Figure I-2 Exemple de bruit d'image	- 3 -
Figure I-3 Images originales transformées en images en niveau de gris.....	- 5 -
Figure I-4 Système de reconnaissance des plaques d'immatriculation	- 13 -
Figure I-5 Images acquises à l'entrée de système	- 14 -
Figure I-6 Transformation de la plaque en niveau de gris.....	- 16 -
Figure I-7 Transformation de la plaque en binaire	- 16 -
Figure II-1 Processus d'apprentissage automatique	- 20 -
Figure II-2 Les types d'apprentissage automatique.....	- 22 -
Figure II-3 Exemple de chiffres manuscrits	- 22 -
Figure II-4 Le concept de L'apprentissage Profond et sa relation avec L'apprentissage Automatique	- 25 -
Figure II-5 Les réseaux de neurones sont organisés en couches constituées d'un ensemble de nœuds interconnectés [42]	- 27 -
Figure II-6 Comparaison de méthodes de catégorisation de véhicules de Machine Learning (gauche) et de Deep Learning (droite) [50]	- 28 -
Figure II-7 Anatomie d'un neurone biologique [52]	- 30 -
Figure II-8 Modèle du neurone formel de Mac Culloch et Pitts (avec biais) [56]	- 31 -
Figure II-9 Structure d'un neurone artificiel [57].....	- 32 -
Figure II-10 Structure et comportement d'un perceptron. (Les connexions des deux entrées x_1 et x_2 au neurone sont pondérées par les poids w_1 et w_2).....	- 35 -
Figure II-11 Schéma illustrant un perceptron multicouche avec deux couches cachées.....	- 36 -
Figure II-12 Une structure en couches de nœuds	- 37 -
Figure II-13 Les branches du réseau neuronal en fonction de l'architecture des couches [42]- 38 -	
Figure II-14 Problème de classification binaire	- 41 -
Figure II-15 Classification binaire des données d'apprentissage	- 41 -
Figure II-16 Réseau de neurones pour les données d'entraînement.....	- 42 -
Figure II-17 La Modification de symboles de la classe par un code numérique	- 42 -
Figure II-18 Données avec trois classes	- 43 -

Figure II-19 Réseau de neurone configuré pour trois classes.....	- 43 -
Figure II-20 Données d'apprentissage avec classification multi-classes	- 44 -
Figure II-21 Chaque nœud de sortie est maintenant mappé à un élément du vecteur de classe..	- 44 -
Figure II-22 Modification des classes dans un nouveau format 1 sur 3	- 45 -
Figure II-23 Carrés de cinq pixels sur cinq affichant cinq nombres de 1 à 5.....	- 45 -
Figure II-24 Le modèle de réseau neuronal pour ce nouvel ensemble de données	- 45 -
Figure II-25 Architecture standard d'un réseau de neurone convolutionnel [69]	- 47 -
Figure III-1 L'architecture du VGG-net.....	- 2 -
Figure III-2 Représentation 3D de l'architecture du VGG-16	- 2 -
Figure III-3 Le Raspberry Pi3	- 4 -
Figure III-4 Caméra Raspberry.....	- 5 -
Figure III-5 Architecture du réseau VGG-16 modifié.....	- 10 -
Figure III-6 Précision d'apprentissage du modèle VGG-16 modifié	- 16 -
Figure III-7 Détection et encadrement de la plaque d'immatriculation	- 18 -
Figure III-8 Segmentation de la plaque d'immatriculation par détection des contours	- 19 -
Figure III-9 Encadrement des chiffres détectés dans une plaque d'immatriculation.....	- 19 -
Figure III-10 L'organigramme de reconnaissance des caractères	- 21 -
Figure III-11 Les broches de GPIO Raspberry PI	- 22 -
Figure III-12 Les boutons de réglage du capteur de mouvement H-SR501C(PIR)	- 23 -
Figure III-13 Câblage du capteur de mouvement HC-SR501	- 23 -
Figure III-14 Connexion du capteur de mouvement PIR Raspberry Pi	- 24 -
Figure III-15 L'écran LCD 16 × 2.....	- 24 -
Figure III-16 Diagramme LCD-16x2-pin.....	- 26 -
Figure III-18 Interface LCD 16x2 avec Raspberry PI 3	- 27 -
Figure III-19 Résultats de détection des plaques d'immatriculation.....	- 28 -
Figure III-20 Résultats de l'extraction des plaques d'immatriculation	- 28 -
Figure III-21 Segmentation des plaques selon le contour	- 29 -
Figure III-22 Segmentation et encadrement des plaques d'immatriculation.....	- 29 -
Figure III-23 Résultats de la reconnaissance des caractères.....	- 30 -
Figure III-24 Début de traitement.....	- 32 -
Figure III-25 Résultat de traitement	- 32 -

LISTE DES TABLEAUX

Tableau 1 Les branches du réseau neuronal en fonction de l'architecture des couches.....	- 38 -
Tableau 2 Description de la base de données	- 9 -

LISTE DES ABREVIATION

ART : Adaptative Résonnance Théorie

CNN: Convolutional Neural Network

ConvNet: Convolution Network

CPU: Central Processor Unit

DL: Deep Learning

DNN: Deep Neural Network

FC: Fully- Connected

FVV : Fichier des Véhicules Volés

GIF : Graphics Interchange Format

GPU : Graphic Processor Unit

IA : Intelligence Artificielle

IRM : Imagerie par Résonnance Magnétique

JPEG : Joint Photographics Expert Group

LED: Light Emitting Diode

ML: Maching Learning

MLP: Multi-Layer Perception

OCR: Optical Character Recognition

OpenCV: Open Computer Vision

PIR: Passive Infra-Red

PMC: Perceptron Multicouche

PNG: Portable Network Graphics

RAPI : Reconnaissance Automatique des Plaques d'Immatriculation

RBF : Radial Basic Fonction

ReLu : Rectified Linear Units

RNA : Réseau de Neurone Artificiel

SNN: Shallow Neural Network

TIFF: Tagged Image File Format

INTRODUCTION GENERALE

La reconnaissance automatique de types de plaque d'immatriculation par la vision artificielle est un thème de recherche qui a été abordé depuis de nombreuses années et a connu une répandre l'adoption par de nombreuses agences dans le monde pour améliorer leur application, capacités de création et de sécurité. Cette technologie aide à recueillir des informations sur les véhicules ce qui réduit le travail manuel fastidieux et long. Et qui continue de susciter les travaux de la communauté scientifique jusqu'à ce jour. Relatant à chaque fois de nouvelles méthodes et approches pour rendre ces systèmes plus efficaces. Tout système de reconnaissance de plaque est confronté à deux problèmes majeurs : détecter l'objet en question et le reconnaître, la détection de l'objet étant une étape critique.

Un système de reconnaissance automatique des plaques d'immatriculation est une technologie qui trouve son essence dans ces 20 dernières années dans le développement des techniques de traitement d'image ainsi que dans les OCR (Optical Character Recognition en anglais). Généralement un système RAPI typique est devisé en quatre phases comme le montre dans **la figure 1** au-dessous :



Figure 1 Etapes d'un système RAPI

1. L'acquisition de l'image à partir d'une séquence vidéo et son envoi vers le système.
2. La détection et l'isolement de la plaque : c'est la phase la plus importante et la plus difficile ; elle détermine la rapidité et la robustesse du système. Plusieurs études et recherches sont consacrées à cette phase depuis plusieurs années ainsi différentes méthodes ont été proposées. On peut citer dans ce cas les caractéristiques d'une image numérique comme la résolution, la taille d'image les contours, La détection de l'emplacement de la plaque.
3. La segmentation de la plaque en caractères : la plaque une fois extraite subira un ensemble de traitements pour être segmenté en séparant les caractères. On propose ici une méthode facile et rapide pour l'exécuter.

Ce mémoire est organisé comme suit :

- **Le 1^{er} chapitre** illustre sur les notions de base sur le traitement d'images, nous allons faire une conception pour le système de reconnaissance automatique des plaques d'immatriculation dont le but est de décomposer le système voulu en un ensemble de modules à détailler, par la suite, quelques techniques système RAPI.
- **Le 2^{ème} chapitre** est à propos les concepts fondamentaux de L'intelligence artificielle, Deep Learning, Machine Learning, les réseaux de neurone et le réseau de neurone convolutifs et de leur fonctionnement et importance dans divers secteurs.
- **Le 3^{ème} chapitre** nous présentera la mise en œuvre du système RAPI. Pour atteindre cet objectif nous avons utilisé le réseau VGG-16 .L'implémentation de système RAPI fait par un Raspberry Pi.3 et la caméra Raspberry. Ainsi que certains outils de développement tel que les bibliothèques : OpenCV, Miniconda, Tensorflow et Keras et le langage de programmation Python3.7. Finalement on va présenter les résultats obtenus avec une discussion sur ces différents résultats.

Chapitre I. LES NOTIONS DE BASE SUR LE TRAITEMENT D'IMAGES ET LES TECHNIQUES DE SYSTEME DE RAPI

I.1 Introduction

Dans ce chapitre nous allons aborder les notions de base de traitement d'images numériques : caractéristiques, différents types et les formats d'une image numérique. A la fin de ce chapitre nous présentons le système de reconnaissance utilisé ensuite on va citer quelques techniques de système RAPI. Ainsi que l'ensemble des étapes de reconnaissance faites afin d'extraire les informations pertinentes aux plaques d'immatriculation.

I.2 Qu'est-ce qu'une image ?

Une image est obtenue par transformation d'une scène réelle par un capteur, donc c'est un signal 2D (x, y) , qui représente souvent une réalité 3D (x,y,z) . D'un point de vue mathématique, une image est une matrice de nombres représentant un signal, plusieurs outils permettent de manipuler ce signal. Une image contient plusieurs informations sémantiques, il faut en interpréter le contenu au-delà de la valeur des nombres.

Pour un ordinateur, une image est un ensemble de pixels. Un pixel, c'est un élément d'image (*picture element*) [2]. Il existe trois principaux types d'images :

1. les images en niveaux de gris, dont la valeur appartient à l'ensemble $\{0, 1, \dots, 255\}$ pour un codage sur 8 bits.
2. les images binaires (uniquement en noir et blanc) et dont la valeur égale soit 0 soit 1.
3. les images couleurs dont plusieurs représentations existent.

I.3 Représentation d'image

Les images numériques, destinées à être visualisées sur les écrans d'ordinateur, se divisent en 2 grandes classes : Les images matricielles et les images vectorielles [3].

I.3.1 Image matricielle

Une image matricielle (image bitmap), est formée d'un assemblage de points ou de pixels. On parle sur des points lorsque ces images sont imprimées ou destinées à l'impression (photographies, publicités, cartes ... etc.) et on parle sur des pixels pour les images stockées sous forme « binaire » ou « numérique ».

L'image est composée d'une matrice (tableau) de points à plusieurs dimensions, chaque dimension représentant une dimension spatiale (hauteur, largeur, profondeur, temporelle [durée]) ou autre (par exemple, un niveau de résolution) [4].

I.3.2 Image vectorielle

Une image vectorielle en informatique (ou image en mode trait), est une image numérique composée d'objets géométriques individuels, définis de manière mathématique, (segments de droite, polygones, arcs de cercle, ... etc.) définis chacun par divers attributs de forme, de position, de couleur, etc. Par exemple, une image vectorielle d'un cercle est définie par des attributs de types : position du centre, rayon, etc. Par nature, un dessin vectoriel est dessiné à nouveau à chaque visualisation, ce qui engendre des calculs sur la machine [4].

I.4 Caractéristiques d'une image numérique

L'image est un ensemble structuré d'informations caractérisé par les paramètres suivants :

I.4.1 Pixel

Le pixel est le plus petit point de l'image, c'est une entité calculable qui peut recevoir une structure et une quantification. Si le bit est la plus petite unité d'information que peut traiter un ordinateur, le pixel est le plus petit élément que peuvent manipuler les matériels et logiciels d'affichage ou d'impression. La quantité d'information que véhicule chaque pixel donne des nuances entre images monochromes et images couleurs. Dans le cas d'une image monochrome, chaque pixel est codé sur un octet, et la taille mémoire nécessaire pour afficher une telle image est directement liée à la taille de l'image. La lettre A, par exemple, peut être affichée comme un groupe de pixels dans la **figure I-1** :



Figure I-1 Lettre « A » affichée comme un groupe de pixels

Dans une image couleur (R.V.B.), un pixel peut être représenté sur trois octets : un octet pour chacune des couleurs : rouge (R), vert (V) et bleu (B) [5].

I.4.2 *Dimension*

C'est la taille de l'image. Cette dernière se présente sous forme de matrice dont les éléments sont des valeurs numériques représentatives des intensités lumineuses (pixels). Le nombre de lignes de cette matrice multiplié par le nombre de colonnes nous donne le nombre total de pixels dans une image [3].

I.4.3 *Résolution*

La résolution est définie par un nombre de pixels par unité de longueur de l'image à numériser en **dpi** (dots per inch) ou **ppp** (points par pouce). On parle de définition pour un écran et de résolution pour une image. La résolution d'une image correspond au niveau de détail qui va être représenté sur cette image. Pour la numérisation il faut considérer les 2 équations suivantes :

- $(X \times \text{résolution}) = x \text{ pixels}$
- $(Y \times \text{résolution}) = y \text{ pixels}$

Où :

- X et Y représentent la taille (pouce ou cm, un pouce=2,54 centimètres) de la structure à numériser.
- Résolution représente la résolution de numérisation,
- x et y représentent la taille (en pixels) de l'image [7].

I.4.4 *La taille d'une image*

Pour connaître la taille d'une image, il est nécessaire de compter le nombre de pixels que contient l'image, cela revient à calculer le nombre des cases du tableau, soit la hauteur de celui-ci que multiplie sa largeur. La taille de l'image est alors le nombre des pixels que multiplie la taille (en octet) de chacun de ces éléments [8].

Exemple : pour une image de 240×420 en vraie couleur (True Color) :

- Nombre de pixels : $240 \times 420 = 100800$
- Taille de chaque pixel : $24 \text{ bits} / 8 = 3 \text{ octets}$

- Le poids de l'image est ainsi égal à : $100800 \times 3 = 302.400$ égal $302.400/1024 = 295$ Ko.

I.4.5 Bruit

Un bruit (parasite) dans une image est considéré comme un phénomène de brusque variation de l'intensité d'un pixel par rapport à ses voisins, il provient de l'éclairage des dispositifs optiques et électroniques du capteur [9].



Figure I-2 Exemple de bruit d'image

I.4.6 Histogramme

L'histogramme des niveaux de gris ou des couleurs d'une image est une fonction qui donne la fréquence d'apparition de chaque niveau de gris (couleur) dans l'image. Pour diminuer l'erreur de quantification, pour comparer deux images obtenues sous des éclairages différents, ou encore pour mesurer certaines propriétés sur une image, on modifie souvent l'histogramme correspondant [10] [11]. Il permet de donner un grand nombre d'informations sur la distribution des niveaux de gris (couleur) et de voir entre quelles bornes est répartie la majorité des niveaux de gris (couleur) dans les cas d'une image trop claire ou d'une image trop foncée.

Il peut être utilisé pour améliorer la qualité d'une image (Rehaussement d'image) en introduisant quelques modifications, pour pouvoir extraire les informations utiles de celle-ci.

I.4.7 Contours et textures

Les contours représentent la frontière entre les objets de l'image, ou la limite entre deux pixels dont les niveaux de gris représentent une différence significative. Les textures décrivent

la structure de ceux-ci. L'extraction de contour consiste à identifier dans l'image les points qui séparent deux textures différentes [12].

I.4.8 Luminance

C'est le degré de luminosité des points de l'image. Elle est définie aussi comme étant le quotient de l'intensité lumineuse d'une surface par l'aire apparente de cette surface, pour un observateur lointain, le mot luminance est substitué au mot brillance, qui correspond à l'éclat d'un objet. Une bonne luminance se caractérise par : [10]

1. Des images lumineuses : brillantes.
2. Un bon contraste : il faut éviter les images où la gamme de contraste tend vers le blanc ou le noir ; ces images entraînent des pertes de détails dans les zones sombres ou lumineuses.
3. L'absence de parasites.

I.4.9 Contraste

C'est l'opposition marquée entre deux régions d'une image, plus précisément entre les régions sombres et les régions claires de cette image. Le contraste est défini en fonction des luminances de deux zones d'images.

Si L_1 et L_2 sont les degrés de luminosité respectivement de deux zones voisines A_1 et A_2 d'une image, le contraste C est défini par le rapport : [13]

$$C = \frac{L_1 - L_2}{L_1 + L_2} \quad (1.1)$$

I.4.10 Images à niveaux de gris

Le niveau de gris est la valeur de l'intensité lumineuse en un point. La couleur du pixel peut prendre des valeurs allant du noir au blanc en passant par un nombre fini de niveaux intermédiaires. Donc pour représenter les images à niveaux de gris (**Figure I-3**), on peut attribuer à chaque pixel de l'image une valeur correspondant à la quantité de lumière renvoyée. Cette valeur peut être comprise par exemple entre 0 et 255. Chaque pixel n'est donc plus représenté par un bit, mais par un octet. Pour cela, il faut que le matériel utilisé pour afficher l'image soit capable de produire les différents niveaux de gris correspondant [6].

Le nombre de niveaux de gris dépend du nombre de bits utilisés pour décrire la "couleur" de chaque pixel de l'image. Plus ce nombre est important, plus les niveaux possibles sont nombreux.



Figure I-3 Images originales transformées en images en niveau de gris

I.4.11 Images en couleurs

Même s'il est parfois utile de pouvoir représenter des images en noir et blanc, les applications multimédias utilisent le plus souvent des images en couleurs. La représentation des couleurs s'effectue de la même manière que les images monochromes avec cependant quelques particularités. En effet, il faut tout d'abord choisir un modèle de représentation. On peut représenter les couleurs à l'aide de leurs composantes primaires. Les systèmes émettant de la lumière (écrans d'ordinateurs,...) sont basés sur le principe de la synthèse additive : les couleurs sont composées d'un mélange de rouge, vert et bleu (modèle R.V.B.)[6].

I.5 Différents types d'images

- Imagerie vidéo (vision industrielle, vidéo surveillance).
- Imagerie thermique (thermographie).
- Imagerie d'écho (radar, sonar, échographe, doppler).
- Imagerie à rayons X (radiologie, scanner).
- Imagerie radioactive (tomographie).
- Imagerie IRM (résonance magnétique).
- Imagerie satellitaire.
- Imagerie multi-spectrale et imagerie couleur.

- Imagerie polarimétrie (permet de révéler des contrastes invisibles à l'œil humain et aux caméras classiques). [14]

I.6 Formats d'images numériques

Un format d'image numérique est une représentation informatique de l'image. Il comprend généralement un en-tête qui contient des informations sur l'image (Par exemple : taille de l'image en pixels, informations sur la façon dont l'image est codée et fournissant éventuellement des indications sur la manière pour la décoder et la manipuler) suivie des données de l'image.

Le format influence la qualité de l'image en fonction du poids (en **Ko**, voire en **Mo**). Plus il sera élevé selon le format choisi plus la qualité de l'image changera [15].

I.6.1 Format GIF (Graphics Interchange Format)

Ce format permet la transparence où les images animées sont plusieurs images séquentielles à l'intérieur du même fichier. Il est utilisé pour des logos, des icônes, des boutons et autres éléments de pages web.

Le format d'image GIF n'atteigne au maximum que 256 couleurs, il n'est donc pas du tout adapté aux photos et à l'impression [15].

I.6.2 Format TIFF (Tagged Image File Format)

Un des formats le plus couramment utilisé pour stocker des images, des photographies. Conçu au départ pour n'accepter que les images en RGB, il permet de coder des images CMJN. Une image CMJN enregistrée en format TIFF et placée dans un logiciel de PAO peut être envoyée à l'impression sans perte de qualité. Il est couramment utilisé dans les environnements professionnels et pour l'impression commerciale. Il est considéré comme étant le format le plus fiable pour des impressions de haute qualité comme pour le textile, les tissus, etc [15].

I.6.3 Format JPG / JPEG (Joint Photographic Experts Group)

C'est le format le plus adéquat pour administrer les photos et les publier. Un des formats les plus utilisées sur le net (les navigateurs l'affichent correctement), et dans les mails. Les appareils photo numériques compacts prennent également les photos au format JPG [15].

I.6.4 *Format PNG (Portable Network Graphics)*

Un des formats le plus couramment utilisé. Créé pour remplacer le GIF il est très peu connu du grand public. Performant, il réunit presque tous les avantages du JPEG et du GIF et permet les fonds transparents. La compression proposée par ce format sans perte de qualité est 5 à 25 % meilleure que la compression GIF [15].

I.7 *Traitement d'images*

C'est une discipline primordiale en informatique mathématiques appliquées qui étudient les images numériques et leurs transformations. Il vise à améliorer la qualité de l'image ou à extraire des informations dans le but d'améliorer sa qualité ou d'en extraire de l'information (Ex : la détection de lettres ou de visages, l'identification de zones cancéreuses en imagerie médicale, la compression JPEG, etc.) [16]. Le mode et les conditions d'acquisition et de numérisation des images traitées conditionnent largement les opérations qu'il faudra réaliser pour extraire de l'information.

I.8 *Principale étapes de traitement d'images*

Il n'existe pas de méthode de traitement d'images générale à tous les domaines d'application possibles. Il faut en général employer des algorithmes spécifiques. Ces derniers sont souvent des combinaisons de techniques classiques (segmentation, classification, reconnaissance de frontières, etc.). De manière schématique, toute méthode de traitement d'images comprend trois étapes majeures [17] :

1. Prétraitement des images.
2. Amélioration des images.
3. Analyse des images.

I.8.1 *Prétraitements des images*

Ils préparent l'image pour son analyse ultérieure. Il s'agit souvent d'obtenir l'image théorique que l'on aurait dû acquérir en l'absence de toute dégradation. Ainsi, ils peuvent par exemple corriger :

- Les défauts radiométriques du capteur : non linéarité des détecteurs, diffraction de l'optique, etc.

- Les défauts géométriques de l'image dus au mode d'échantillonnage spatial, à l'obliquité de la direction de visée, au déplacement de la cible, etc.
- Le filtrage ou réduction de fréquences parasites, par exemple dus à des vibrations du capteur.
- Les dégradations de l'image dues à la présence de matière entre le capteur et le milieu observé.

I.8.2 Amélioration d'image

Elle a pour but d'améliorer la visualisation des images. Pour cela, elle élimine/réduit le bruit de l'image et/ou met en évidence certains éléments (frontières, etc.) de l'image. Elle est souvent appliquée sans connaissance à priori des éléments de l'image. Les principales techniques sont :

1. L'amélioration de contraste : avant l'étape de segmentation, l'image d'entrée doit être transformée en une image niveau de gris, puis elle doit subir un prétraitement constitué d'une amélioration du contraste de l'image.
2. Le filtrage linéaire (lissage, mise en évidence des frontières avec l'opérateur "Image -Image lissée", etc.) et transformée de Fourier pour faire apparaître/disparaître certaines fréquences dans l'image.
3. Filtrage non linéaire (filtres médians, etc.) pour éliminer le bruit sans trop affecter les frontières, etc.

I.8.3 Analyse d'image

Le but de l'analyse d'images est d'extraire et de mesurer certaine caractéristique de l'image traitée en vue de son interprétation. Ces caractéristiques peuvent être des données statistiques sur des comptes numériques (moyenne, histogramme, etc.), ou sur des données dérivées (ex. dimensions, ou orientation d'objets présents dans l'image). En général, le type d'information recherché dépend du niveau de connaissance requis pour interpréter l'image. Les applications dans le domaine du guidage et de la télédétection nécessitent souvent des connaissances différentes et de plus haut niveau (ex., cartes 3-D) que dans les domaines du médical, de la géologie, du contrôle de qualité, etc. Ainsi, un robot en déplacement ne nécessite pas le même type d'information qu'un système utilisé pour détecter la présence de matériaux défectueux. Pour ce dernier, il faut prendre uniquement la décision "non défectueux ou défectueux". Cette décision peut être prise à partir d'un "raisonnement" plus ou moins complexe (ex. détection de la présence de raies d'absorption caractéristiques d'une impureté). Pour le

robot, il faut simuler le processus décisionnel d'un individu en déplacement, ce qui nécessite au préalable de reconstituer une carte du lieu de déplacement en temps réel, avec les obstacles à éviter.

I.9 Système de reconnaissance automatique des plaques d'immatriculation

Les systèmes RAPI capturent une image de la plaque minéralogique d'un véhicule et extraient son numéro d'immatriculation. Le numéro extrait de la plaque est transformé en caractères alphanumériques pour la reconnaissance. Ces caractères alphanumériques sont comparés à une ou plusieurs bases de données pour identifier le numéro de la plaque. Un système RAPI typique utilise cinq étapes qui doivent être réalisées pour que le logiciel puisse identifier une plaque d'immatriculation [18] :

1. Localisation de la plaque : identifie et isole la plaque d'immatriculation de l'image capturée par la caméra.
2. Orientation et redimensionnement de la plaque : redresse et redimensionne l'image selon les besoins. Parce que la caméra capture l'image sous un angle, les images de la plaque d'immatriculation sont quadrilatère, non rectangulaire. Le processus de redressement aide à l'inclinaison, cisaillement et corrections de l'image capturée.
3. Normalisation de l'image : modifie les valeurs d'intensité de l'image pour qu'elles correspondent caractéristiques d'une image de référence. Certaines des approches comprennent la mise à l'échelle et « Égalisation » d'histogramme.
4. Segmentation des caractères : extrait les caractères alphanumériques de la plaque.
5. Reconnaissance des caractères : reconnaît le numéro d'immatriculation du véhicule.

I.9.1 Les difficultés rencontrées par RAPI

Les systèmes RAPI efficaces doivent surmonter un certain nombre de difficultés, à savoir : [18]

1. Une mauvaise résolution de l'image à cause d'une plaque trop éloignée ou d'une caméra de mauvaise qualité.

2. Mauvais éclairage et faible contraste dus à la surexposition, réflexions et des ombres, un objet assombrissant une partie de la plaque (barre de remorquage ou de la poussière).
3. Flou dû à la vitesse du véhicule ou à cause du mouvement.
4. Différents formats de conceptions de plaques, qui diffèrent d'un état à un autre.
5. Une police de caractère trop originale comme dans certains pays asiatiques.

Certaines difficultés peuvent être surmontées dans certains logiciels, tandis que d'autres nécessitent du matériel à surmonter, d'autres peuvent être évités en normalisant le format de la plaque d'immatriculation.

1.9.2 Les techniques de reconnaissance des plaques d'immatriculation

Dans leur version la plus simple, les systèmes de RAPI utilisent, en cascade, deux techniques de traitements d'images [19] : la détection d'objet pour repérer les plaques d'immatriculation potentielles suivie de la reconnaissance optique des caractères afin d'identifier les caractères alphanumériques de la plaque.

Pour obtenir des résultats satisfaisants, ce sont des séries de techniques de traitement d'image qui sont appliquées afin de détecter, normaliser et agrandir l'image de la plaque d'immatriculation, et enfin de pouvoir réaliser la reconnaissance optique de caractères (OCR) pour extraire les caractères alphanumériques de la plaque.

La fiabilité et la précision d'un système RAPI dépendent de la complexité de chacune de ces étapes. Dans quelques cas, la RAPI est couplée à une phase de détection/reconnaissance de véhicule afin de renforcer les performances de la détection de plaque. Afin d'accélérer et d'améliorer les performances de la phase de détection des plaques, une technique courante est d'utiliser à priori les zones potentielles de détection et de réduire la recherche à celles-ci. Ces zones sont déterminées par l'analyse des localisations des détections précédentes. Cette technique est appliquée dans le cadre des caméras ayant une prise de vue large (plusieurs voies). L'application de cette technique permet d'accélérer les traitements et donc d'augmenter le nombre de plaque lisible à la seconde. Cela est utile dans le cas où les dispositifs doivent travailler dans un environnement proche du temps réel tout en ayant une puissance de calcul modérée. Toutefois, elles s'exposent également au risque de ne pas détecter les véhicules qui ne passent pas dans les zones de pré-détection. C'est pourquoi elles ne sont pas appliquées dans le cas des dispositifs qui nécessitent d'identifier toutes les plaques à des fins de sécurité. Dans

ces cas-là, soit le traitement est fait off-line, en temps différé, ou bien en temps réel mais avec un système plus coûteux car nécessitant une puissance de calcul élevée.

Nous avons ici un ensemble de méthodes qui ont été proposées pour la détection et la reconnaissance des plaques ; généralement on peut classer ces méthodes en quatre catégories [20] :

1.9.2.a *Techniques basées sur les propriétés de la plaque*

Pour *Anagnostopoulos* [21], il utilise la couleur de la plaque comme une propriété déterminante, l'image passe par un filtre de couleur et la sortie est comparée à la forme de la plaque. *Wang* et al [22], utilisent un filtre spécial en plus du filtre de couleur. Ces filtres sont appliqués à une image au niveau de gris et la sortie contiendra les formes des caractères. Toutes les régions sont testées pour en extraire celles susceptibles d'être la plaque. Pour *Beatriz* et *Broumandnia* [23] et [24], ils appliquent une méthode qui travaille avec des seuils afin de segmenter l'image en noir et blanc pour ensuite en extraire la plaque. *Comeli* et al [25], utilisaient un test se basant sur la taille des caractères et la distance qui existe entre eux. Ils ont eu jusqu'à 91.07% de réussite.

Les algorithmes basés sur les couleurs sont moins efficaces face à un changement d'éclairage car les couleurs apparaissent différemment sous différents éclairages et en plus les plaques ont plusieurs couleurs et plusieurs formes. Avec ce type de méthodes on n'est pas sûre que les régions détectées aillent correspondre à la plaque.

1.9.2.b *Techniques basées sur les contours de la plaque*

Pour détecter la plaque, certains algorithmes utilisent les contours des caractères et de la plaque comme des points de référence pour l'extraction. L'intensité des pixels dans les caractères et dans les contours des plaques est complètement différente que celle des voisins. En effet dans la majorité des cas ils sont d'une couleur différente que celle du reste de l'image. Une arête peut être détectée dans une image en utilisant le gradient comme par exemple avec le seuil OTSU. La seconde partie de la détection localise les contours de la plaque à partir d'une image en noir et blanc.

Pour *Dai* et al, *David* et al [26] et [27], ils ont utilisé des opérations morphologiques sur l'image transformée en contours. Pour *Dlagnekovet Elliman* [28] et [29], une transformée de Hough est appliquée sur l'image, et les lignes qui traversent la plaque seront déterminées ainsi que les objets de formes rectangulaires. *Anagnostopoulos* [30] a développé une méthode qui

décrit les irrégularités dans la plaque. Cette méthode ressemble à la méthode précédente parce qu'elle dépend de l'intensité des pixels.

Les algorithmes présentés sont défaillants lorsque les bords de la plaque ne présentent pas une grande variation par rapport au reste de l'image en plus ces algorithmes utilisent un seuil qui doit être déterminé automatiquement ce qui est difficile sous différentes conditions d'éclairage.

1.9.2.c Techniques basées sur l'intelligence artificielle

Nijhuis et al [31] ont utilisé les caractéristiques de la plaque allemande qui se compose d'un arrière-plan jaune pour créer une fonction pour chaque pixel à travers un histogramme. Les autres propriétés floues sont dégagées à partir du niveau de gris de chaque pixel et de ses voisins. *Zimic* et al [32] ont appliqué le même concept de la logique floue. Ils ont divisé l'image en un ensemble de rectangles de la taille de la plaque. Les auteurs de *Matas* [33] ont réussi à localiser la plaque en utilisant la logique floue avec un pourcentage de réussite de 97.9%. Néanmoins cette méthode dépend de l'éclairage parce qu'elle est tributaire de la couleur de la plaque. *Park* et al [34] ont utilisé un réseau de neurones à deux entrées comme un filtre horizontal et vertical pour détecter les plaques coréennes. L'intersection entre les deux filtres localise la région de la plaque. *Chacon* et *Zimmermann* [35] ont utilisé un réseau de neurones pour déterminer les régions susceptibles d'être la plaque qui seront analysées par la suite par une transformé de Fourier pour détecter la bonne région, l'algorithme sera répété jusqu'à la détermination de la plaque. Ils ont réussi à extraire 85% des plaques ainsi la réussite de ce type d'algorithme dépend de leur capacité à s'adapter aux différents éclairages car l'entrée des réseaux est la valeur de niveau de gris pixel.

1.9.2.d Techniques basées sur la signature de la plaque

*Dlagneko*va [36] implémente l'algorithme d'Adaboost comme il est utilisé pour la détection des visages (*Viola and Jones*). *Matlas* et *Zemmerman* [37] proposent un algorithme qui choisit une région du texte à partir d'un ensemble puis il exploite le fait qu'une plaque contient des caractères et des symboles qui sont clairement visibles même lorsque plusieurs propriétés de la plaque sont cachées. Cependant le problème est que certaines autres régions outre que plaque peuvent contenir du texte. *Elliman* et *Lancoster* [38] ont fait évoluer l'utilisation du spectre de Fourier pour distinguer les différentes régions de l'image présentant des informations. *Broumandnia* et al ont parcouru l'image verticalement et ont distingué le nombre d'arêtes pour chaque parcours pour localiser la ligne horizontale à laquelle est lié la plaque.

I.10 Etapes de système RAPI

La nécessité de reconnaître les véhicules et leurs plaques d'immatriculation a donné lieu à de nombreux systèmes de reconnaissance. Dans ce projet on va présenter un système de reconnaissance des plaques d'immatriculation basé sur les étapes (**Figure I-4**) comme méthode d'extraction des caractéristiques et les fonctionnalités [39]. Ce système comprend trois thèmes principaux qui sont : une phase d'acquisition d'image et une phase de prétraitements qui inclue une étape de détection de la plaque ainsi qu'une étape de segmentation, la troisième phase représente l'étape d'extraction des attributs qui est l'étape de la reconnaissance en utilisant des algorithmes de classification supervisée.

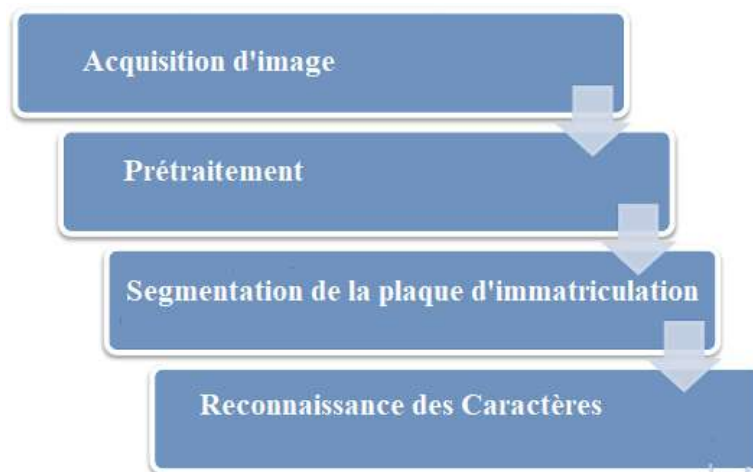


Figure I-4 Système de reconnaissance des plaques d'immatriculation

I.10.1 Acquisition d'une image

L'acquisition d'images constitue un des maillons essentiels de toute chaîne de conception et de production d'images. Pour pouvoir manipuler une image sur un système informatique, il est avant tout nécessaire de lui faire subir une transformation qui la rendra lisible et manipulable par ce système. Le passage de cet objet externe l'image d'origine (**Figure I-5**) à sa représentation interne (dans l'unité de traitement) se fait grâce à une procédure de numérisation. Ces systèmes de saisie, dénommés optiques, peuvent être classés en deux catégories principales : Les caméras numériques et les scanners [6].



Figure I-5 Images acquises à l'entrée de système

I.10.2 Prétraitement

Cette étape consiste à traiter et à préparer une image nécessaire à la détection ultérieure de la plaque d'immatriculation et à la reconnaissance des caractères [39].

I.10.2.a Localisation de la plaque d'immatriculation

La détection de l'emplacement de la plaque d'immatriculation est une étape critique dans les systèmes de reconnaissance des plaques d'immatriculation, puisqu'il est difficile de détecter la plaque rapidement, et précisément pour des images avec des fonds compliqués et des conditions lumineuses variées. Ce qui nous donne un domaine de recherche vaste dans lequel plusieurs chercheurs ont proposé des méthodes de détection, qui varient dans leur concept et dans leurs résultats. Il y'a quatre méthodes de localisation de plaque couramment utilisés : Le premier type est basé sur la couleur de l'information (c.à.d. la plaque), le deuxième type est basé sur la méthode de la recherche de la frontière, le troisième type est basé sur la morphologie mathématique, et finalement le quatrième type basé sur les réseaux de neurones. Tous ces algorithmes peuvent donner des bons résultats concernant le positionnement de la plaque pour certains cas, mais ils ont aussi des limites [41].

I.10.3 Segmentation des plaque d'immatriculation

Une fois que la plaque d'immatriculation extraite de l'image, la troisième étape d'un système RAPI est la segmentation de la plaque d'immatriculation. Elle consiste à extraire de l'image de la plaque d'immatriculation chacun des caractères qui y figurent. À cette fin, tout d'abord, une étape de prétraitement est généralement effectuée sur l'image afin d'améliorer sa qualité et de faciliter ainsi l'extraction des caractères. Par exemple, un problème couramment traité dans cette étape de prétraitement est la correction d'inclinaison. En général, l'amélioration de la qualité de l'image de plaque d'immatriculation extraite est un aspect essentiel pour réussir

la segmentation de la plaque d'immatriculation. Ensuite, une fois l'étape de prétraitement terminée, la segmentation appropriée de la plaque d'immatriculation est effectuée [39].

En analyse d'image, ce qui nous intéresse, ce sont les objets des images. Un objet peut être défini comme une partie sémantiquement cohérente dans l'image. La segmentation est une partition d'une image en un nombre fini de régions $R_1, R_2, \dots, R_j, \dots, R_s$ telle que :

$$R = \bigcup_{i=1}^s R_i \quad R_i \cap R_j \neq \varnothing \quad i \neq j \quad (1.2)$$

Les méthodes de segmentation doivent faire correspondre ces régions à des objets. On peut distinguer deux types de segmentation : la segmentation par régions et la segmentation par contours. La segmentation concerne les étapes suivantes :

I.10.3.a *La segmentation basée sur la reconnaissance*

Ce système cherche les images qui correspondent à des classes dans son alphabet. Les méthodes de ce type de segmentation ne sont basées sur aucune dissection mais au contraire, l'image est divisée systématiquement en plusieurs morceaux qui se chevauchent sans tenir compte de contenu, ils sont classés dans le cadre d'une tentative de trouver un résultat (segmentation/reconnaissance) cohérent.

De plus les méthodes de ce type de segmentation appelé aussi « Segmentation-libre » ont comme principe l'utilisation d'une fenêtre mobile avec une largeur variable pour fournir des séquences de segmentations indicatives qui sont confirmées (ou non) par une reconnaissance.

I.10.3.b *La segmentation basée sur la plaque en caractères*

L'algorithme de segmentation est assez simple ; en effet si les caractères sont en noir et l'arrière-plan est en blanc, on suppose alors que les caractères sont séparés par des lignes de pixels noirs. On trace ses lignes par une couleur différente.

La plaque une fois détectée, subira un ensemble de traitements dans le but de la segmenter en caractères isolés et cela selon les étapes suivantes :

1. Conversion de l'image originale en niveau de gris.
2. Conversion de l'image résultante en une image binaire.

I.10.3.b.1.1 Transformation de la plaque en niveau de gris

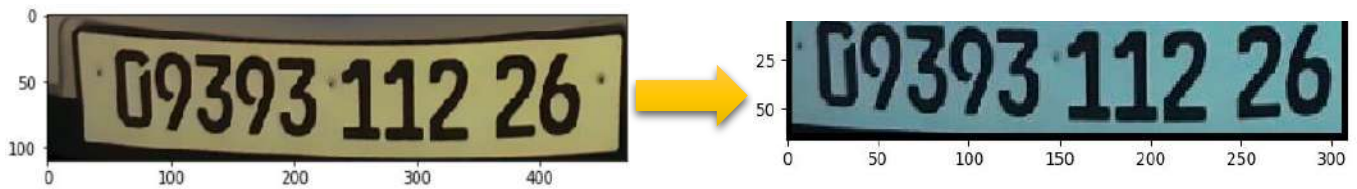


Figure I-6 Transformation de la plaque en niveau de gris

I.10.3.b.1.2 Binarisation de la plaque

Dans cette partie, l'image sera transformée en noire et blanc. Pour ce faire, nous avons fixé un seuil s_1 . Si on considère $f(i)$ comme la valeur du pixel i en niveau de gris, alors on pose la condition suivante :

Si $f(i) > s_1$ **alors** $f(i) = 0$;

Sinon $f(i) = 255$;

On obtient les résultats suivants :



Figure I-7 Transformation de la plaque en binaire

I.10.4 Reconnaissance de caractères

La dernière étape de chaque système RAPI est la reconnaissance des caractères de la plaque d'immatriculation précédemment extraits. À ce stade, de nouveaux problèmes apparaissent, tels que la taille et l'épaisseur des caractères dues aux facteurs de zoom, différentes polices de caractères pour différents pays, des caractères bruités ou brisés.

Ces systèmes de reconnaissance des plaques d'immatriculation peuvent être utilisés dans plusieurs domaines comme l'identification automatique des véhicules dans les parkings, le contrôle d'accès des véhicules dans une zone restreinte, la détection et la vérification des véhicules volés [40]. Certains systèmes utilisés pour la RAPI, notamment le capteur vidéo, peuvent être utilisés pour d'autres applications comme [20] : la surveillance, le recueil de données (débit, vitesse, taux d'occupation), et gestion dynamique du trafic (régulation d'accès, régulation de carrefours à feux ...), la surveillance de sites et de zones, les passages de frontière, les stations-service (enregistrement quand un client part sans payer), le contrôle des entrées et sorties de parkings, de péages, ouverture automatique, la lutte contre criminalité en comparant

les plaques d'immatriculations au Fichier des Véhicules Volés (FVV) ou à la liste des propriétaires de véhicules suspects, la traçabilité des parcours individuels afin d'alimenter les modèles de trafic.

Pour certaines applications, le système peut être associé à d'autres algorithmes de reconnaissance de type facial ou de véhicule. Plusieurs solutions de gestion de trafic ont déjà été éprouvées à l'étranger. Cette liste, non exhaustive, permet d'illustrer la variété des applications en lien avec les caméras vidéo.

I.11 Conclusion

La représentation des images numériques est l'un des éléments essentiels des applications multimédias, comme dans la plupart des systèmes de communication. Nous avons introduit dans ce chapitre les notions de base qui représentent l'image et ses caractéristiques puis les différents types et le principe étapes de traitement d'images et leurs formats. Par la suite nous avons fait un tour sur le système automatique des plaques d'immatriculation et leurs techniques de reconnaissance avec une présentation de quelques étapes de reconnaissance et le domaine d'utilisation de système RAPI.

A l'issue de cette étude, nous avons choisi le Deep Learning comme approche pour la reconnaissance des plaques d'immatriculation. Dans le chapitre suivant, nous allons présenter les généralités de l'intelligence artificielle, Machine Learning et le Deep Learning et les réseaux de neurones.

Chapitre II. LES CONCEPTS FONDAMENTAUX DE L'INTELLIGENCE
ARTIFICIELLE, DEEP LEARNING, MACHING LEARNING ET
RESEAUX DE NEURONE

II.1 Introduction

Le Machine Learning est une approche inductive qui dérive un modèle des données. Il est utile pour la reconnaissance d'images, la reconnaissance vocale et le traitement du langage naturel, etc. Le Deep Learning (DL) est l'une des raisons qui ont conduit les récentes avancées et progrès de l'IA, et c'est la principale cause qui fait penser qu'il existe une possibilité pour l'intelligence artificielle (IA) de devenir plus réaliste. A présent que nous avons précisé comment fonctionnent les réseaux de neurones de manière générale, nous allons aborder le domaine du Deep Learning.

II.2 L'intelligence artificielle

En général, **l'intelligence artificielle**, l'apprentissage automatique et l'apprentissage profond sont liés comme suit : « L'apprentissage profond est une sorte de l'apprentissage automatique, et l'apprentissage automatique est une sorte d'intelligence artificielle. »

Le fait que l'apprentissage profond soit un type de l'apprentissage automatique est très important, et c'est pourquoi nous passons dans ce chapitre sur la façon dont l'Intelligence Artificielle (IA), l'apprentissage profond et l'apprentissage automatique sont liés. L'apprentissage profond a été à l'honneur récemment car il a résolu efficacement certains problèmes qui ont défié l'intelligence artificielle. Ses performances sont sûrement exceptionnelles dans de nombreux domaines. Cependant, il fait également face à des limites. Les limites de l'apprentissage en profond découlent de ses concepts fondamentaux qui ont été hérités de son ancêtre, l'apprentissage automatique. En tant que type de l'apprentissage automatique, l'apprentissage profond ne peut pas éviter les problèmes fondamentaux auxquels le Machine Learning est confronté [44]. C'est pourquoi que nous devons revoir l'apprentissage automatique avant de discuter du concept de l'apprentissage profond.

II.3 Qu'est-ce que le Machine Learning ?

L'apprentissage automatique (le Machine Learning) est une technique de modélisation qui implique des données. Cette la définition peut être trop courte pour les débutants pour saisir ce que cela signifie. Alors laisse-moi élaborer un peu sur cela. L'apprentissage automatique est une technique qui permet de comprendre le « modèle » à partir des « données ». Ici, les données

signifient littéralement des informations telles que documents, audio, images, etc. Le «modèle» est le produit final de l'apprentissage automatique [42].

Dans les premiers temps des applications dites « intelligentes », de nombreux systèmes utilisaient des règles figées dans le code, basées sur des décisions de type « si... sinon... » pour traiter les données ou encore ajuster les entrées de l'utilisateur. Soit par exemple un filtre anti-spam, son travail consiste à déplacer des messages électroniques entrants dans un dossier spécifique. Vous pourriez créer une liste noire de termes qui ferait en sorte qu'un message soit marqué comme étant un spam. Il s'agirait d'un exemple typique de système de règles « expert » servant à créer une application « intelligente ». De telles règles décisionnelles, ciselées à la main, sont tout à fait applicables dans le cas de certaines applications, en particulier celles pour lesquelles les êtres humains ont une bonne compréhension des processus à modéliser. Cependant, cette façon de procéder présente deux inconvénients majeurs : [43]

- La logique nécessaire à la prise de décision est spécifique à un certain domaine et une certaine tâche. Modifier celle-ci, même légèrement, peut impliquer la réécriture de l'ensemble du système.
- Concevoir des règles nécessite une compréhension approfondie de la manière dont une décision devrait être prise par un expert humain.

La Figure II-1 illustre ce qui se passe dans le processus d'apprentissage automatique.

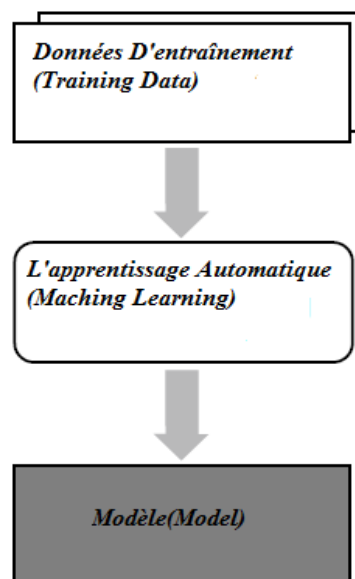


Figure II-1 Processus d'apprentissage automatique

II.4 Les problèmes que le Maching Learning peut résoudre

Les types les plus efficaces d'algorithmes d'apprentissage automatique sont ceux qui automatisent des processus de prise de décision en généralisant des données fournies à partir d'exemples connus. Dans ce processus, également appelé apprentissage supervisé, l'utilisateur alimente en quelque sorte l'algorithme en lui fournissant des paires entrée/sortie désirées, et cet algorithme trouve ensuite une manière de produire la sortie désirée à partir d'une certaine entrée. En particulier, l'algorithme est capable de créer une sortie depuis une entrée qu'il n'a jamais rencontrée auparavant, et ce sans aucune aide humaine. Revenons à notre exemple de classification des spams. Avec un système d'apprentissage automatique, l'utilisateur va fournir à l'algorithme un grand nombre de messages électroniques (qui constituent donc l'entrée) ainsi que des informations indiquant quels messages sont des spams, et lesquels n'en sont pas (ce qui forme la sortie désirée). Par la suite, cet algorithme sera capable de produire une prédiction sur le fait qu'un nouvel email est un spam ou non.

Les algorithmes qui apprennent à partir de paires entrée/sortie sont dits supervisés, car les choses se déroulent comme si un « professeur » supervisait l'apprentissage automatique en expliquant le lien entre entrées et sorties désirées. Alors que la création des jeux de données pour les entrées et les sorties est souvent un processus manuel très laborieux. Les algorithmes d'apprentissage automatique supervisé sont bien compris et leurs performances sont faciles à mesurer. [45]

II.5 Les types du Maching Learning

De nombreux types de techniques d'apprentissage automatique ont été développés pour résoudre des problèmes dans divers domaines. Ces techniques d'apprentissage automatique peuvent être classées en cinq types en fonction de la méthode d'apprentissage (**Figure II-2**).

- Apprentissage supervisé
- Apprentissage non supervisé
- Apprentissage semi supervisé
- Apprentissage par transfert
- Apprentissage par renforcement

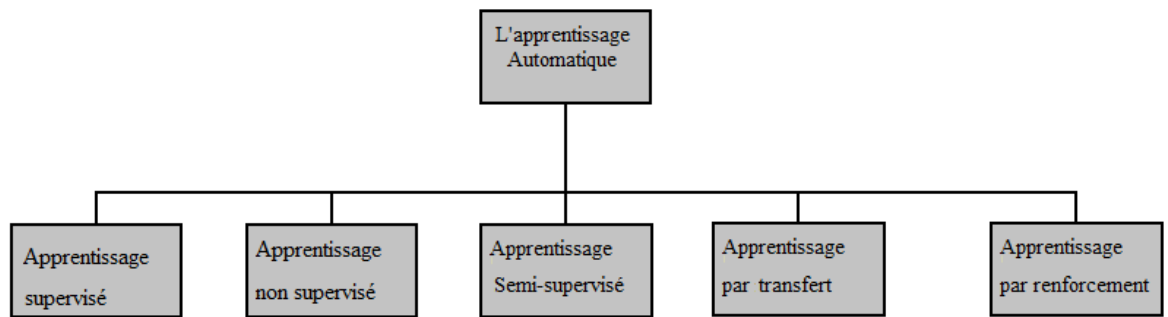


Figure II-2 Les types d'apprentissage automatique

1. **L'apprentissage supervisé** est très similaire au processus dans lequel un humain apprend des choses. Dans l'apprentissage supervisé, chaque ensemble de données d'apprentissage doit consister en paires d'entrée/sortie correspondantes. La sortie correcte est ce que le modèle est censé produire pour l'entrée donnée.

Voici quelques exemples **d'applications de l'apprentissage supervisé** :

- Identifier un code postal parmi des chiffres écrits à la main sur une enveloppe. Ici, l'entrée est une numérisation de l'écriture manuscrite (un exemple est illustré sur la **Figure II-3**), et la sortie désirée est une suite de chiffres formant un code postal. Pour créer un jeu de données adapté à un tel algorithme, il est nécessaire de numériser un grand nombre d'enveloppes. Vous pouvez alors lire vous-même les codes, et enregistrer les chiffres correspondants en tant que sorties désirées.

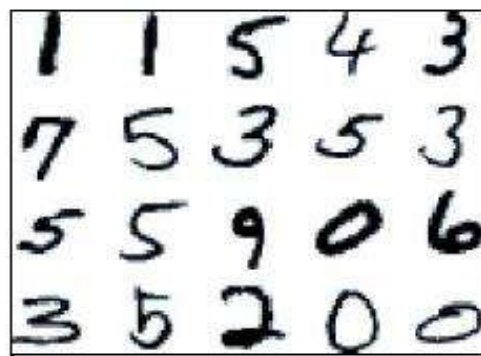


Figure II-3 Exemple de chiffres manuscrits

- Déterminer si une tumeur est bénigne à partir d'une image médicale. Ici, l'entrée est l'image, et la sortie doit indiquer si la tumeur est bénigne. Pour créer un jeu de données afin de bâtir ce modèle, vous devez disposer d'une base de données formée d'images médicales. Vous avez également besoin de l'avis d'un expert, et donc un docteur doit consulter toutes ces images et décider quelles tumeurs sont bénignes, et lesquelles ne le

sont pas. Il peut même être nécessaire de pousser le diagnostic plus loin que l'imagerie elle-même afin de déterminer si la tumeur visible sur une radio est ou non cancéreuse.

- Détecter une activité frauduleuse dans des transactions faites avec une carte de crédit. Ici, l'entrée est un enregistrement d'une transaction effectuée avec une carte de crédit, et la sortie doit indiquer si cette transaction est susceptible d'être frauduleuse ou non. En supposant que vous représentez l'entité qui fournit ces cartes de crédit, la constitution du jeu de données va consister à enregistrer toutes les transactions, et à marquer celles pour lesquelles un client a signalé une fraude. Chaque entité, ou ligne, formant un point de donnée destiné à être représentatif de l'ensemble s'appelle ici un échantillon. Les colonnes, de leur côté, donc les propriétés qui décrivent ces entités, sont appelées des caractéristiques (features en anglais), ou encore variables. [43]

2. L'apprentissage non supervisé : est généralement utilisé pour enquêter sur caractéristiques des données et prétraitement des données. Ce concept est similaire à un étudiant qui vient de trier les problèmes par construction et attribuer et n'apprend pas à les résoudre car il n'existe pas de sortie correcte connue.

Voici quelques exemples **d'application de l'apprentissage non supervisé :**

- Identifier des sujets dans des postes sur un blog ou un forum. Si vous disposez d'une vaste collection de données textuelles, vous pouvez souhaiter en faire une compilation afin d'en extraire les thèmes les plus répandus. Mais vous ne savez pas nécessairement à l'avance quels peuvent être ces sujets, pas plus que le nombre de ceux-ci. Par conséquent, vous ne connaissez pas les sorties susceptibles d'être produites.
- Segmenter une clientèle en groupes ayant des préférences similaires. Partant d'enregistrements concernant vos clients, vous pourriez vouloir identifier lesquels présentent des similitudes, et s'il est possible de former des groupes avec des préférences de même nature. Dans le cas d'un site de vente en ligne, ces références pourraient par exemple se définir comme « parents », « rats de bibliothèque » ou encore « joueurs ». Comme vous ne savez pas à l'avance ce que pourraient être ces groupes, pas plus que leur nombre, vous ne disposez pas de sorties connues.
- Détection de séries d'accès anormaux à un site Web. Pour identifier des abus (ou des bogues), il est souvent utile de trouver des schémas d'accès qui diffèrent des procédures normales. Chaque schéma anormal peut être très différent des autres, et il se peut donc que vous n'ayez pas une trace enregistrée de ce type de comportement. Comme, dans cet exemple, vous ne pouvez rien faire d'autre que d'observer un trafic, et que vous ne

savez pas par avance ce qui constitue un comportement normal ou anormal, il s'agit bien d'un problème non supervisé. [43]

3. L'apprentissage semi-supervisé

Effectué de manière probabiliste ou non, il vise à faire apparaître la distribution sous-jacente des exemples dans leur espace de description. Il est mis en œuvre quand des données (ou « étiquettes ») manquent. Le modèle doit utiliser des exemples non étiquetés pouvant néanmoins renseigner. Exemple : en médecine, il peut constituer une aide au diagnostic ou au choix des moyens les moins onéreux de tests de diagnostic [44].

4. L'apprentissage par renforcement

Utilise des ensembles d'entrées, de sorties et de notes données d'entraînement. Il est généralement utilisé lorsqu'une interaction optimale est requise, telle que contrôle et jeux. L'algorithme apprend un comportement étant donné une observation. L'action de l'algorithme sur l'environnement produit une valeur de retour qui guide l'algorithme d'apprentissage. L'apprentissage par renforcement est utilisé dans plusieurs **applications** : robotique, gestion de ressources, vol d'hélicoptères, chimie [45].

5. L'apprentissage par transfert

L'apprentissage par transfert peut être vu comme la capacité d'un système à reconnaître et appliquer des connaissances et des compétences, apprises à partir de tâches antérieures, sur de nouvelles tâches ou domaines partageant des similitudes (exp : imaginons une personne qui veuille apprendre à jouer du piano, elle pourra plus facilement le faire si elle sait jouer de la guitare. Elle pourra utiliser ses connaissances en musique pour apprendre à jouer un nouvel instrument) [46].

II.6 Le Deep Learning (L'apprentissage Profond)

L'apprentissage profond (Deep Learning) est une branche de l'apprentissage automatique. Cette technologie se base principalement sur les réseaux de neurones artificiels à multiples couches imitant ainsi le fonctionnement du cerveau humain. Ces dernières années l'apprentissage profond a prouvé son efficacité dans plusieurs domaines : traitement des images, traduction automatique, traitement automatique du langage naturel, etc.. [47]. Le but de notre projet est de prendre avantage des algorithmes et techniques de l'apprentissage profond

pour apprendre, identifier ou traiter le contenu de l'image en se basant sur des méthodes optimisées comme les réseaux de neurones convolutifs.

La Figure II-4 illustre le concept du l'apprentissage en profond et sa relation avec l'apprentissage automatique.

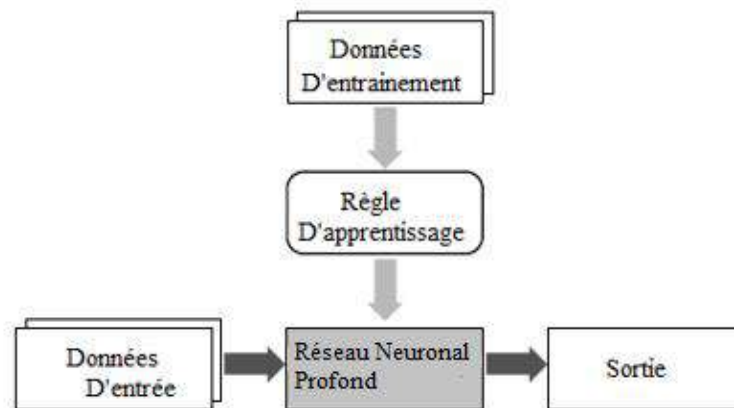


Figure II-4 Le concept de L'apprentissage Profond et sa relation avec L'apprentissage Automatique

II.7 Pourquoi le Deep Learning est important ?

Comment le Deep Learning obtient-il des résultats si impressionnants ?

Grâce à un seul principe : la précision. Les niveaux de précision de reconnaissance du Deep Learning n'ont jamais été aussi élevés. Les produits électroniques grand public peuvent ainsi répondre aux attentes client, ce qui est vital pour les applications où prime la sécurité, comme c'est le cas pour les véhicules autonomes. Des progrès récents ont amélioré le Deep Learning à tel point qu'il surpasse désormais les capacités humaines dans la réalisation de certaines tâches, telles que la classification d'objets dans des images.

Alors que les premières théories concernant le Deep Learning remontent aux années 1980, ce n'est que récemment qu'il est devenu exploitable. En voici les raisons :

1. Le Deep Learning nécessite un vaste jeu de **données labellisées**. Le développement de voitures autonomes s'appuie par exemple sur des millions d'images et des milliers d'heures de vidéo.
2. Le Deep Learning exige une **puissance de calcul** considérable. Les GPU haute performance sont dotés d'une architecture parallèle, qui est efficace pour le Deep Learning. Lorsque l'on y associe des clusters ou du cloud computing, il devient possible

pour les équipes de développement de réduire la durée d'entraînement des réseaux de Deep Learning, de plusieurs semaines à quelques heures ou moins. [47]

II.8 Exemples d'application de Deep Learning

Des applications de Deep Learning sont utilisées dans divers secteurs, de la conduite automatisée aux dispositifs médicaux.

1) Conduite automatisée :

Les chercheurs du secteur automobile ont recours au Deep Learning pour détecter automatiquement des objets tels que les panneaux stop et les feux de circulation. Le Deep Learning est également utilisé pour détecter les piétons, évitant ainsi nombre d'accidents.

2) Aérospatiale et défense :

Le Deep Learning sert à identifier des objets à partir de satellites utilisés pour localiser des zones d'intérêt et identifier quels secteurs sont sûrs ou dangereux pour les troupes au sol.

3) Recherche médicale :

À l'aide du Deep Learning, les chercheurs en cancérologie peuvent dépister automatiquement les cellules cancéreuses. Des équipes de l'Université de Californie à Los Angeles (UCLA) ont conçu un microscope qui génère un ensemble de données de grande dimension afin d'entraîner une application de Deep Learning à identifier avec précision des cellules cancéreuses.

4) Automatisation industrielle :

Le Deep Learning participe à l'amélioration de la sécurité des employés travaillant à proximité d'équipements lourds, en détectant automatiquement les situations dans lesquelles la distance de sécurité qui sépare le personnel ou les objets des machines est insuffisante.

5) Électronique :

Le Deep Learning est utilisé pour la reconnaissance audio et vocale. Par exemple, les appareils d'assistance à domicile qui répondent à votre voix et connaissent vos préférences fonctionnent grâce à des applications de Deep Learning. [47]

II.9 Fonctionnement du Deep Learning

La plupart des méthodes de Deep Learning utilisent des architectures de réseaux de neurones, ce qui explique pourquoi il est souvent question de réseaux de neurones profonds pour désigner des modèles de Deep Learning.

Le terme « profond » se rapporte généralement au nombre de couches cachées du réseau de neurones. Les réseaux de neurones classiques ne comportent que 2 à 3 couches cachées, comme le montre dans **Figure II-5**, constituées d'un ensemble de nœuds interconnectés, tandis que les réseaux profonds peuvent en compter jusqu'à 150.

L'entraînement des modèles s'effectue à l'aide de vastes ensembles de données labellisées et d'architectures de réseaux de neurones qui apprennent des caractéristiques directement depuis les données, sans avoir à effectuer une extraction manuelle.

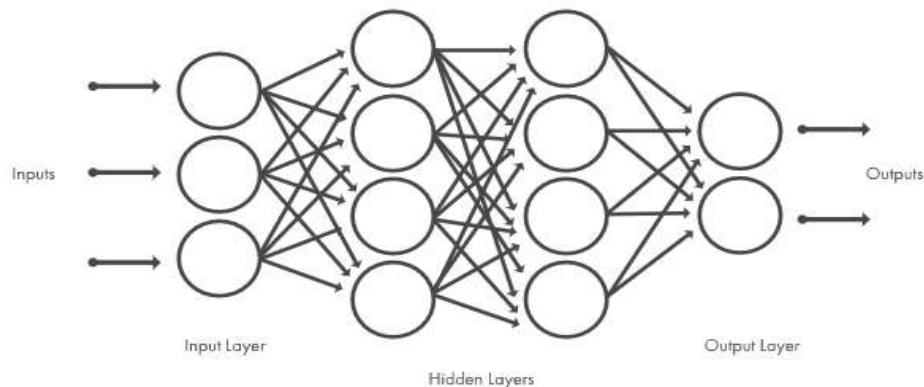


Figure II-5 Les réseaux de neurones sont organisés en couches constituées d'un ensemble de nœuds interconnectés [42]

II.10 Qu'est-ce qui différencie le Machine Learning du Deep Learning ?

Le Deep Learning est une branche particulière du Machine Learning. Un processus de Machine Learning commence par l'extraction manuelle de caractéristiques pertinentes à partir des données d'entrée. En s'appuyant sur ces caractéristiques, un modèle qui catégorise les données est ensuite créé. Dans un processus de Deep Learning, l'extraction des caractéristiques pertinentes à partir des données d'entrée est automatique (**Figure II-6**). En outre, le Deep Learning effectue un apprentissage « de bout en bout » : à partir de données brutes, un réseau se voit assigner des tâches à accomplir (une classification, par exemple) et apprend comment les automatiser.

Une autre différence majeure est le fait que les algorithmes de Deep Learning évoluent avec les données, tandis que le Shallow Learning « apprentissage peu profond » converge. Le

Shallow Learning désigne les méthodes de Machine Learning dont la progression s'arrête à partir d'un certain niveau de performance après l'alimentation du réseau en exemples supplémentaires et en données d'apprentissage.

Un des avantages majeurs des réseaux de Deep Learning réside dans leur capacité à continuer à s'améliorer en même temps que le volume de vos données augmente.



Figure II-6 Comparaison de méthodes de catégorisation de véhicules de Machine Learning (gauche) et de Deep Learning (droite) [50]

Pour classer des images avec le Machine Learning, les choix de caractéristiques et de classificateur doivent être effectués manuellement. Avec le Deep Learning, l'extraction de caractéristiques et le processus de modélisation sont automatiques [48].

II.11 Choisir entre le Machine Learning et le Deep Learning

Le Machine Learning met diverses méthodes et modèles à votre disposition, que vous pouvez choisir en fonction de votre application, de la quantité de données que vous traitez et du type de problème que vous voulez résoudre. Pour réussir une application de Deep Learning, vous avez besoin d'un volume de données très important (des milliers d'images) pour entraîner le modèle, en plus d'un ou de plusieurs GPU (processeur graphique) pour traiter les données rapidement.

Lorsque vous devez faire un choix entre le Machine Learning ou le Deep Learning, posez-vous la question de savoir si vous pouvez utiliser un GPU haute performance et si vous disposez d'un grand volume de données labellisées. Si vous n'avez aucun de ces éléments en votre possession, il est probablement plus judicieux d'utiliser le Machine Learning plutôt que le Deep Learning. D'ordinaire, le Deep Learning est plus complexe, et nécessite un minimum de quelques milliers d'images pour obtenir des résultats fiables. Si vous pouvez utiliser un GPU haute performance, le modèle analysera toutes ces images plus rapidement. [47]

II.12 Fondations de réseaux de neurones

II.12.1 Définition

Les réseaux de neurones artificiels (brièvement, les filets) représentent une classe de modèles d'apprentissage automatique, inspiré par des études sur les systèmes nerveux centraux des mammifères. Chaque filet est composé de plusieurs neurones interconnectés, organisés en couches, qui échangent des messages (ils tirent, en jargon) quand certaines conditions se produisent. Les premières études ont commencé à la fin des années 50 avec l'introduction du perceptron [49].

Un réseau à deux couches utilisé pour des opérations simples, et encore élargi dans la fin des années 1960 avec l'introduction de l'algorithme de rétro propagation, utilisé pour un apprentissage efficace des réseaux multicouches [50].

Les réseaux de neurones ont fait l'objet d'études universitaires intensives jusqu'aux années 1980, lorsque d'autres approches plus simples plus pertinent. Cependant, il y a eu une résurgence d'intérêt à partir du milieu des années 2000, grâce à la fois à un algorithme révolutionnaire d'apprentissage rapide proposé par G. Hinton.

Ces améliorations ont ouvert la voie à l'apprentissage profond moderne, une classe de réseaux de neurones caractérisée par un nombre important de couches de neurones, qui sont capables d'apprendre plutôt modèles sophistiqués basés sur des niveaux d'abstraction progressifs. Les gens l'ont appelé profond avec 3-5 couches il y a quelques années, et maintenant il est passé à 100-200. [51]

II.12.2 Principe de fonctionnement

Dans un réseau de neurones, deux modes de fonctionnement peuvent être distingué. Dans le premier, les paramètres du réseau sont ajustés grâce à la présentation d'exemples pour lesquels on connaît la réponse désirée : ce mode de fonctionnement est appelé « phase d'apprentissage ». Dans le deuxième mode, il s'agit d'exploiter le réseau en lui présentant des données inconnues : ce mode d'utilisation est souvent appelé « phase de reconnaissance ».

Un neurone reçoit des entrées et fournit une sortie, grâce à différentes caractéristiques :

- Des poids accordés à chacune des entrées, permettant de modifier l'importance de certaines par rapport aux autres.

- Une fonction d'agrégation, qui permet de calculer une unique valeur à partir des entrées et des poids correspondants.
- Un seuil (ou biais), permettant d'indiquer quand le neurone doit agir.
- Une fonction d'activation, qui associe à chaque valeur agrégée une unique valeur de sortie dépendant du seuil. [52]

II.12.3 Neurone biologique

Le neurone est une cellule composée d'un corps cellulaire et d'un noyau. Le corps cellulaire se ramifie pour former ce que l'on nomme les dendrites. C'est par les dendrites que l'information est acheminée de l'extérieur vers le soma, corps du neurone [53].

L'information traitée par le neurone se propage ensuite le long de l'axone pour être transmise aux autres neurones. La transmission entre deux neurones n'est pas directe. La jonction entre deux neurones est appelée la synapse (**Figure II-7**).

Bref un neurone biologique est une cellule qui se caractérise par :

- Des synapses, les points de connexion avec les autres neurones, Fibres Nerveuses ou musculaires.
- Des dendrites ou entrées des neurones.
- Les axones, ou sorties du neurone vers d'autres neurones ou fibres musculaires

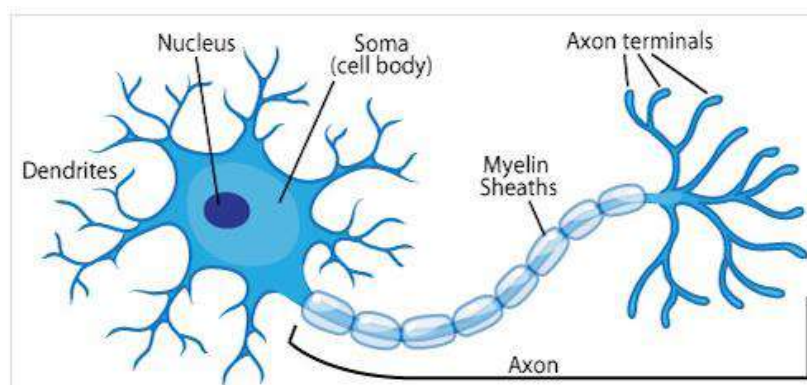


Figure II-7 Anatomie d'un neurone biologique [52]

II.12.4 Neurone formel

Un neurone formel est un modèle mathématique. Il est utilisé dans le cadre des recherches sur l'intelligence artificielle, principalement en association avec d'autres neurones [55]. Il contient plusieurs formules mathématiques afin de reproduire le plus fidèlement possible le fonctionnement d'un neurone avec ses différentes entrées (dendrites), leurs importance sortie (axone) et ainsi comprendre son potentiel d'interaction avec les autres neurones (**Figure II-8**).

Il est caractérisé par :

- Des entrées x_n et leur poids w_{ij} synaptique
- Une fonction, f , d'activation (ou de transfert)
- Une sortie y

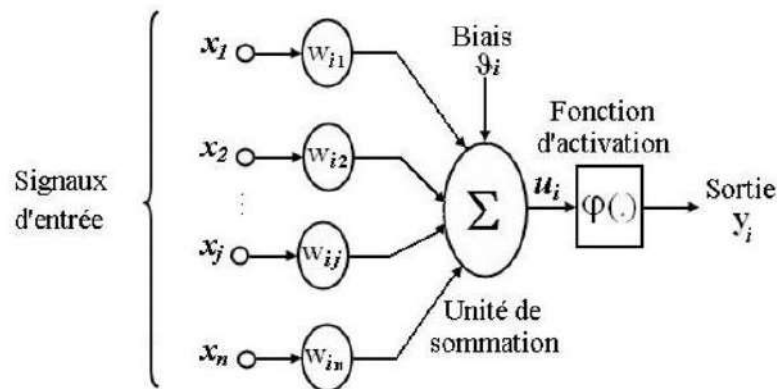


Figure II-8 Modèle du neurone formel de Mac Culloch et Pitts (avec biais) [56]

- Chaque poids possède une valeur notée w_{ij} . Cette notation, la plus répandue dans la littérature scientifique, désigne le poids allant d'un neurone formel i au neurone formel j .
- Chaque poids transmet une information/un stimulus provenant du neurone source i noté x_i .
- Ce stimulus (sa valeur) correspondant à l'information envoyé par le neurone source i est modulé par le poids liant les neurones i et j . Mathématiquement cela se traduit par :

$$w_{ij} * x_i \quad (2.1)$$

- Ainsi neurone j reçoit autant de stimuli que de poids, dont il fait la somme

$$\sum (w_{ij} * x_i) \quad (2.2)$$

Si l'on note n le nombre de neurones sources liées aux neurones j , une notation mathématique plus complète serait :

$$\sum_{i=0}^n (W_{ij} * x_i) \quad (2.3)$$

Cette expression se lit alors comme ce qui suit : "la somme de toutes les multiplications des valeurs des n neurones sources par les poids associant ces neurones sources au neurone j considéré " (i prenant les valeurs : 0, 1, 2, ..., n).

C'est cette somme que le neurone formel j doit alors traiter ! Il utilise pour cela la fonction de transfert.

II.12.5 Neurone artificiel

Un réseau de neurones artificiel (RNA) est constitué de cellules connectées entre elles par des liaisons affectées de poids. Ces liaisons permettent à chaque cellule de disposer d'un canal pour envoyer et recevoir des signaux en provenance d'autres cellules du réseau [57].

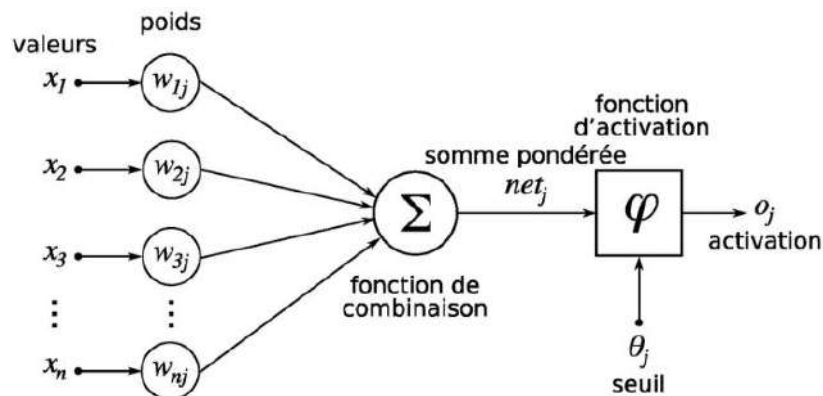


Figure II-9 Structure d'un neurone artificiel [57]

Le neurone calcule la somme de ses entrées puis cette valeur passée à travers la fonction d'activation pour produire sa sortie [58].

II.13 Architecture des réseaux de neurones

L'architecture d'un réseau de neurones définit son fonctionnement et joue un rôle important dans son comportement. Elle est en fonction de ses couches et de la structure des connexions de ses neurones ; ces paramètres permettent de distinguer les différentes classes

et/ou types d'architecture neuronales [59]. Donc Il existe trois grands types d'architectures de réseaux de neurones :

II.13.1 Réseaux statiques

Ce type de réseaux est organisé généralement en couches de neurones. Chaque neurone d'une couche reçoit ses entrées à partir des neurones de la couche précédente ou tout simplement de l'entrée du réseau. Dans tels réseaux il n'existe pas de feed-back (boucles de retour d'informations). Le traitement des données se fait en sens unique et le flux d'informations circule directement de la couche d'entrée vers la couche de sortie ; le traitement est donc réalisé en boucle ouverte. Ces réseaux peuvent être utilisés pour les problèmes de classification ou d'approximation des fonctions [59].

II.13.2 Réseaux dynamiques

Les réseaux dynamiques ou bien récurrents, sont des réseaux pouvant comporter des boucles (feed-back) entre les neurones. En général, la sortie de chaque neurone peut être envoyée vers l'entrée de tous les autres neurones du réseau. Ainsi, ces boucles ramènent l'information en provenance de la couche de sortie sur la couche d'entrée simultanément avec le signal d'entrée présent au même instant. Un réseau dynamique peut donner une sortie différente en lui présentant la même entrée à des instants différents [59].

II.13.3 Réseaux auto-organisés

Les réseaux de neurones auto-organisés sont des réseaux qui changent leurs structures internes pendant l'apprentissage. Ainsi les neurones se regroupent typologiquement suivant la représentation des exemples. Ces réseaux sont des dérivées des modèles de Kohonen [59].

II.14 Apprentissage des neurones artificiels

II.14.1 Définition

L'apprentissage est vraisemblablement la propriété la plus intéressante des réseaux neuronaux. Elle ne concerne cependant pas tous les modèles, mais les plus utilisés [60].

L'apprentissage est la modification des poids du réseau dans l'optique d'accorder la réponse du réseau aux exemples et l'expérience. Les poids sont initialisés avec des valeurs

aléatoires. Puis des données représentatives du fonctionnement du procédé dans un domaine donné, sont présentées au réseau de neurones [61].

II.14.2 Sur-apprentissage

Il arrive qu'à faire apprendre un réseau de neurones toujours sur le même échantillon, celui-ci devient inapte à reconnaître autre chose que les éléments présents dans l'échantillon. Le réseau ne cherche plus l'allure générale de la relation entre les entrées et les sorties du système, mais cherche à reproduire les allures de l'échantillon. On parle alors de sur-apprentissage : le réseau est devenu trop spécialisé et ne généralise plus correctement. Ce phénomène apparaît aussi lorsqu'on utilise trop d'unités cachées (de connexion), la phase d'apprentissage devient alors trop longue [61].

II.15 Les types d'apprentissage

Au niveau des algorithmes d'apprentissage, Il existe deux approches d'apprentissage :

II.15.1 Apprentissage supervisé

Apprentissage supervisé implique un mécanisme fournissant au réseau le résultat souhaité, soit en "évaluant" manuellement les performances du réseau, soit en fournissant les résultats souhaités avec les entrées. La grande majorité des réseaux utilisent l'apprentissage supervisée. En bref un apprentissage est dit supervisé lorsque le réseau est forcé à converger vers un état final précis, en même temps qu'un motif lui est présenté. Les coefficients synaptiques sont évalués en minimisant l'erreur (entrée/sortie-souhaitée et sortie obtenue) sur une base d'apprentissage [62].

II.15.2 Apprentissage non supervisé

Pour un apprentissage non supervisé, le réseau doit donner un sens aux entrées sans aide extérieure, il est utilisé pour effectuer une caractérisation initiale des entrées. En bref, à l'inverse de l'apprentissage supervisé, lors d'un apprentissage non-supervisé, le réseau est laissé libre de converger vers n'importe quel état final lorsqu'un motif lui est présenté. On ne dispose pas de base d'apprentissage. Les coefficients synaptiques sont déterminés par rapport à des critères de conformité : spécifications générales [62].

II.16 Les types de réseaux de neurones

On peut généralement retenir 6 principaux RNA (réseaux de neurones artificiels) à partir de deux grandeurs catégories de réseaux :

- * Réseaux Feed-Forward (Les RNA à apprentissage supervisé non bouclés)
- * Réseaux Feed-Back (Les RNA à apprentissage non supervisé récurrents ou bouclés)

II.16.1 Les RNA à apprentissage supervisé (non bouclés)

II.16.1.a Perceptron monocouche

Avant d'aborder le comportement collectif d'un ensemble de neurones, nous allons présenter le Perceptron (un seul neurone) en phase d'utilisation. L'apprentissage ayant été réalisé, les poids sont fixes. Le neurone de la **Figure II-10** réalise une simple somme pondérée de ses entrées, compare une valeur de seuil, et fourni une réponse binaire en sortie.

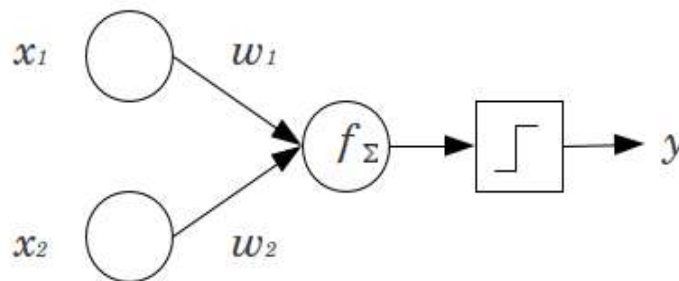


Figure II-10 Structure et comportement d'un perceptron. (Les connexions des deux entrées x_1 et x_2 au neurone sont pondérées par les poids w_1 et w_2)

II.16.1.b Perceptron multicouche

Dans ce paragraphe, nous définissons le premier exemple d'un réseau à plusieurs couches linéaires. Historiquement, le perceptron était le nom donné à un modèle ayant une seule couche linéaire, et par conséquent, s'il a plusieurs couches, vous l'appelleriez perceptron multicouche (MLP en anglais et PMC en français). Le perceptron multicouche (PMC) est l'un des réseaux de neurones les plus utilisés actuellement, pour la classification supervisée notamment. Le perceptron est organisé en plusieurs couches. La première est reliée aux entrées, par la suite chaque couche est reliée à la couche précédente. C'est la dernière couche qui produit les sorties du PMC [63], de telle sorte que les neurones sont reliés entre eux par des connexions pondérées, ce sont les poids de ces connexions qui gouvernent le fonctionnement du réseau comme le montre dans la **Figure II-11** au-dessous :

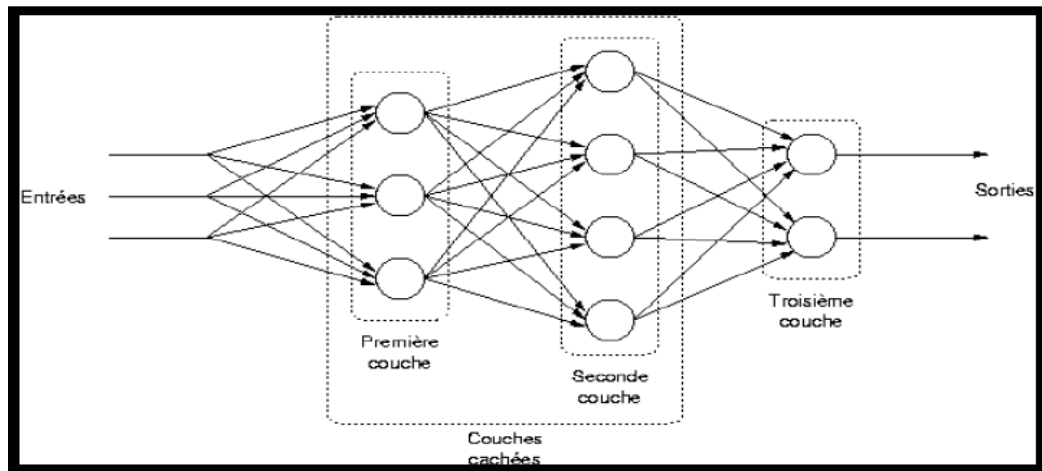


Figure II-11 Schéma illustrant un perceptron multicouche avec deux couches cachées.

Pour le premier mode de fonctionnement du perceptron multicouche on distingue trois algorithmes :

- Algorithme de rétro-propagation du gradient.
- Algorithme de gradient conjugué.
- Méthodes de second ordre.

L'article [64] décrit la méthode exacte du fonctionnement d'adaptation des poids du perceptron multicouche.

II.16.1.c Les réseaux à fonction radiale de base (Radial basis fonction RBF)

Ce sont les réseaux que l'on nomme aussi RBF (« radial basic fonction »). L'architecture est la même que pour les PMC, cependant les fonctions de base utilisées ici sont des fonctions Gaussiennes. Les RBF seront donc employés dans les mêmes types de problèmes que les PMC à savoir, en classification et en approximation de fonctions, particulièrement. L'apprentissage le plus utilisé pour les RBF est le mode hybride et les règles sont soit, la règle de correction de l'erreur soit, la règle d'apprentissage par compétition [64].

II.16.2 Les RNA à apprentissage non supervisé (récurrents ou bouclés)

II.16.2.a Le réseau de Kohonen

Ou bien carte auto-organisatrice de Kohonen, où n individus sont représentés par k neurones sans perdre la notion de distance. Lorsqu'un neurone est choisi comme représentant, il se déplace vers l'individu et attire les neurones voisins. Les neurones de ce réseau sont constitués d'un vecteur de poids dans l'espace des entrées. La carte des neurones définit quant

à elle des relations de voisinage entre les neurones. L'algorithme d'apprentissage du réseau de Kohonen est de type compétitif [65].

II.16.2.b Réseaux Hopfield

Les réseaux de Hopfield sont des réseaux récurrents et entièrement connectés. Dans ce type de réseau, chaque neurone est connecté à chaque autre neurone et il n'y a aucune différenciation entre les neurones d'entrée et de sortie. Ils fonctionnent comme une mémoire associative non-linéaire et sont capables de trouver un objet stocké en fonction de représentation partielles ou bruitées. L'application principale des réseaux de Hopfield est la reconnaissance mais aussi la résolution de problèmes d'optimisation. Le mode d'apprentissage utilisé ici est le mode non supervisé [61].

II.16.2.c Réseaux ART

Les réseaux ART ("Adaptative Résonance Théorie") sont des réseaux à apprentissage par compétition. Le problème majeur qui se pose dans ce type de réseaux est le dilemme "stabilité/plasticité". En effet, dans un apprentissage par compétition, rien ne garantit que les catégories formées aillent rester stables. La seule possibilité, pour assurer la stabilité, serait que le coefficient d'apprentissage tende vers zéro, mais le réseau perdrait alors sa plasticité [61].

II.17 Les branches du réseau neuronal

Le réseau neuronal est un réseau de nœuds. Une variété de réseaux de neurones peut être créée selon la façon dont les nœuds sont connectés [42]. L'un des types de réseaux de neurones les plus couramment utilisés emploie une structure en couches de nœuds comme le montre la **Figure II-12** :

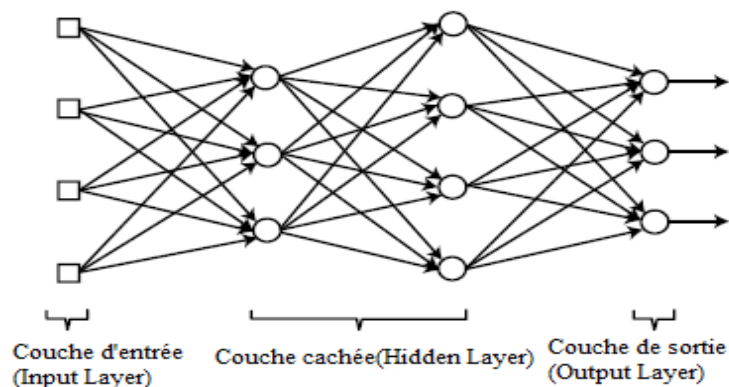


Figure II-12 Une structure en couches de nœuds

Le tableau suivant résume les branches du réseau neuronal en fonction de l'architecture des couches :

Tableau 1 Les branches du réseau neuronal en fonction de l'architecture des couches

Single-Layer Neural Network (Réseau Neuronal Monocouche)		Input Layer-Output Layer (Couche d'entrée-couche de sortie)
Multi-Layer Neural Network (Réseau Neuronal Multicouche)	Shallow Neural Network SNN (Réseau Neuronal Peu Profond)	Input Layer-Hidden Layer-Output Layer (Couche d'entrée cachée Couche de sortie)
	Deep Neural Network DNN (Réseau de Neurones Profonds)	Input Layer-Hidden Layer-Output Layer (Couche d'entrée cachée Couche de sortie)

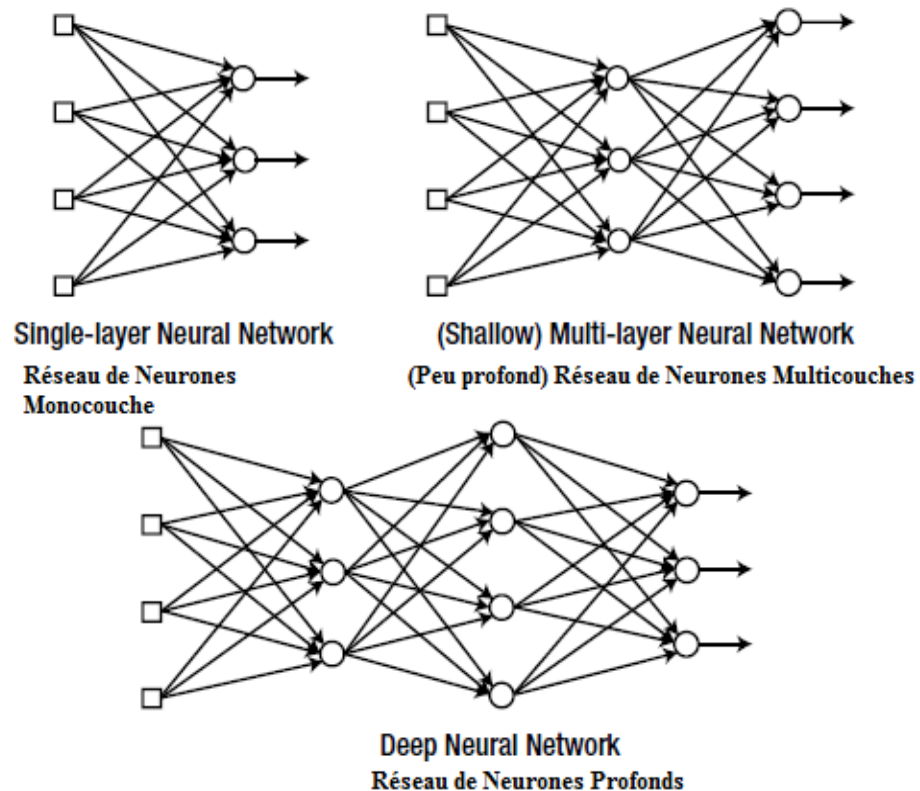


Figure II-13 Les branches du réseau neuronal en fonction de l'architecture des couches [42]

II.18 Domaines d'application du réseau de neurones

Grâce à leur capacité dans la classification, approximation, commande robotique, prédiction, mémoire associative et généralisation les réseaux de neurones ont été utilisés dans plusieurs domaines [61] :

- Traitement d'image : compression d'images, reconnaissance de caractères et de signatures, reconnaissance de formes et de motifs, chiffrement, classification, etc.
- Traitement du signal : traitement de la parole, identification de sources, filtrage, classification, ...
- Traitement automatique des langues : segmentation en mots, représentation sémantique des mots, étiquetage morphosyntaxique, traduction automatique, ...
- Contrôle : diagnostic de pannes, commande de processus, contrôle qualité, robotique.
- Optimisation : allocation de ressources, planification, régulation de trafic, finance.
- Simulation : simulation boîte noire, prévisions météorologiques.
- Classification d'espèces animales étant donnée une analyse ADN.
- Modélisation de l'apprentissage et perfectionnement des méthodes d'apprentissage.
- Approximation d'une fonction inconnue ou modélisation d'une fonction connue mais complexe à calculer avec précision. [66]

II.19 Avantages et Inconvénients du réseau de neurone

II.19.1 Avantage

- Capacité de représenter n'importe quelle fonction, linéaire ou pas, simple ou complexe.
- Les réseaux de neurones sont capables de tolérer des données bruitées, ainsi que capable de classer les modèles sur lesquels ils ne sont pas été entraînés.
- Ils peuvent être utilisés lorsque nous avons peu d'informations sur la relation entre les attributs et les classes.
- Bien adapté pour les entrées et les sorties d'une valeur continue.
- Le succès sur plusieurs applications du monde réel comme la reconnaissance manuscrite de caractère, de la pathologie et de médecine de laboratoire, et beaucoup plus. [67]

II.19.2 Inconvénients

Parmi ses inconvénients on cite [67] :

- Nécessitent un long temps d'apprentissage, donc ils sont adaptable aux applications dont ceci est faisable.
- Pauvre interprétation de la connaissance représentée sous la forme d'un réseau d'unités connectées par des liens pondérés.
- Exigent un nombre de paramètres qui doivent être déterminées empiriquement, par exemple la topologie du réseau et sa structure, le nombre de couches cachées, le nombre d'unités dans chaque couche cachée et dans la couche de sortie.

II.20 Réseau de neurones et Classification

Les principales applications d'apprentissage automatique nécessitent un apprentissage supervisé sont la classification et la régression. La classification est utilisée pour déterminer le groupe auquel les données appartiennent. Quelques applications typiques de la classification sont le filtrage du courrier indésirable et la reconnaissance des caractères. En revanche, la régression déduit les valeurs des données. Il peut être illustré par la prédiction de revenu pour un âge et un niveau d'éducation donnés.

Bien que le réseau neuronal soit applicable à la fois à la classification et régression, il est rarement utilisé pour la régression. Ce n'est pas parce qu'il donne de mauvais résultats performances, mais parce que la plupart des problèmes de régression peuvent être résolus en utilisant modèles plus simples. [42]

II.20.1 Classification binaire

Nous allons commencer par le réseau neuronal de classification binaire, qui classe le saisir des données dans l'un des deux groupes [42]. Pour la classification binaire, un seul nœud de sortie est suffisant pour le réseau de neurones. En effet, les données d'entrée peuvent être classées selon la valeur de sortie, supérieure ou inférieure au seuil. Par exemple, si la fonction sigmoïde est utilisée comme fonction d'activation du nœud de sortie, les données peuvent être classées selon que la sortie est supérieure à 0,5 ou non. Comme la fonction sigmoïde varie de 0 à 1, nous pouvons diviser les groupes au milieu, comme le montre la **Figure II-14** :

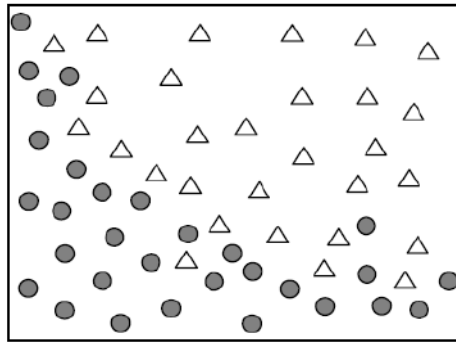


Figure II-14 Problème de classification binaire

Considérez le problème de classification binaire montré dans la **Figure II-14**. Étant donné les coordonnées (x, y), le modèle consiste à déterminer à quel groupe appartiennent les données. Dans ce cas, les données d'entraînement sont données dans le format illustré à la **Figure II-15**. Les deux premiers chiffres indiquent respectivement les coordonnées x et y et le symbole représente le groupe auquel appartiennent les données. Les données se composent de l'entrée et sortie correcte car elle est utilisée pour l'apprentissage supervisé.

{ 5, 7, Δ }
{ 9, 8, $\bullet$ }
...
{ 6, 5, $\bullet$ }

Figure II-15 Classification binaire des données d'apprentissage

Maintenant, construisons le réseau neuronal. Le nombre de nœuds d'entrée est égal au nombre de paramètres d'entrée. Comme l'entrée de cet exemple consiste de deux paramètres, le réseau utilise deux nœuds d'entrée. Nous avons besoin d'un nœud de sortie car cela implémente la classification de deux groupes comme précédemment adressé. La fonction sigmoïde est utilisée comme fonction d'activation et la fonction la couche cachée a quatre nœuds. La **Figure II-16** montre l'architecture de ce réseau neuronal.

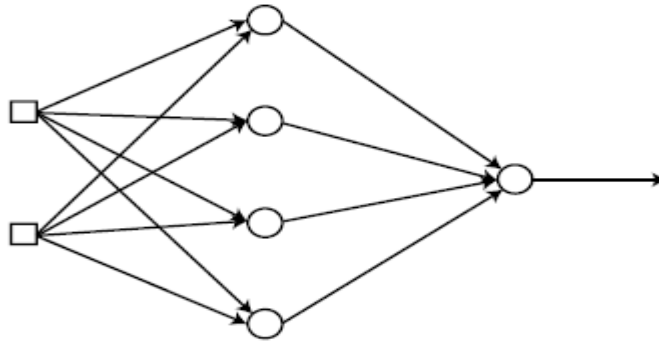


Figure II-16 Réseau de neurones pour les données d'entraînement

Lorsque nous formons ce réseau avec les données d'apprentissage, nous pouvons obtenir la classification binaire que nous voulons. Cependant, il y a un problème. Le neurone réseau produit des sorties numériques allant de 0 à 1, alors que nous avons le sorties correctes symboliques données par Δ et $\bullet$. Nous ne pouvons pas calculer l'erreur dans par ici ; nous devons changer les symboles en codes numériques. Nous pouvons attribuer les valeurs maximales et minimales de la fonction sigmoïde aux deux classes suit :

Class $\Delta \rightarrow 1$

Class $\bullet \rightarrow 0$

Le changement des symboles de classe donne les données d'apprentissage montrées dans la **Figure II-17** :

{ 5, 7, 1 }
{ 9, 8, 0 }
...
{ 6, 5, 0 }

Figure II-17 La Modification de symboles de la classe par un code numérique

II.20.2 Classification Multi-classe

Cette section présente comment utiliser le réseau de neurones pour gérer la classification de trois classes ou plus. Considérons une classification des données entrées de coordonnées (x, y) dans l'une des trois classes (**Figure II-18**).

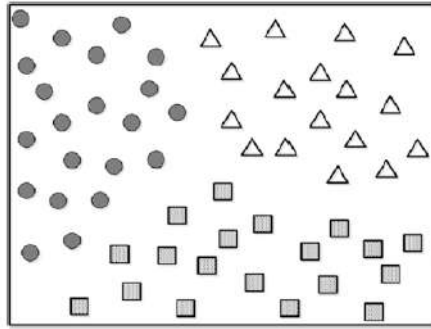


Figure II-18 Données avec trois classes

Nous devons d'abord construire le réseau neuronal. Nous utiliserons deux nœuds pour la couche d'entrée car l'entrée se compose de deux paramètres. Pour plus de simplicité, les couches cachées ne sont pas prises en compte pour le moment. Nous devons déterminer le nombre de nœuds de sortie également. Il est bien connu que la correspondance du nombre de sorties nœuds au nombre de classes est la méthode la plus prometteuse. Dans cet exemple, nous utilisons trois nœuds de sortie, car le problème nécessite trois classes. **Figure II-19** illustre le réseau neuronal configuré : [40]

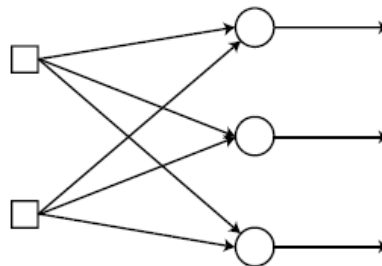


Figure II-19 Réseau de neurone configuré pour trois classes

Une fois que le réseau est entraîné avec les données fournies, nous obtenons le classificateur multi-classe que nous voulons. Les données d'entraînement sont données dans la **Figure II-20**. Pour chaque point de données, les deux premiers nombres sont respectivement les coordonnées x et y et la troisième valeur est la classe correspondante. Les données comprennent l'entrée et sortie correcte car elle est utilisée pour l'apprentissage supervisé.

{ 5, 7, Class1 }
{ 9, 8, Class 3 }
{ 2, 4, Class 2 }
...
{ 6, 5, Class 3 }

Figure II-20 Données d'apprentissage avec classification multi-classes

Afin de calculer l'erreur, nous changeons les noms de classe en codes numériques, comme nous l'avons fait dans la section précédente. Considérant que nous avons trois nœuds de sortie du réseau neuronal, nous créons les classes comme les vecteurs suivants :

Class 1 $\rightarrow$ [1 0 0]

Class 2 $\rightarrow$ [0 1 0]

Class 3 $\rightarrow$ [0 0 1]

Cette transformation implique que chaque nœud de sortie est mappé à un élément du vecteur de classe, qui ne donne que 1 pour le nœud correspondant. Par exemple, si les données appartiennent à la classe 2, la sortie ne produit que 1 pour le deuxième nœud et 0 pour les autres (**Figure II-21**) :

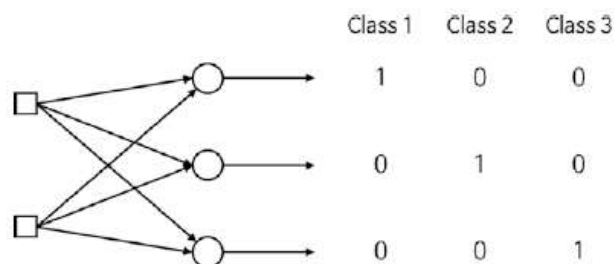


Figure II-21 Chaque nœud de sortie est maintenant mappé à un élément du vecteur de classe

Cette technique d'expression est appelée codage à chaud ou codage 1 sur N. La raison pour laquelle nous faisons correspondre le nombre de nœuds de sortie au nombre de classes est d'appliquer cette technique d'encodage. Maintenant, les données d'entraînement sont affichées dans le format montré dans la **Figure II-22** :

{ 5, 7, 1, 0, 0 }
{ 9, 8, 0, 0, 1 }
{ 2, 4, 0, 1, 0 }
...
{ 6, 5, 0, 0, 1 }

Figure II-22 Modification des classes dans un nouveau format 1 sur 3

Exemple : Classification Multi-classes

Dans cette section, nous implémentons un réseau de classification multi-classes qui reconnaît des chiffres des images d'entrée. Il s'agit d'une classification multi-classes, comme il classe l'image en chiffres spécifiés. Les images d'entrée sont de cinq par cinq pixels carrés, qui affichent cinq nombres de 1 à 5, comme le montre la **Figure II-23** :

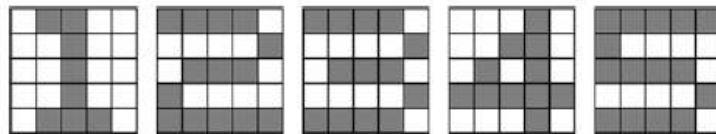


Figure II-23 Carrés de cinq pixels sur cinq affichant cinq nombres de 1 à 5

Le modèle de réseau de neurones contient une seule couche cachée, comme illustré dans **Figure II-24**. Comme chaque image est définie sur une matrice, nous définissons 25 nœuds d'entrée. En plus, comme nous avons cinq chiffres à classer, le réseau contient cinq nœuds de sortie [42].

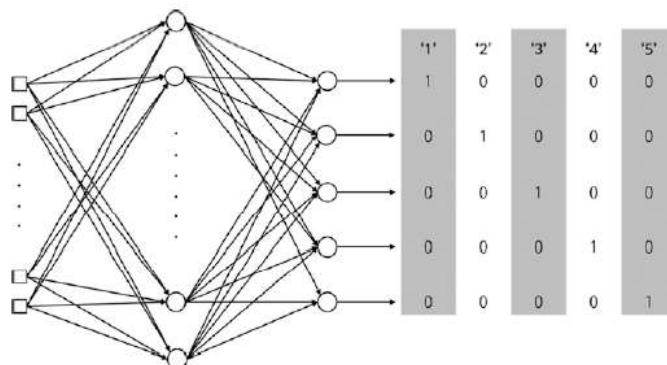


Figure II-24 Le modèle de réseau neuronal pour ce nouvel ensemble de données

II.21 Réseaux de neurones convolutifs CNN

II.21.1 Présentation

Les réseaux de neurones convolutifs CNN (Convolutional Neural Network), sont les structures les plus performantes pour faire la classification des images. Ils comportent deux parties bien distinctes. En entrée, une image est fournie sous la forme d'une matrice de pixels. Elle a 2 dimensions pour une image en niveaux de gris. La couleur est représentée par une troisième dimension, de profondeur 3 pour représenter les couleurs fondamentales [Rouge, Vert, Bleu]. La première partie d'un CNN est la partie convolutive à proprement parler. Elle fonctionne comme un extracteur de caractéristiques des images. Une image est passée à travers une succession de filtres, ou noyaux de convolution, créant de nouvelles images appelées cartes de convolutions. Certains filtres intermédiaires réduisent la résolution de l'image par une opération de maximum local. Au final, les cartes de convolutions sont mises à 1-D et concaténées en un vecteur de caractéristiques, appelé code CNN.

Le nom « réseau de neurones convolutif » indique que le réseau emploie une opération mathématique appelée convolution. La convolution est une opération linéaire spéciale. Les réseaux convolutifs sont simplement des réseaux de neurones qui utilisent la convolution à la place de la multiplication matricielle dans au moins une de leurs couches [68].

II.21.2 Architecture de réseaux de neurone convolutifs

Comme nous l'avons mentionnée précédemment, les réseaux de neurones convolutifs sont basés sur le perceptron multicouche (MLP). Bien qu'efficaces pour le traitement d'images, les MLP ont beaucoup de mal à gérer des images de grande taille, ce qui est dû à la croissance exponentielle du nombre de connexions avec la taille de l'image. Par exemple, si on prend une image de taille 32x32x3 (32 de largeur, 32 de hauteur, 3 canaux de couleurs), un seul neurone entièrement connecté dans la première couche cachée du MLP aurait 3072 entrées ($32 \times 32 \times 3$). Une image 200x200 conduirait ainsi à traiter 120 000 entrées par neurone ce qui, multiplié par le nombre de neurones, devient énorme [69].

L'architecture CNN est formée par un empilement de couches de traitement indépendantes :

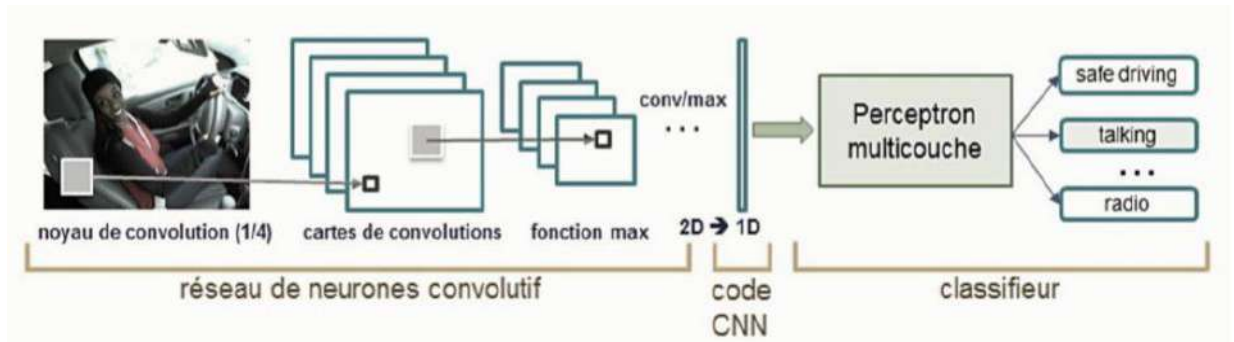


Figure II-25 Architecture standard d'un réseau de neurone convolutionnel [69]

- La couche de convolution (CONV) qui traite les données d'un champ récepteur.
- La couche de pooling (POOL), qui permet de compresser l'information en réduisant la taille de l'image intermédiaire (souvent par sous-échantillonnage).
- La couche de correction (ReLU), souvent appelée par abus 'ReLU' en référence à la fonction d'activation (Unité de rectification linéaire).
- La couche "entièrement connectée" (FC : Fully connected), qui est une couche de type perceptron.
- La couche de perte (LOSS).

II.21.3 Quelques réseaux convolutifs célèbres

- **LeNet [70]** : Les premières applications réussies des réseaux convolutifs ont été développées par Yann LeCun dans les années 1990. Parmi ceux-ci, le plus connu est l'architecture LeNet utilisée pour lire les codes postaux, les chiffres, etc.
- **AlexNet [71]** : Le premier travail qui a popularisé les réseaux convolutifs dans la vision par ordinateur était AlexNet, développé par Alex Krizhevsky, Ilya Sutskever et Geoff Hinton. Ce CNN été soumis au défi de la base ImageNet en 2012 et a nettement surpassé ses concurrents. Le réseau avait une architecture très similaire à LeNet, mais était plus profond, plus grand et comportait des couches convolutives empilées les unes sur les autres (auparavant, il était commun de ne disposer que d'une seule couche convolutifs toujours immédiatement suivie d'une couche de pooling).
- **ZFnet [72]** : C'était une amélioration d'AlexNet en ajustant les hyper-paramètres de l'architecture, en particulier en élargissant la taille des couches convolutifs et en réduisant la taille du noyau sur la première couche.

- **GoogLeNet** : C'est un modèle de Google. Sa principale contribution a été le développement d'un module inception qui a considérablement réduit le nombre de paramètres dans le réseau (4M, par rapport à AlexNet avec 60M). En outre, ce module utilise le global Average pooling ce qui élimine une grande quantité de paramètres. Il existe également plusieurs versions de GoogLeNet, parmi elles, Inception-v4 [73] et Xception [74].
- **ResNet [75]** : Residual network développé par Kaiming He et al. Été le vainqueur de ILSVRC 2015. Il présente des sauts de connexion et une forte utilisation de la batch normalisation. Il utilise aussi le global AVG pooling au lieu du PMC à la fin [76].
- **VGG Net : [77]** Il s'agit d'une structure du Visual Geometry Group d'Oxford réalisée par Andrea Vedaldi et Andrew Zisserman (en 2017).

II.21.4 Avantages de CNN

Un avantage majeur des réseaux convolutifs est l'utilisation d'un poids unique associé aux signaux entrants dans tous les neurones d'un même noyau de convolution. Cette méthode réduit l'empreinte mémoire, améliore les performances et permet une invariance du traitement par translation. C'est le principal avantage du CNN par rapport au MLP, qui lui considère chaque neurone indépendant et donc affecte un poids différent à chaque signal entrant.

II.21.5 Exemples de modèles de CNN

La forme la plus commune d'une architecture CNN empile quelques couches Conv-ReLU, les suit avec des couches Pool, et répète ce schéma jusqu'à ce que l'entrée soit réduite dans un espace d'une taille suffisamment petite. À un moment, il est fréquent de placer des couches entièrement connectées (FC : fully-connected). La dernière couche entièrement connectée est reliée vers la sortie. Voici quelques architectures communes CNN qui suivent ce modèle :

- ✓ INPUT -> CONV -> RELU -> FC
- ✓ INPUT -> [CONV -> RELU -> POOL] * 2 -> FC -> RELU -> FC Ici, il y a une couche de CONV unique entre chaque couche POOL
- ✓ INPUT -> [CONV -> RELU -> CONV -> RELU -> POOL] * 3 -> [FC -> RELU] * 2 -> FC Ici, il y a deux couches CONV empilées avant chaque couche POOL.

L'empilage des couches CONV avec de petits filtres de pooling (plutôt un grand filtre) permet un traitement plus puissant, avec moins de paramètres. Cependant, avec l'inconvénient

de demander plus de puissance de calcul (pour contenir tous les résultats intermédiaires de la couche CONV) [79].

II.22 Conclusion

Nous avons présenté dans ce chapitre les notions de base sur l'intelligence artificielle, Deep Learning et le Machine Learning et la relation entre eux ainsi que leurs fonctionnements et leurs utilisations dans le domaine de traitement d'images. Ensuite nous avons passé aux réseaux de neurones en citant les différents types de réseaux de neurones et en détaillant les réseaux de neurones convolutionnels. Ces réseaux sont capables d'extraire des caractéristiques d'images présentées en entrée et de classifier ces caractéristiques. Les réseaux de neurones convolutionnels présentent cependant un certain nombre de limitations, en premier lieu, les hyper paramètres du réseau sont difficiles à évaluer a priori. En effet, le nombre de couches, le nombre de neurones par couche ou encore les différentes connexions entre couches sont des éléments cruciaux et essentiellement.

Pour mieux maîtriser l'implémentation d'un tel système on doit passer dans le chapitre suivant à la préparation d'une conception détaillée permettant de comprendre le système à implémenter en un ensemble de modules. Les modules seront détaillés par la suite selon la démarche à suivre.

Chapitre III. IMPLEMENTATION DU SYSTEME RAPI

III.1 Introduction

Avec l'augmentation du nombre de voitures, le contrôle de ces voitures est trop difficile et coûte très cher. Pour surmonter ces difficultés, un système automatisé qui repose sur la reconnaissance des plaques d'immatriculation (RAPI) est fortement recommandé. Nous allons intégrer le système RAPI dans un système embarqué, pour cela nous avons choisi le Raspberry Pi 3 et la caméra Raspberry. De ce fait, ce chapitre fournit des informations sur le Raspberry Pi 3 et son appareil photo (caméra Raspberry) et leurs composants de base. Nous allons exploiter le réseau VGG-16 une version du réseau de neurones convolutif, où nous avons modifié ce réseau pour la partie de prédiction. Ensuite on va acheminer notre travail par l'environnement logiciel qui assure la réalisation de notre plateforme électronique et finaliser notre projet par l'obtention de différents résultats de tests.

III.2 Le réseau VGG-16

III.2.1 Définition

Il s'agit d'une structure du Visual Geometry Group d'Oxford réalisée par Andrea Vedaldi et Andrew Zisserman (en 2017). Une version du réseau de neurones convolutif très connu appelé VGG-Net. C'est un réseau de type encodeur-décodeur, adapté aux images de petite taille, pour le nombre et les dimensions des couches de convolution [78]. Avant de nous lancer dans l'implémentation d'un réseau de neurones, nous devons impérativement comprendre son architecture dans les moindres détails. Nous allons donc passer un peu de temps à étudier la configuration des différentes couches de VGG-16 [77].

III.2.2 L'architecture de VGG-16

Le VGG-16 est constitué de plusieurs couches, dont 13 couches de convolution et 3 entièrement connectés. Il doit donc apprendre les poids de 16 couches. Il prend en entrée une image en couleurs de taille 224×224 pixels et la classifie dans une des 1000 classes. Il renvoie donc un vecteur de taille 1000, qui contient les probabilités d'appartenance à chacune des classes. L'architecture du réseau VGG-16 est illustrée par les schémas des **Figure III-1et III-2**.

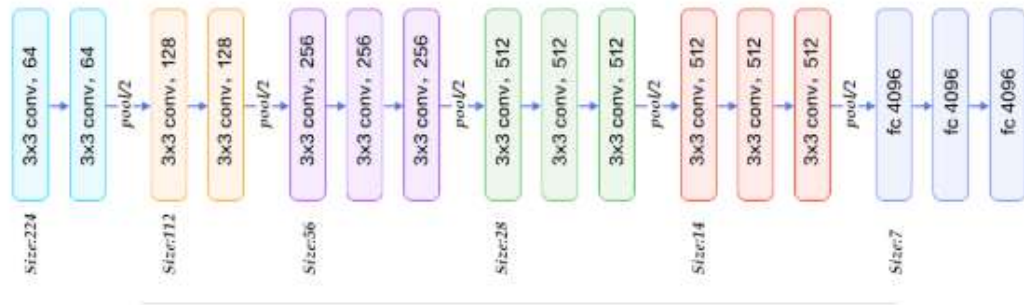


Figure III-1 L'architecture du VGG-net

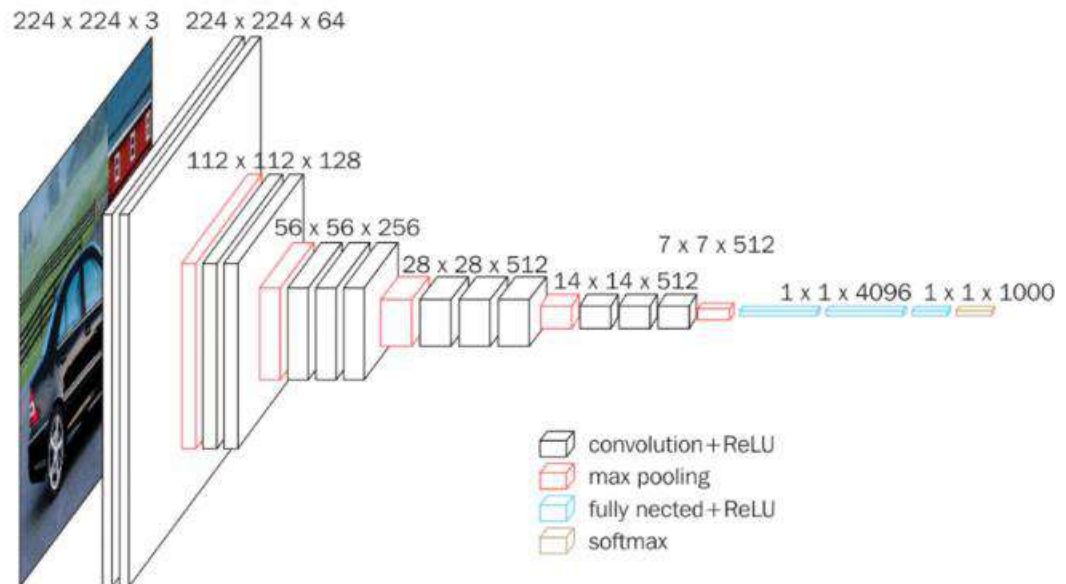


Figure III-2 Représentation 3D de l'architecture du VGG-16

L'entrée de la couche cov1 est d'une image RGB de taille fixe 224 x 224 px. L'image est passée à travers un empilement de couches convolutives (**conv.**), Où les filtres ont été utilisés avec un très petit champ récepteur : 3×3 (qui est la plus petite taille pour capturer la notion de gauche / droite, haut / bas, centre). Dans l'une des configurations, il utilise également des filtres de convolution 1×1 , qui peuvent être considérés comme une transformation linéaire des canaux d'entrée (suivie d'une non-linéarité). La foulée de convolution est fixée à 1 pixel ; le remplissage spatial de **conv.** L'entrée de couche est telle que la résolution spatiale est préservée après convolution, c'est-à-dire que le remplissage est de 1 pixel pour 3×3 (**conv.**). Couches **pooling** spatiale est effectuée par cinq couches de **max pooling**, qui suivent une partie de la **conv** (toutes les couches de **conv** ne sont pas suivies de la couche **pooling** maximale). **Max pooling** est effectuée sur une fenêtre de 2×2 pixels, avec foulée 2. Trois couches entièrement connectées (FC) suivent un empilement de couches convolutives (qui a une profondeur différente dans différentes architectures) : les deux premières ont 4096 canaux chacune, la troisième effectue

une classification ILSVRC à 1000 voies et contient donc 1000 canaux (un pour chaque classe). La dernière couche est la couche ***soft-max***. La configuration des couches entièrement connectées est la même dans tous les réseaux. Malheureusement, VGG-Net présente deux inconvénients majeurs :

1. Il est péniblement lent à s'entraîner.
2. Les poids de l'architecture de réseau sont eux-mêmes assez importants.

III.3 Environnement de développement

Dans cette partie on décrit le matériel sur lequel l'expérimentation a été réalisée ainsi que les logiciels utilisés :

III.3.1 Aspect matériel

Pour faire l'implémentation du système RAPI, nous avons choisi d'utiliser un Raspberry Pi 3 et la caméra Raspberry.

III.3.1.a Raspberry Pi 3

III.3.1.a.1 Description

Le Raspberry Pi est une nano-ordinateur mono carte à processeur ARM de la taille d'une carte de crédit que l'on peut brancher à un écran et utilisé comme un ordinateur standard. Conçu par des professeurs du département informatique de l'université de Cambridge dans le cadre de la fondation Raspberry Pi [80]. Sa petite taille, et son prix intéressant fait du Raspberry Pi un produit idéal pour tester différentes choses. Évidemment, pour sa taille il ne faut pas s'attendre à des performances incroyables, mais pour mettre en ligne des projets à montrer aux clients ou expérimenter avec linux c'est largement suffisant [81]. En le souhaitant pratique à manipuler de par sa taille et son faible poids, peu cher et facile à utiliser tout en préservant une très bonne performance de l'ensemble, il désire offrir aux utilisateurs la possibilité de s'initier à l'informatique de manière ludique et efficace. De plus, le fait qu'il soit livré tel quel impose à l'utilisateur de construire peu à peu son ordinateur en ajoutant au fur et à mesure le matériel adéquat pour le faire fonctionner.

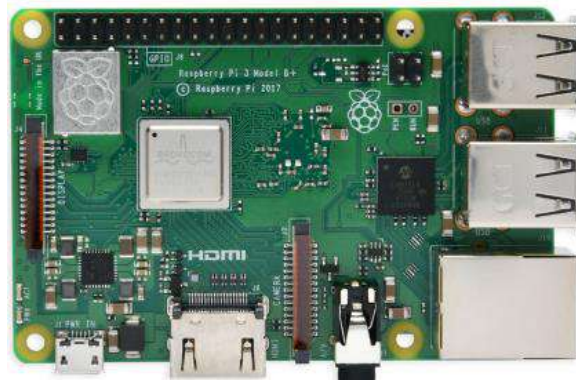


Figure III-3 Le Raspberry Pi3

III.3.1.a.2 Le fonctionnement

Avant de nous lancer dans des cas d'utilisation, il est important de comprendre comment fonctionne un Raspberry Pi. Le Raspberry Pi ne dispose pas d'un disque dur interne (cela augmenterait grandement sa taille) et on stockera les données sur une carte SD. Par défaut, le Raspberry Pi, est nu, comprendre par là qu'il est vendu sans accessoires. Pour pouvoir l'utiliser on doit donc avoir : [82]

- ✓ Une carte micro SD compatible avec le modèle Raspberry Pi.
- ✓ Un câble d'alimentation micro USB standard
- ✓ Un câble RJ45 pour se connecter au réseau
- ✓ Un câble HDMI afin de connecter le Raspberry Pi un écran ou une télévision.
- ✓ Un clavier pour taper les commandes, et éventuellement une souris.

Pour la capture des vidéos et des images nous allons utiliser une caméra Raspberry.

III.3.1.b Caméra Raspberry V1.3

III.3.1.b.1 Description

C'est un module miniature avec une définition de 5 Mégapixels et 1080p en vidéo. La caméra se branche sur le connecteur CSI existant sur la carte Raspberry Pi. Elle convient pour Raspberry Pi modèle A ou B.

III.3.1.b.2 Caractéristique

- Capture Omnivision 5674 avec objectif à focale fixe
- Capteur 5 Mégapixels.
- Résolution photo : 2592 x 1944
- Résolution vidéo maximum : 1080p
- Images par seconde maximum : 30fps

- Taille du module : 20 x 25 x 10mm
- Connexion par câble plat à l'interface 15-pin MIPI Camera Serial Interface (CSI)
(Connecteur S5 du Raspberry Pi)

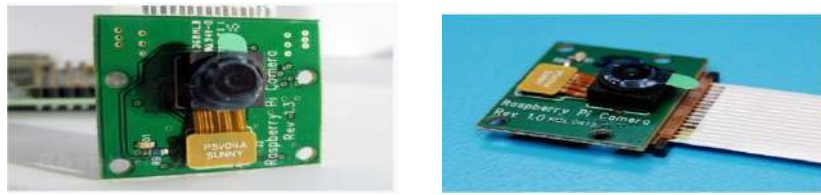


Figure III-4 Caméra Raspberry

III.3.2 Aspect logiciel

Pour la réalisation de ce projet, nous avons utilisé un ensemble d'outils de développements tel que : le langage de programmation Python 3.7, Anaconda (sur PC) et Miniconda (sur Raspberry), OpenCV, TensorFlow et Keras.

III.3.2.a Python

Python est un langage de programmation de haut niveau utilisé pour la programmation générale. Créé par Guido van Rossum et sorti en 1991, c'est un langage interprété. Un script Python n'a pas besoin d'être compilé pour être exécuté, contrairement à des langages comme le C ou le C++. Python a une philosophie de conception qui met l'accent sur la lisibilité du code, notamment en utilisant des espaces importants. Il est gratuit et multiplateformes : Windows, Mac OS X, Linux, Android, iOS, depuis les mini-ordinateurs Raspberry Pi jusqu'aux supercalculateurs. Il fournit des constructions qui permettent une programmation claire à petite et à grande échelle. Python dispose d'un système de type dynamique et d'une gestion automatique de la mémoire. Il prend en charge de multiples paradigmes de programmation, y compris orientés objet, impératifs, fonctionnels et procéduraux, et dispose d'une bibliothèque standard vaste et complète. Les interpréteurs de Python sont disponibles pour de nombreux systèmes d'exploitation [82].

III.3.2.b Anaconda

Anaconda est un environnement gratuit et facile à utiliser pour Python scientifique. L'installation d'un environnement Python complet peut-être une vraie galère. Déjà, il faut télécharger Python et l'installer. Par la suite, télécharger un à un les packages dont on a besoin. Parfois, le nombre de ces librairies peut-être grand.

Pour les besoins du Machine Learning, on a généralement besoin des librairies suivantes :

- ✓ NumPy
- ✓ SciPy
- ✓ Matplotlib
- ✓ Opencv
- ✓ Scikit-learn
- ✓ Keras

III.3.2.c *OpenCV*

Initialement développée par Intel, OpenCV (Open Computer Vision) est une bibliothèque graphique. Elle est spécialisée dans le traitement d'images, que ce soit pour de la photo ou de la vidéo. Sa première version est sortie en juin 2000. Elle est disponible sur la plupart des systèmes d'exploitation et existe pour les langages Python, C++ et Java.

OpenCV fournit un ensemble de plus de 2500 algorithmes de vision par ordinateur, accessibles au travers d'API. Ce qui permet d'effectuer tout un tas de traitements sur des images (extraction de couleurs, détection de visages, de formes, application de filtres,...). Ces algorithmes se basent principalement sur des calculs mathématiques complexes, concernant surtout les traitements sur les matrices (car une image peut être considérée comme une matrice de pixels). [83]

III.3.2.d *Keras et Tensorflow*

III.3.2.d.1 *Keras*

Keras est une bibliothèque de réseaux neuronaux de haut niveau qui fournit une API Machine Learning pratique par-dessus d'autres bibliothèques de bas niveau pour le traitement et la manipulation des tenseurs, appelées Backends. Keras peut être utilisé sur l'un des trois backends disponibles : Tensorflow, Théano et CNTK. Il a été développé dans le but de permettre une expérimentation rapide. Être en mesure de passer de l'idée au résultat le plus rapidement possible est la clé pour faire de la recherche. L'utilisation de Keras est intéressante lorsqu'on a besoin d'une bibliothèque d'apprentissage profond qui :

- Permet un prototypage facile et rapide (grâce à la modularité totale, au minimalisme et à l'extensibilité).
- Prend en charge les réseaux convolutionnels et les réseaux récurrents, ainsi que les combinaisons des deux.

- Prend en charge les schémas de connectivité arbitraires (y compris l'apprentissage à entrées multiples et à sorties multiples).
- Fonctionne de manière transparente sur le processeur et le processeur graphique GPU. [84]

III.3.2.d.2 TensorFlow

Tensorflow est plus qu'un simple cadre d'apprentissage profond. C'est un cadre de calcul général permettant d'effectuer des opérations mathématiques générales de manière parallèle et distribuée. Son nom est notamment inspiré du fait que les opérations courantes sur des réseaux de neurones sont principalement faites via des tables de données multidimensionnelles, appelées Tenseurs (Tensor). Un Tenseur à deux dimensions est l'équivalent d'une matrice. Aujourd'hui, les principaux produits de Google sont basés sur TensorFlow: Gmail, Google Photos, Reconnaissance de voix.

III.3.2.e *Miniconda*

Miniconda est un installateur minimal pour Conda qui permet de gérer des bibliothèques pour développer en python, notamment pour l'analyse de données, telle que Numpy, pandas, dask, la visualisation : matplotlib. C'est une version réduite d'Anaconda qui inclut uniquement Conda, Python, les paquets dont ils dépendent, et un petit nombre d'autres paquets utiles, dont pip, zlib et quelques autres [85].

III.3.2.e.1 La différence entre Anaconda et Miniconda

Anaconda : est une distribution complète de Python qui contient un gestionnaire de paquets très puissant nommé conda. Anaconda installe de très nombreux paquets et outils mais nécessite un espace disque de plusieurs gigaoctets.

Miniconda : est une version allégée d'Anaconda, donc plus rapide à installer et occupant peu d'espace sur le disque dur. Le gestionnaire de paquet conda est aussi présent dans Miniconda. [85]

III.3.2.e.1.1 L'installation de Miniconda sur Raspberry Pi

```
wget http://repo.continuum.io/miniconda/Miniconda3-latest-Linux-armv7l.sh
sudo md5sum Miniconda3-latest-Linux-armv7l.sh
sudo /bin/bash Miniconda3-latest-Linux-armv7l.sh
```

- ✓ Acceptez le contrat de licence avec « yes »
- ✓ Lorsque vous y êtes invité, changez l'emplacement d'installation :

```
/home/pi/miniconda3
```

Souhaitez-vous que l'installateur préfixe l'emplacement d'installation de Miniconda3 sur PATH dans votre/root /.bashrc? yes

✓ Maintenant, ajoutez le chemin d'installation à la variable PATH :

```
sudo nano /home/pi/.bashrc
```

✓ Nous allons à la fin du fichier .bashrc et ajoutons la ligne suivante :

```
export PATH="/home/pi/miniconda3/bin:$PATH"
```

✓ Enregistrez le fichier et quittez.

✓ Pour tester si l'installation a réussi, ouvrez un nouveau terminal et entrez :

```
conda
```

✓ Si vous voyez une liste de commandes, vous êtes prêt à partir.

III.3.2.e.1.2 Ajout de Python 3.5/3.6/3.7 à Miniconda sur Raspberry Pi

✓ Après l'installation de Miniconda, nous ne pouvons pas encore installer Python supérieures à Python 3.4, mais nous avons besoin de Python 3.7. Voici la solution qui a fonctionné pour moi sur mon Raspberry Pi 3 :

✓ D'abord, nous avons ajoutés le gestionnaire de paquets Berryconda de jjhelmus (une sorte de version à jour de la version armv7l de Miniconda) :

```
conda config --add channels rpi
```

➤ Ce n'est que maintenant que nous avons pu installer Python 3.5, 3.6 et 3.7 sans avoir à les compiler nous-mêmes :

```
conda install python=3.5
```

```
conda install python=3.6
```

```
conda install python=3.7
```

➤ Par la suite, nous avons pu créer des environnements avec la version Python ajoutée, par exemple avec Python 3.7 :

```
conda create --name py37 python=3.7
```

➤ Le nouvel environnement "py37" peut maintenant être activé :

```
source activate py37
```

III.4 Résultats d'implémentation

III.4.1 Description de la base de données

Dans la description de la base, nous allons parler sur les chiffres (base de données) utilisée dans la phase de l'apprentissage. Une fois que cette base de données est téléchargée nous l'organisons et structurons dans un répertoire (nommé myData2 dans notre cas) contenant 10 sous-répertoires nommés de 0 à 9 (**Tableau 2**). Chaque sous-répertoire contient un ensemble d'images (plus de 1000 images pour chaque chiffre) pour le même chiffre correspondant au nom du dossier conteneur. Au total, nous avons utilisé 10606 images de 28×28 pixels, dont certaines ont la résolution 30×60 pixels. Les images sont en format JPG binaire, c'est-à-dire que les images sont codées sur deux niveaux seulement en noir et blanc, dont le noir est la couleur de l'arrière-plan.

Tableau 2 Description de la base de données

Chiffre	0	1	2	3	4	5	6	7	8	9
Nombre d'images	1085	1074	1083	1036	1025	1085	1067	1038	1056	1057
Total	10606									

III.4.2 Détection de la plaque d'immatriculation

Maintenant nous passons à la partie expérimentale, dans cette partie nous allons apprendre à utiliser les bibliothèques de base dont nous avons besoin dans la section de mise en œuvre comme Tensorflow et keras qui sont des bibliothèques très intuitives de Deep Learning en Python.

Notre projet est basé sur l'architecture VGG16 qui est considérée comme l'une des excellentes architectures de modèles de vision jusqu'à ce jour. La chose la plus unique à propos de VGG16 est qu'au lieu d'avoir un grand nombre d'hyper-paramètres, ils se sont concentrés sur des couches de convolution. Nous allons d'abord l'implémenter de A à Z, sous une version modifiée, pour découvrir Keras, puis nous allons voir comment l'utiliser pour classifier les chiffres de manière efficace. Pour qu'on puisse utiliser le model VGG-16 dans la reconnaissance des chiffres nous avons introduit les changements suivants :

- Changement de la taille des images d'entrée. Nous avons choisi une taille de (80×80) pixels et un seul canal pour des images binaires.

- Changement du nombre de filtres pour les différentes couches de convolution.
- Elimination de la partie qui permet de compenser la taille du filtre de 3 pixels, une bordure de 1 pixel qui ajouté à l'image d'entrée avant chaque convolution pour que la sortie de la convolution soit de même taille, à partir de la commande suivant :

```
model.add(ZeroPadding2D((1,1)))
```

III.4.2.a Architecture de VGG-16 modifié

Le VGG-16 est constitué de plusieurs couches, dont 13 couches de convolution et 3 entièrement connectés. Il doit donc apprendre les poids de 16 couches. Il prend en entrée une image en couleurs de taille 80×80 pixels et la classifie dans une des 10 classes (chiffres de 0 à 9). Il renvoie donc un vecteur de taille 10, qui contient les probabilités d'appartenance à chacune des classes. L'architecture de VGG-16 modifié est illustrée par le schéma ci-dessous :

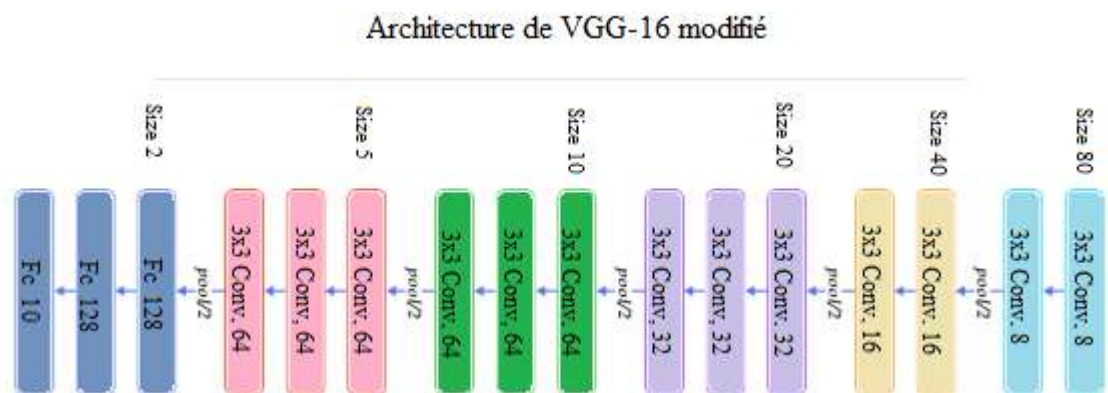


Figure III-5 Architecture du réseau VGG-16 modifié

L'entrée de la couche cov1 est d'une image RVB de taille fixe 80×80 . L'image est passée à travers une pile de couches convolutives (conv.), Où les filtres ont été utilisés avec un très petit champ réceptif 3×3 (qui est la plus petite taille pour capturer la notion de gauche / droite, haut / bas, centre).

La **pooling** spatiale est effectuée par cinq couches de **pooling** max, qui suivent une partie de la conv (toutes les couches de conv. ne sont pas suivies de la mise en pool maximale). La **pooling** maximale est effectuée sur une fenêtre de 2×2 pixels, avec foulée 2.

Trois couches entièrement connectées (FC) suivent un empilement de couches convolutives (qui a une profondeur différente dans différentes architectures) les deux premières ont 128 canaux chacune, la troisième effectue une classification ILSVRC renvoie le vecteur de

probabilités de taille 10 (le nombre de classes un pour chaque classe). La dernière couche est la couche **soft-max**. La configuration des couches entièrement connectées est la même dans tous les réseaux.

III.4.2.b Importation des bibliothèques

Nous avons implémenté le VGG16 complet à partir de zéro dans Keras. Cette implémentation sert à classifier l'ensemble de données (chiffres) des plaques d'immatriculation algérienne. Nous commençons par le script ci-dessous :

```
1  # -*- coding: utf-8 -*-
2
3  import keras,os,cv2  #importer les paquets nécessaires
4  from keras.models import Sequential
5  from keras.layers import Dense,Dropout, Conv2D, MaxPool2D , Flatten
6  from keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
7  from sklearn.preprocessing import LabelEncoder
8  from sklearn.model_selection import train_test_split
9  import glob
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from keras.utils import to_categorical
```

Ici, Nous importons d'abord toutes les bibliothèques et les paquets (packages) requis. De brèves descriptions et leur fonction sont illustrées ci-dessous :

- **Cv2** : bibliothèque de Computer Vision, également appelée OpenCV, que nous utiliserons pour réaliser des différentes opérations en traitement d'images.
- **Numpy** : une bibliothèque qui prend en charge les tableaux multidimensionnels et les matrices.
- **Matplotlib** : Une bibliothèque qui prend en charge le tracé et la visualisation de nos données.
- **Local utils** : un script python comprend des fonctions qui seront utilisées pour traiter les données dans le réseau Wpod-Net.
- **os.path / glob** : package/bibliothèque d'interface du système d'exploitation pour python. Nous les utiliserons pour gérer les répertoires et le système de fichiers.
- **keras.models** : Nous utiliserons le package `model_from_json` de cette bibliothèque pour charger l'architecture de notre modèle au format JSON.

Nous avons le modèle séquentiel. Le modèle séquentiel signifie que toutes les couches du modèle seront disposées en séquence. Ici, nous allons importer :

1. « ImageDataGenerator, load_img » et « img_to_array » à partir de « keras.preprocessing »:

- L'objectif d'ImageDataGenerator est d'importer facilement des données avec des étiquettes dans le modèle. C'est une classe très utile car elle a de nombreuses fonctions pour redimensionner, faire pivoter, zoomer, retourner etc.
- La fonction « load_img » permet de charger l'image et de la redimensionner
- « img_to_array » permet de convertir l'image chargée en tableau numpy.
- 2. « LabelEncoder » à partir de « sklearn.preprocessing » : cette fonction permet de normaliser et encodez les étiquettes cibles avec une valeur comprise entre 0 et n_classes.
- 3. « train_test_split » à partir de « sklearn.model_selection » qui consiste une utilitaire rapide qui encapsule la validation d'entrée.
- 4. « to_categorical » à partir de keras, permet de convertir les classes des chiffres (0 à 9) en un vecteur de classe qui ne doit contenir que des éléments dans [0, 1].
- 5. La commande suivant « os.environ['CUDA_VISIBLE_DEVICES'] = '-1' » permet de forcer l'exécution du programme dans le CPU.

III.4.2.c *Création du modèle de prédiction des chiffres*

Dans une première étape, nous avons construit notre modèle de prédiction des chiffres en se basant sur le réseau VGG-16 modifié :

- ✓ Lignes 16 → 28 : Organisez les données d'entrée et leurs étiquettes correspondantes. La taille d'origine de l'image d'entrée pour le modèle de VGG-16 est de 80x80 comme indiqué à la ligne 21. Cette configuration a réussi à atteindre une «accuracy» d'environ 99%.
- ✓ Lignes 33 → 40 : Convertissez nos étiquettes sous forme de tableau 1D en étiquettes d'encodage à chaud. Cela nous donne une meilleure représentation de nos classes. L'enregistrement d'étiquettes des classes doit être sauvegardé localement (ligne 40), il peut donc être utilisé ultérieurement pour la transformation inverse (prédiction).
- ✓ Lignes 41 → 43 : Divisez l'ensemble de données en ensemble d'entraînement (80%) et ensemble de validation (20%). Cela nous permet de surveiller la précision de notre modèle et d'éviter le sur-apprentissage.
- ✓ Lignes 44 → 52 : Créez une méthode d'augmentation de données avec des techniques de transformation de base telles que la rotation, le décalage, le zoom, etc. Néanmoins, la rotation est à utiliser avec prudence, car le nombre «6» inversé verticalement nous donnerait le numéro «9».


```

16 X=[]
17 labels=[]
18
19 for image_path in dataset_paths:
20     label = image_path.split(os.path.sep)[-2]
21     image=load_img(image_path,target_size=(80,80))
22     image=img_to_array(image)
23
24     X.append(image)
25     labels.append(label)
26
27 X = np.array(X,dtype="float16")
28 labels = np.array(labels)
29
30 print("[INFO] Find {:d} images with {:d} classes".format(len(X),len(set(labels))))
31
32 # perform one-hot encoding on the labels
33 lb = LabelEncoder()
34 lb.fit(labels)
35 labels = lb.transform(labels)
36 y = to_categorical(labels)
37
38 # save label file so we can use in another script
39 np.save('license_character_classes.npy', lb.classes_)
40
41 # split 10% of data as validation set
42 (trainX, testX, trainY, testY) = train_test_split(X, y, test_size=0.20, stratify=y, random_state=42)
43
44 # generate data augmentation method
45 image_gen = ImageDataGenerator(rotation_range=10,rescale=1./255,
46                                width_shift_range=0.1,
47                                height_shift_range=0.1,
48                                shear_range=0.1,
49                                zoom_range=0.1,
50                                fill_mode="nearest"
51                                )

```

Une fois la base de données est préparée pour l'apprentissage, on définit l'architecture de notre réseau en spécifiant les ses différentes couches.

```

model = Sequential()
model.add(Conv2D(input_shape=(80,80,1),filters=8,kernel_size=(3,3),padding="same", activation="relu"))
model.add(Conv2D(filters=8,kernel_size=(3,3),padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=16, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=16, kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=32, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=32, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=32, kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))
model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))
model.add(Flatten())
model.add(Dense(units=128,activation="relu"))
model.add(Dense(units=128,activation="relu"))
model.add(Dense(units=10, activation="softmax"))

```

La première ligne définit un modèle séquentiel. D'autres architectures plus compliquées existent, mais le réseau VGG16 est un empilement de couches dont chaque couche prend en entrée le résultat de la couche précédente.

```

model.add(Conv2D(input_shape=(80,80,1),filters=8,kernel_size=(
3,3),padding="same", activation="relu"))

```

Le réseau VGG16 prend en entrée des images de taille 224 par 224, or les images des chiffres de la base de données utilisée ont une taille plus petite, c'est pour ça que nous avons fixé une taille de 80x80 pixels. Les lignes :

```
model.add(Conv2D(filters=8, kernel_size=(3, 3), padding="same",  
activation="relu"))
```

Définissent des couches de neurones ReLU avec une taille de filtre de 3×3 pixels et un nombre de canaux de 8, 16, 32 ou 64. Les lignes :

```
model.add(MaxPool2D(pool_size=(2, 2), strides=(2, 2)))
```

Définissent les étapes de réduction de la taille de l'image le long du réseau. La méthode MaxPooling est utilisée et chaque étape divise la taille par 2.

Nous ajoute également l'activation RELU (Rectified Linear Unit) à chaque couche afin que toutes les valeurs négatives ne soient pas transmises à la couche suivante.

```
75 model.add(Flatten())  
76 model.add(Dense(units=128, activation="relu"))  
77 model.add(Dense(units=128, activation="relu"))  
78 model.add(Dense(units=10, activation="softmax"))
```

Après avoir créé toutes les convolutions nous passons les données à la couche **dense** après avoir les convertir en 1-D (model.add(flatten())). Les différentes couches **dense** sont les suivantes :

→ 1 x couche **dense** de 128 unités

→ 1 x couche **dense** de 128 unités

→ 1 x couche **Softmax** dense de 10 unités

Après ces étapes, le modèle est enfin prêt. Maintenant, nous devons compiler le modèle.

```
80 from keras.optimizers import Adam  
81 opt = Adam(lr=0.001)  
82 model.compile(optimizer=opt, loss=keras.losses.categorical_crossentropy, metrics=['accuracy'])  
83  
84 model.summary()
```

Ici, pour atteindre de bonnes performance tout en entraînant le modèle nous utiliserons l'optimiseur **Adam**. En suit, nous pouvons réaliser et vérifier le résumé du modèle que nous avons créée en utilisant le code ci-dessous.

```
model.summary ()
```

La sortie pour cela sera le résumé du modèle que nous venons de créer :

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 80, 80, 8)	80
conv2d_2 (Conv2D)	(None, 80, 80, 8)	584
max_pooling2d_1 (MaxPooling2)	(None, 40, 40, 8)	0
conv2d_3 (Conv2D)	(None, 40, 40, 16)	1168
conv2d_4 (Conv2D)	(None, 40, 40, 16)	2320
max_pooling2d_2 (MaxPooling2)	(None, 20, 20, 16)	0
conv2d_5 (Conv2D)	(None, 20, 20, 32)	4640
conv2d_6 (Conv2D)	(None, 20, 20, 32)	9248
conv2d_7 (Conv2D)	(None, 20, 20, 32)	9248
max_pooling2d_3 (MaxPooling2)	(None, 10, 10, 32)	0
conv2d_8 (Conv2D)	(None, 10, 10, 64)	18496
conv2d_9 (Conv2D)	(None, 10, 10, 64)	36928
conv2d_10 (Conv2D)	(None, 10, 10, 64)	36928
max_pooling2d_4 (MaxPooling2)	(None, 5, 5, 64)	0
conv2d_11 (Conv2D)	(None, 5, 5, 64)	36928
conv2d_12 (Conv2D)	(None, 5, 5, 64)	36928
conv2d_13 (Conv2D)	(None, 5, 5, 64)	36928
max_pooling2d_5 (MaxPooling2)	(None, 2, 2, 64)	0
flatten_1 (Flatten)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32896
dense_2 (Dense)	(None, 128)	16512
dense_3 (Dense)	(None, 10)	1290
Total params: 281,122		
Trainable params: 281,122		
Non-trainable params: 0		

III.4.2.d Construction et évaluation du modèle

Après la création du modèle, nous pouvons maintenant commencer à former notre modèle. En fonction des spécifications de l'environnement d'exécution, on peut augmenter ou

diminuer la valeur « BATCH_SIZE ». Fondamentalement, une taille de « BATCH_SIZE » plus grande signifie que moins d'échantillons sont entraînés à chaque époque, ce qui peut diminuer les performances du modèle mais réduire les coûts de calcul.

```
86 BATCH_SIZE = 64
87 #EPOCHS=30
88
89 from keras.callbacks import ModelCheckpoint, EarlyStopping
90 checkpoint = ModelCheckpoint("VGG16.h5", monitor='val_accuracy', verbose=1, save_best_only=True, save_weights_only=False,
91                             mode='auto', period=1)
92 early = EarlyStopping(monitor='val_accuracy', min_delta=0, patience=5, verbose=1, mode='auto')
93
94 hist = model.fit_generator(steps_per_epoch=100, generator=image_gen.flow(trainX, trainY, batch_size=BATCH_SIZE),
95                             validation_data= (testX, testY), validation_steps=10, epochs=100, callbacks=[checkpoint, early])
96
97 # save model architecture
98 model.save('D:/CodeTest/VGG16.h5')
```

Afin d'éviter de perdre du temps avec un apprentissage insuffisant, nous avons utilisé une fonction de rappel « EarlyStopping » (ligne 90) pour surveiller et arrêter le processus d'entraînement si « val_accuracy » ne s'améliore pas après 5 époques, tandis que « ModelCheckpoint » (ligne 89) n'enregistre pas les poids de notre modèle à chaque fois que si « val_accuracy » est améliorée. Le modèle a acquis une bonne précision après 5 époques (d'environ 95%) et à la fin du processus d'apprentissage, il a atteint une précision d'environ 99% (**Figure III-6**) sur l'ensemble de validation.

Ensuite nous avons enregistré l'architecture du modèle dans le chemin spécifié comme le montre la ligne au-dessous :

```
model.save ('D : /CodeTest/VGG16/VGG16.h5')
```

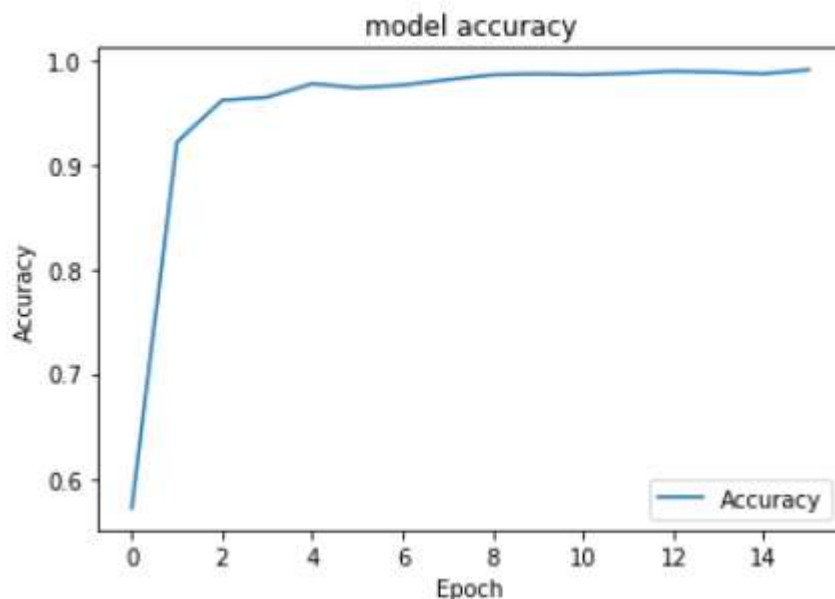


Figure III-6 Précision d'apprentissage du modèle VGG-16 modifié

III.4.2.e Utilisation du modèle dans le système de RAPI

Pour faire des prédictions sur le modèle formé (reconnaissance des chiffres), nous devons charger le meilleur modèle enregistré. En premier lieu, nous devons détecter puis segmenter la plaque d'immatriculation, pour cela nous avons utilisé un modèle à partir du modèle pré-entraîné. Ce modèle a été créé par Sergio Silva. Dans le code de source ci-après nous pouvons trouver deux fichiers (wpod-net.h5 (poids enregistrés du réseau) et « wpod-net.json » (architecture du modèle).

```
13 #%%
14 def load_model(path):
15     try:
16         path = splitext(path)[0]
17         with open('%s.json' % path, 'r') as json_file:
18             model_json = json_file.read()
19             model = model_from_json(model_json, custom_objects={})
20             model.load_weights('%s.h5' % path)
21             print("Loading model successfully...")
22             return model
23     except Exception as e:
24         print(e)
```

La détection de la plaque d'immatriculation se fait à l'aide de la fonction suivante :

```
def get_plate(image_path):
    Dmax = 608
    Dmin = 288
    vehicle = preprocess_image(image_path)
    ratio = float(max(vehicle.shape[:2])) / min(vehicle.shape[:2])
    side = int(ratio * Dmin)
    bound_dim = min(side, Dmax)
    _, LpImg, _, cor = detect_lp(wpod_net, vehicle, bound_dim, lp_threshold=0.5)
    return LpImg, cor
```

Cette fonction reçoit en paramètre un chemin vers l'image du véhicule, et renvoie la plaque d'immatriculation « LpImg » et ses coordonnées « cor ». L'image du véhicule subit un prétraitement qui consiste en un dé-bruitage, une conversion de couleur et une normalisation, selon le code suivant :

```
def preprocess_image(image_path,resize=False):
    imgg = cv2.imread(image_path)
    img = cv2.fastNlMeansDenoisingColored(imgg, None, 10, 10, 7, 15)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = img / 255
    return img
```

Il faut mentionner que Wpod-Net est capable de détecter uniquement les plaques standard (caractère noir, fond blanc), et si l'image est trop floue ou si la plaque est masquée par des obstacles, le modèle risque de ne pas la prédire.

Dans l'étape suivante, nous pouvons dessiner un cadre avec les coordonnées obtenues de la plaque détectée. La fonction « draw_box » permet d'encadrer une zone, indiquée par ses coordonnées dans une image, comme illustré au dessous en utilisant le code suivant:

```
43 def draw_box(image_path, cor, thickness=12):
44     pts=[]
45     x_coordinates=cor[0][0]
46     y_coordinates=cor[0][1]
47
48     for i in range(4):
49         pts.append([int(x_coordinates[i]),int(y_coordinates[i])])
50
51     pts = np.array(pts, np.int32)
52     pts = pts.reshape((-1,1,2))
53     vehicle_image = preprocess_image(image_path)
54
55     cv2.polylines(vehicle_image,[pts],True,(0,255,0),thickness)
56     plt.imshow(vehicle_image)
57     plt.show()
58     return vehicle_image
```



Figure III-7 Détection et encadrement de la plaque d'immatriculation

Une fois la plaque est détectée, on passe à la segmentation des chiffres. Dans un premier temps, l'image de la plaque est conditionnée avant la segmentation, ce conditionnement comprend un filtrage, une binarisation et une dilatation selon le code suivant :

```

# Scales, calculates absolute values, and converts the result to 8-bit.
plate_image = cv2.convertScaleAbs(LpImg[0], alpha=(255.0))
# convert to grayscale and blur the image
gray = cv2.cvtColor(plate_image, cv2.COLOR_BGR2GRAY)
blur = cv2.bilateralFilter(gray,8,5,5)#cv2.GaussianBlur(gray,(5,5),0)#
# Applied inversed thresh_binary
binary = cv2.threshold(blur, 180, 255,
                      cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]
nh,nw=np.shape(binary)
binary1=(binary-binary)
binary1[15:nh-15,20:nw-20]=binary[15:nh-15,20:nw-20]
## Applied dilation
kernel3 = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
thre_mor = cv2.morphologyEx(binary1, cv2.MORPH_DILATE, kernel3)

# Create sort_contours() function to grab the contour of each digit from left to right
_,cont, _ = cv2.findContours(binary1, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

```

Comme peut-on le remarquer, la plaque est recadrée afin d'éviter une segmentation erronée au limites de la plaque. Nous avons couper 15 pixels de chaque coté dans la hauteur et 20 pixels de chaque coté dans la largeur. Un exemple de segmentation de la plaque est illustré sur la **Figure III-8**:



Figure III-8 Segmentation de la plaque d'immatriculation par détection des contours

Après détection des contours et segmentation de la plaque, nous pouvons découper chaque chiffre dans la plaque. Un contour peut contenir un chiffre si le rapport de sa hauteur sur sa largeur est compris entre 1 et 6, sinon le contour ne peut contenir un chiffre et par conséquent sera ignoré. Pour la plaque de la **Figure III-8**, les chiffres sont encadrés et représentés sur la **Figure III-9**.



Figure III-9 Encadrement des chiffres détectés dans une plaque d'immatriculation

La dernière étape consiste à prédire les chiffres détectés dans l'étape précédente. Pour cela, il suffit d'isoler chaque chiffre seul (connaissant ses coordonnées dans l'image binaire), et l'envoyer à notre modèle de reconnaissance préétabli (le VGG16 modifié). Pour l'exemple précédent, le code suivant affiche la chaîne : 0939311226


```

144     #fig = plt.figure(figsize=(15,3))
145     plt.imshow(test_roi)
146     plt.tight_layout(True)
147     plt.show()
148     cols = len(crop_characters)
149     final_string = ''
150     for i,character in enumerate(crop_characters):
151         #fig.add_subplot(grid[i])
152         title = np.array2string(predict_from_model(character,model,labels))
153         #plt.title('{}'.format(title.strip("'[""),fontsize=20))
154         final_string+=title.strip("'["")
155         #final_string+=title
156
157     print("Le numéro de la plaque est: ", final_string)
158     #plt.savefig('final_result.png', dpi=300)
159

```



III.4.3 Implémentation sur Raspberry Pi 3

III.4.3.a Organigramme générale

Tout d'abord, nous pouvons indiquer les principales étapes de notre projet dans la partie expérimentale sur Raspberry Pi 3 qui illustrée dans l'organigramme suivant :

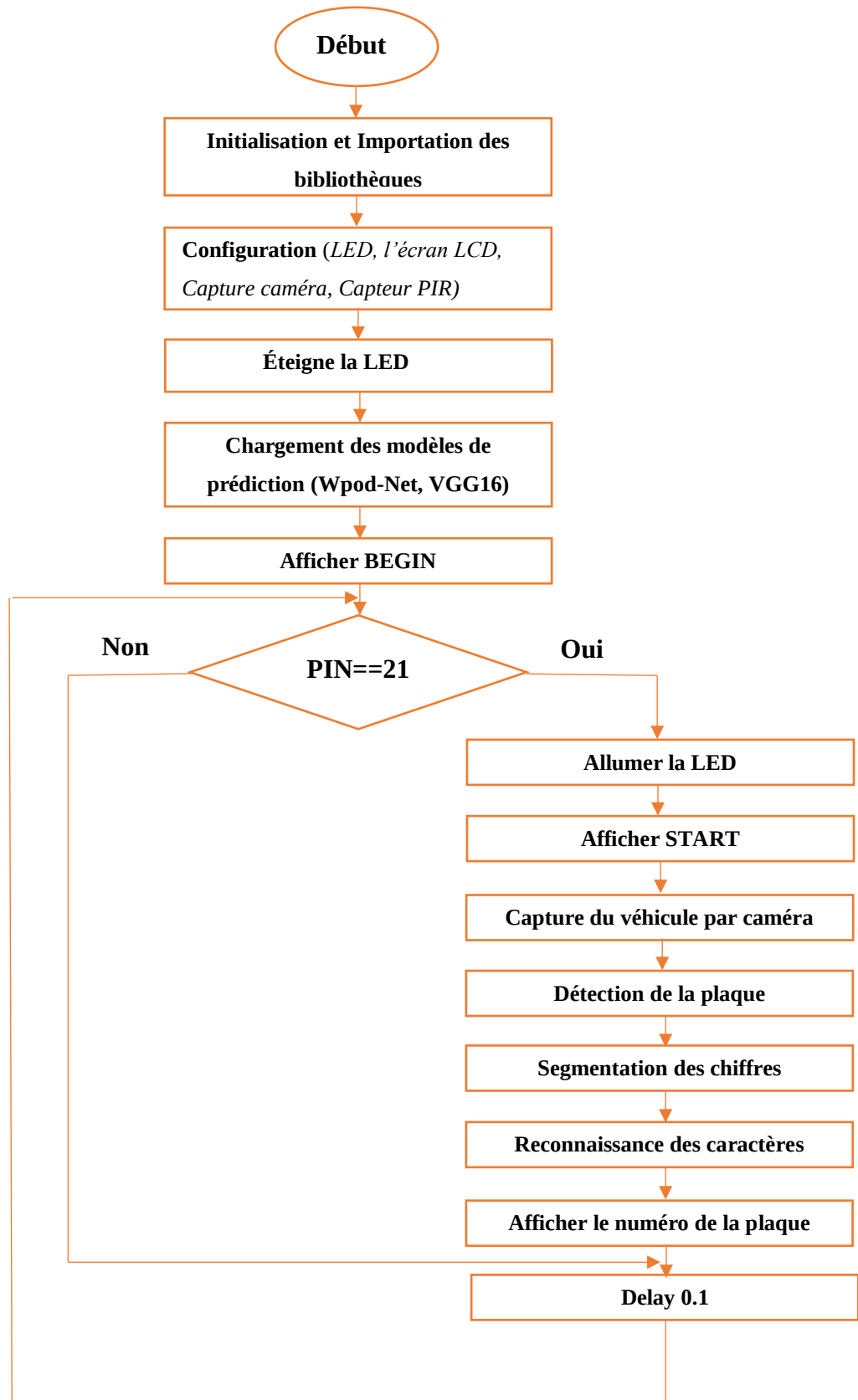


Figure III-10 L'organigramme de reconnaissance des caractères

Dans cette partie, l'image de la voiture est capturée par la caméra Raspberry utilisant code suivant :

```
camera = PiCamera()
camera.resolution = (640,480)
camera.start_preview()
sleep(5)
camera.capture('Plate_examples/Vehicule.jpg')
camera.stop_preview()
images = glob.glob("Plate_examples/*.jpg")
```

La résolution de la caméra est ajustée à 640×480 pour minimiser le temps de traitement. L'image capturée est sauvegardée pour son traitement ultérieur dans la même manière décrite dans la partie III.4.2.e. Les différents résultats obtenus sur un exemple réel (capture par caméra et reconnaissance de la plaque) sont illustrés sur les **Figure III-7,8 et 9**.

III.4.3.b Le capteur de mouvement PIR

Les capteurs PIR sont également connus sous le nom de capteurs infrarouges passifs. Ils sont conçus à partir d'une gamme de capteurs à semi-conducteurs fabriqués dans du matériel pyroélectrique générant une tension quand ils sont exposés à la chaleur. Ils sont utilisés dans des applications de détection thermique, et comprennent des capteurs de sécurité et des détecteurs de mouvement [86].

III.4.3.c Utilisation d'un détecteur de mouvement sur Raspberry Pi

Le GPIO Raspberry Pi est accessible via un programme Python. Chaque broche du Raspberry Pi est nommée en fonction de son ordre (1, 2, 3, ...40) comme indiqué dans la **Figure III-11** ci-dessous :



Raspberry Pi B Rev 2 P1 GPIO Header				Raspberry Pi B+ B+ J8 GPIO Header			
	Pin No.			Pin No.			
3.3V	1	2	5V	1	2	5V	
GPIO2	3	4	5V	GPIO2	3	5V	
GPIO3	5	6	GND	GPIO3	5	GND	
GPIO4	7	8	GPIO14	GPIO4	7	GPIO14	
GND	9	10	GPIO15	GND	9	GPIO15	
GPIO17	11	12	GPIO18	GPIO17	11	GPIO18	
GPIO27	13	14	GND	GPIO27	13	GND	
GPIO22	15	16	GPIO23	GPIO22	15	GPIO23	
3.3V	17	18	GPIO24	3.3V	17	GPIO24	
GPIO10	19	20	GND	GPIO10	19	GND	
GPIO9	21	22	GPIO25	GPIO9	21	GPIO25	
GPIO11	23	24	GPIO8	GPIO11	23	GPIO8	
GND	25	26	GPIO7	GND	25	GPIO7	
Key				Key			
Power +			UART	GPIO5	29	30	GND
GND			SPI	GPIO6	31	32	GPIO12
PC			GPIO	GPIO13	33	34	GND
				GPIO19	35	36	GPIO16
				GPIO26	37	38	GPIO20
				GND	39	40	GPIO21

Figure III-11 Les broches de GPIO Raspberry PI

Ici, nous utilisons un capteur de mouvement PIR. Ce capteur de mouvement se compose d'une lentille de Fresnel, d'un détecteur infrarouge et d'un circuit de détection de présence. La lentille du capteur focalise tout rayonnement infrarouge présent autour d'elle vers le détecteur infrarouge. Lorsqu'un corps génère de la chaleur infrarouge et, par conséquent, cette chaleur est captée par le capteur de mouvement. Le capteur émet un signal 5V pendant une durée d'une minute dès qu'il détecte la présence d'un objet. Il offre une plage de détection d'environ 6 à 7 mètres et est très sensible. Lorsque le capteur de mouvement PIR détecte une personne, il envoie un signal 5V au Raspberry Pi via son GPIO et nous définissons ce que le Raspberry Pi doit faire lorsqu'il détecte un objet via le codage Python [86]. Le capteur est alimenté par l'une des broches 5V du Raspberry Pi. En plus du connecteur à 3 broches, il y a 2 potentiomètres pour ajuster le temps de retard et la sensibilité, comme indique **la Figure III-12** :

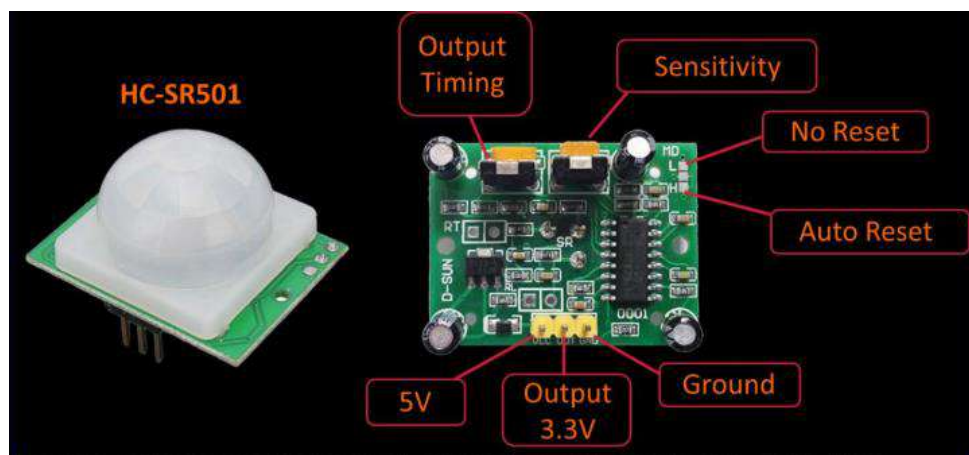


Figure III-12 Les boutons de réglage du capteur de mouvement H-SR501C(PIR)

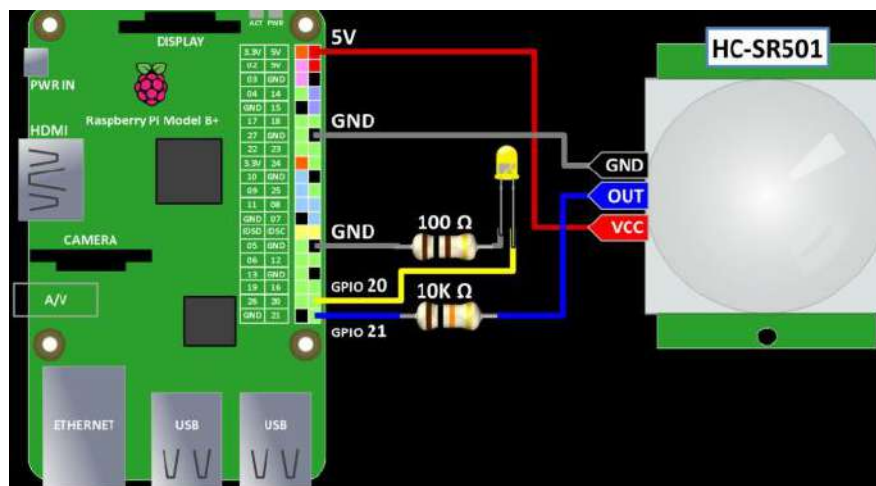


Figure III-13 Câblage du capteur de mouvement HC-SR501

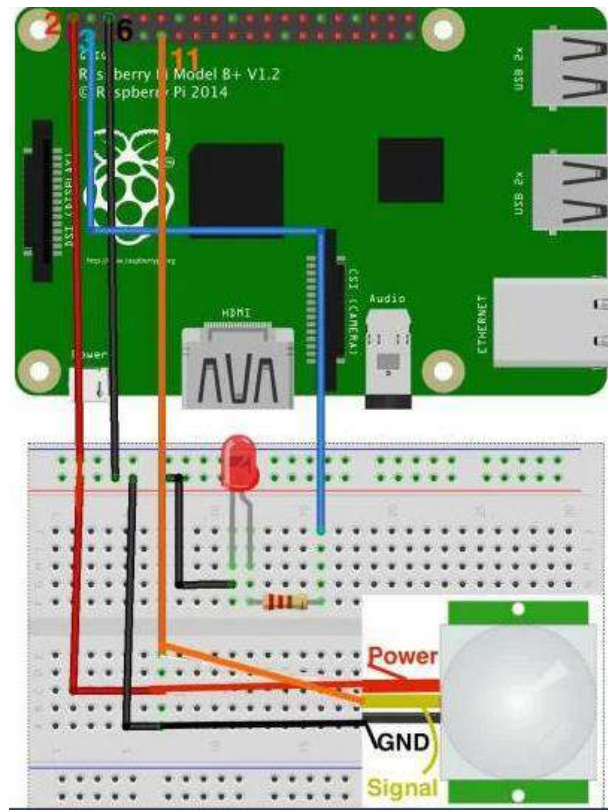


Figure III-14 Connexion du capteur de mouvement PIR Raspberry Pi

III.4.3.d L'écran LCD 16 × 2 ?

Le terme LCD signifie affichage à cristaux liquides. Il s'agit d'un type de module d'affichage électronique utilisé dans une large gamme d'applications telles que divers circuits et appareils tels que les téléphones mobiles, les calculatrices, les ordinateurs, les téléviseurs, etc. L'écran LCD 16 x 2 est l'une des unités d'affichage les plus populaires parmi les amateurs, les étudiants et même les professionnels de l'électronique. Prend en charge 16 caractères par ligne et contient deux de ces lignes (**Figure III-15**). Un module LCD 16 x 2 se compose généralement de 16 broches et 2 lignes [87].



Figure III-15 L'écran LCD 16 × 2

Le brochage (PIN) de l'écran LCD 16×2 est illustré ci-dessous : [87]

Pin1 (Ground / Source Pin) : Il s'agit d'une broche GND de l'écran, utilisée pour connecter la borne GND de l'unité de microcontrôleur ou de la source d'alimentation.

Pin2 (VCC / Source Pin) : Il s'agit de la broche d'alimentation en tension de l'écran, utilisée pour connecter la broche d'alimentation de la source d'alimentation.

Pin3 (V0 / VEE / Control Pin) : Cette broche régule la différence de l'affichage, utilisée pour connecter un POT modifiable qui peut fournir 0 à 5V.

Pin4 (Register Select / Control Pin) : Cette broche bascule entre le registre de commande ou de données, utilisé pour connecter une broche d'unité de microcontrôleur et obtient 0 ou 1 (0 = mode de données et 1 = mode de commande).

Pin5 (Read/Write/Control Pin) : cette broche fait basculer l'affichage entre les opérations de lecture ou d'écriture, et elle est connectée à une broche d'unité de microcontrôleur pour obtenir 0 ou 1 (0 = opération d'écriture et 1 = opération de lecture).

Pin 6 (Enable/Control Pin) : cette broche doit être maintenue haute pour exécuter le processus de lecture / écriture, et elle est connectée à l'unité de microcontrôleur et constamment maintenue haute.

Pin 7-14 (Data Pins) : ces broches sont utilisées pour envoyer des données à l'écran. Ces broches sont connectées en modes à deux fils comme le mode 4 fils et le mode 8 fils. En mode 4 fils, seules quatre broches sont connectées à l'unité de microcontrôleur comme 0 à 3, tandis qu'en mode 8 fils, 8 broches sont connectées à l'unité de microcontrôleur comme 0 à 7.

Pin15 (+Ve pin of the LED) : Cette broche est connectée à + 5V

Pin 16 (-Ve pin of the LED) : Cette broche est connectée à GND.

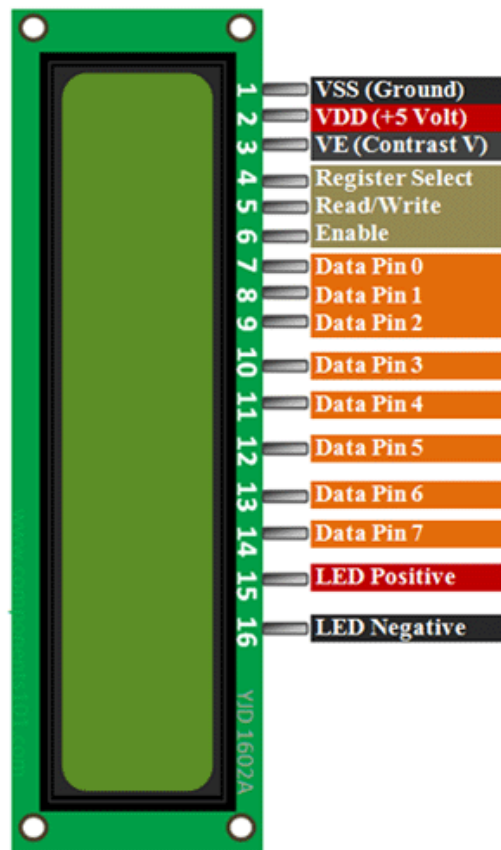


Figure III-16 Diagramme LCD-16x2-pin

III.4.3.e Connexion de l'écran LCD avec Raspberry Pi 3

Pour utiliser l'écran 16x02 avec un Raspberry, il faut : Un Raspberry Pi, Un écran 16x02, Des câbles, Un potentiomètre 10K Ω . Il existe deux façons de connecter l'écran LCD à notre Raspberry Pi 3, en mode 4 bits ou en mode 8 bits. Le mode 4 bits utilise 6 broches GPIO (comme le montre dans **la Figure III-17** au-dessous), tandis que le mode 8 bits en utilise 10. Comme il utilise moins de broches, le mode 4 bits est la méthode la plus courante.

Chaque caractère et chaque commande sont envoyés à l'écran LCD sous forme d'octet (8 bits) de données. En mode 8 bits, l'octet est envoyé en une seule fois via 8 fils de données, un bit par fil. En mode 4 bits, l'octet est divisé en deux ensembles de 4 bits - les bits supérieurs et les bits inférieurs, qui sont envoyés l'un après l'autre sur 4 fils de données.

Théoriquement, le mode 8 bits transfère les données environ deux fois plus vite que le mode 4 bits, puisque l'octet entier est envoyé en une seule fois. Cependant, le pilote LCD prend un temps relativement long pour traiter les données, donc quel que soit le mode utilisé, nous ne remarquons pas vraiment de différence de vitesse de transfert de données entre les modes 8 bits et 4 bits [87].

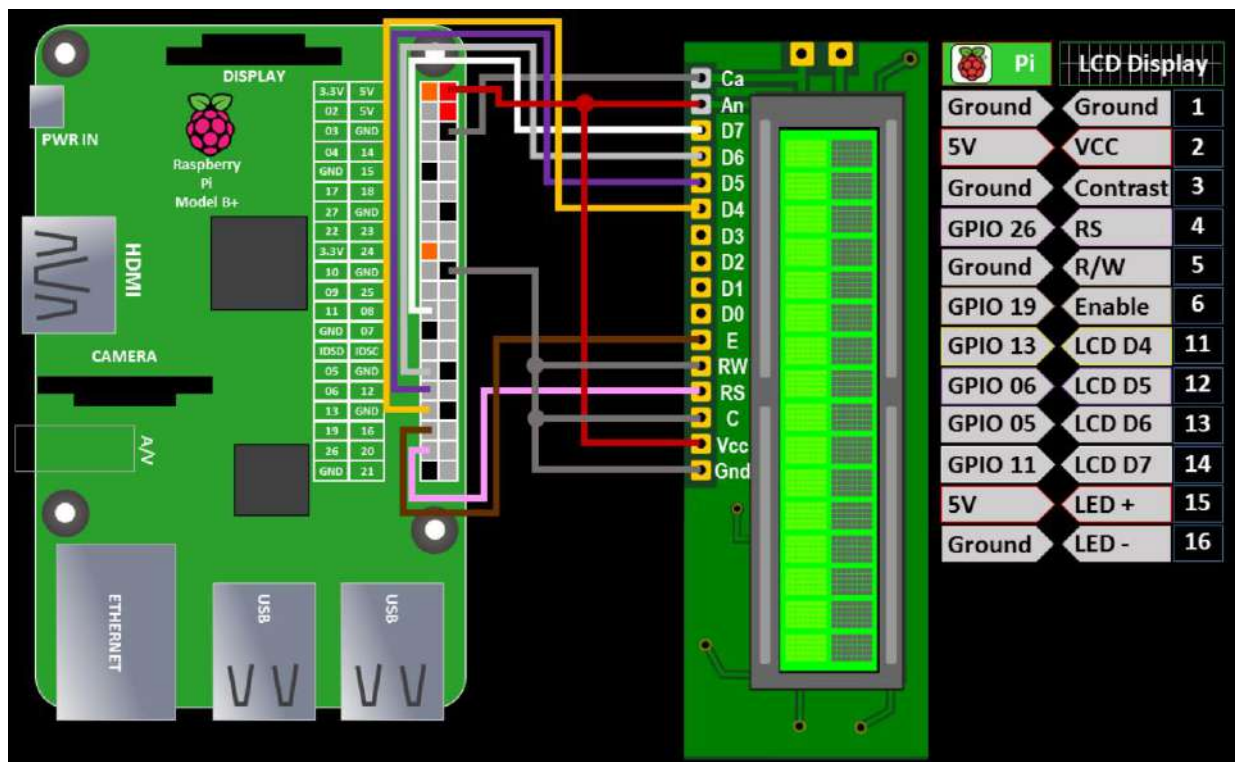
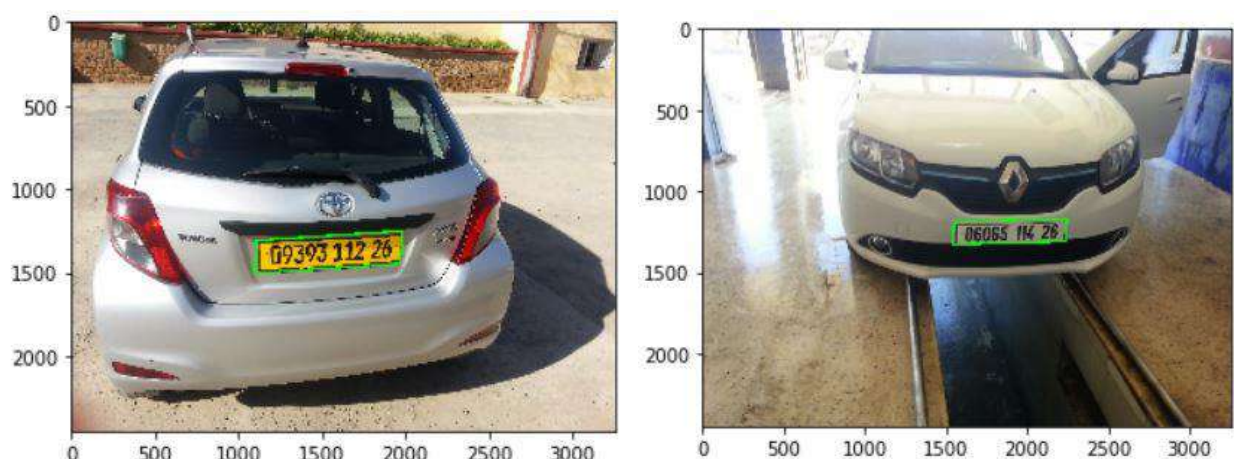


Figure III-18 Interface LCD 16x2 avec Raspberry PI 3

III.4.4 Résultats finaux

III.4.4.a Résultats d'implémentation de système RAPI sur le PC

Les résultats finaux ont été obtenus après l'exécution de deux programmes (VGG 16 modifié, CodeSource) sur l'environnement Spyder. Pour analyser les résultats obtenues en utilisant un ensemble d'images de voitures dans différents formes et cas pour pouvoir évaluer les performances de la méthode réalisée, les résultats sont illustrés sur les figures au-dessous :



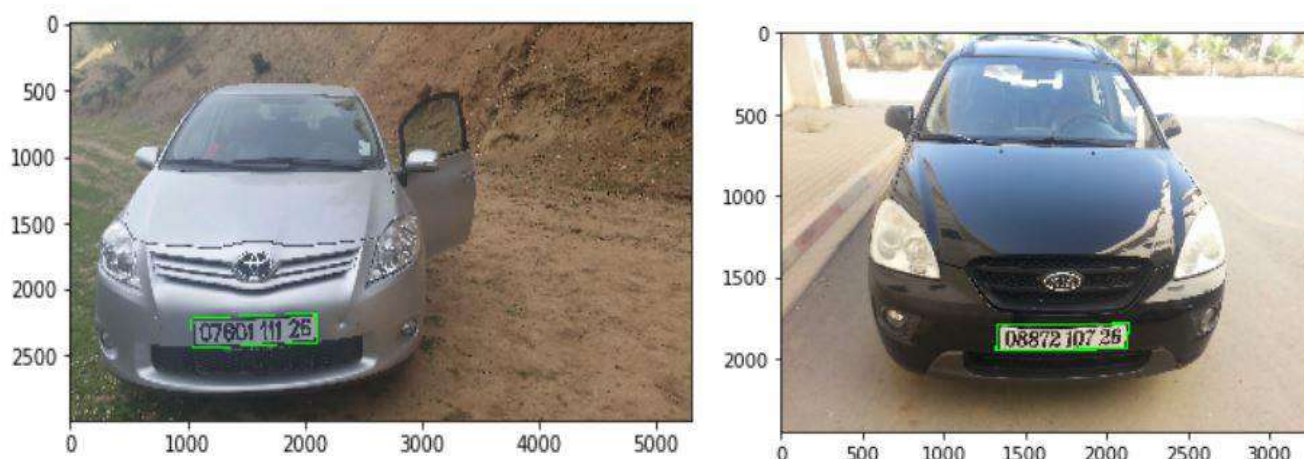


Figure III-19 Résultats de détection des plaques d'immatriculation

- ✓ Une fois que l'image du véhicule est obtenue et prétraiter (détecter), l'étape suivante consiste à extraire la plaque d'immatriculation, comme le montre dans les exemples suivants :



Figure III-20 Résultats de l'extraction des plaques d'immatriculation

- ✓ Étant leur forme rectangulaire, le fait d'identifier les bords de ce rectangle est un moyen d'extraire des plaques d'immatriculation, lorsque la plaque est détectée et extraite, on passe à la segmentation des chiffres selon le contour. La plaque est recadrée afin d'éviter une segmentation erronée aux limites de la plaque. Pour cela nous avons coupé 15 pixels de chaque côté dans la hauteur et 20 pixels de chaque côté dans la largeur, comme illustrée au-dessous :



Figure III-21 Segmentation des plaques selon le contour

- ✓ La plaque une fois détectée, subira un ensemble de traitements dans le but de la segmenter en caractères isolés, de sorte que chaque chiffre dans la plaque d'immatriculation soit découpé et encadré :

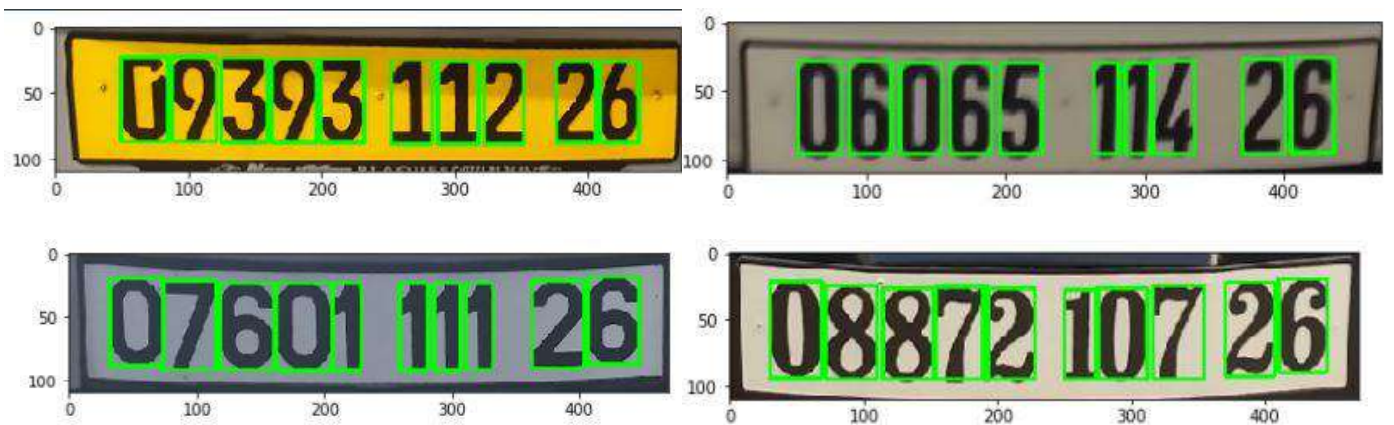


Figure III-22 Segmentation et encadrement des plaques d'immatriculation

- ✓ Après la segmentation des caractères, la dernière étape consiste à faire une prédiction des chiffres détectés, nous pouvons reconnaître tous les caractères lorsque chaque chiffre isolé envoyer à le modèle de reconnaissance sous forme des chaines de caractères, comme illustrée dans les résultats suivants :

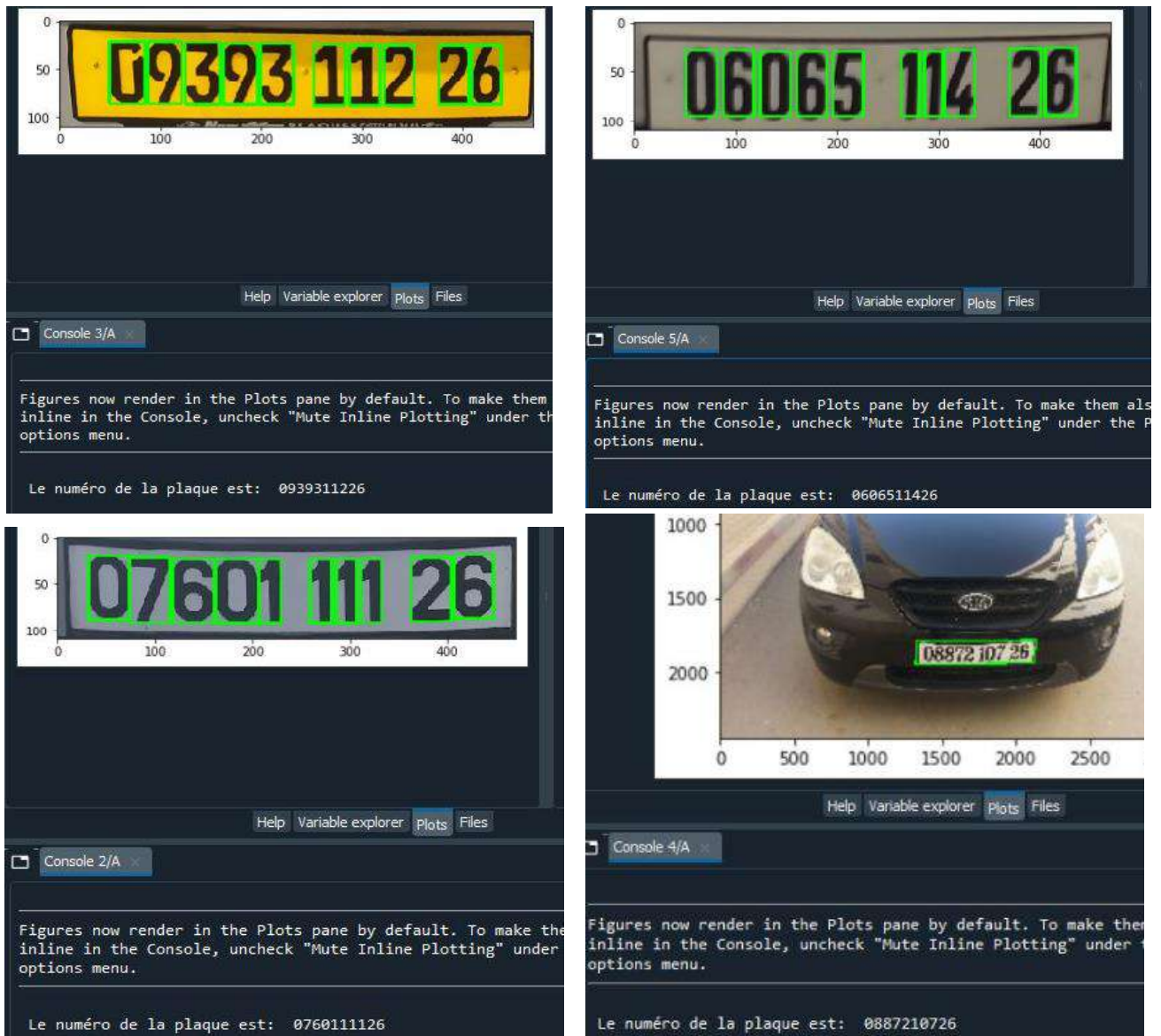


Figure III-23 Résultats de la reconnaissance des caractères

III.4.4.b Résultats d'implémentation sur le Raspberry Pi3

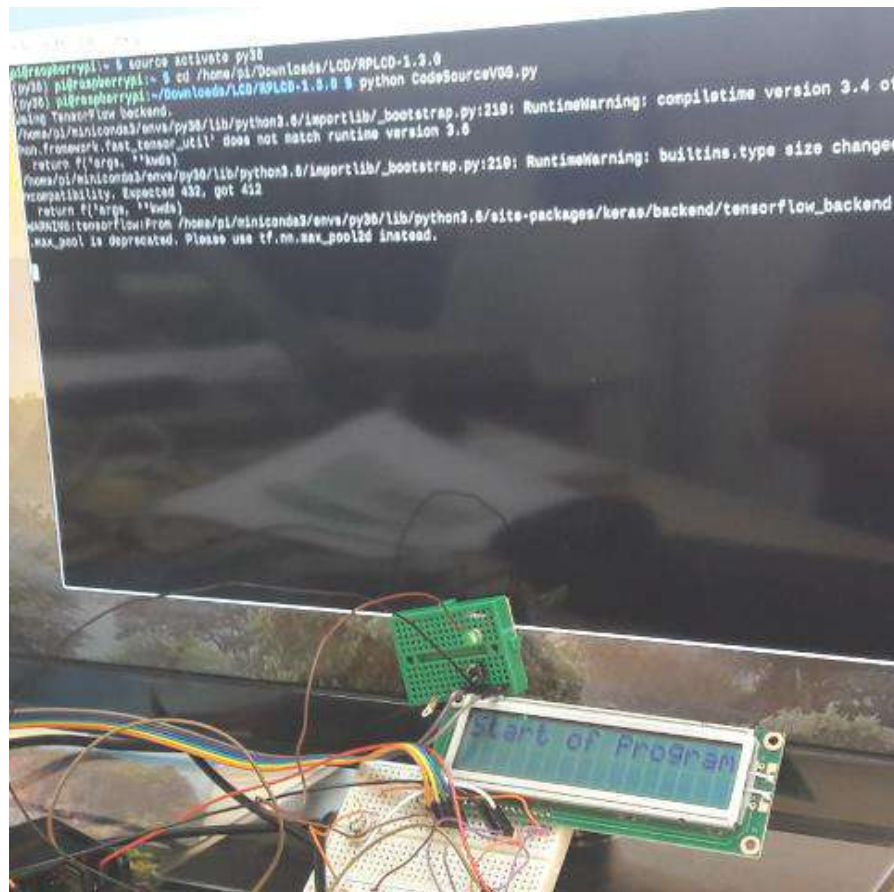
Finalement nous pouvons réaliser notre projet et le tester sur le Raspberry Pi 3. Pour fonctionner de manière pratique on utilisant le capteur PIR, LED, l'écran LCD. On garde le Raspberry Pi éteint jusqu'à ce que le circuit soit connecté pour éviter de court-circuiter accidentellement des composants. Au début, nous devons activer la source et l'environnement de code Python en entrant les commandes suivantes :

Source activate py36

Cd (ajouter le répertoire du programme exécutable CodeSourceVGG.py)

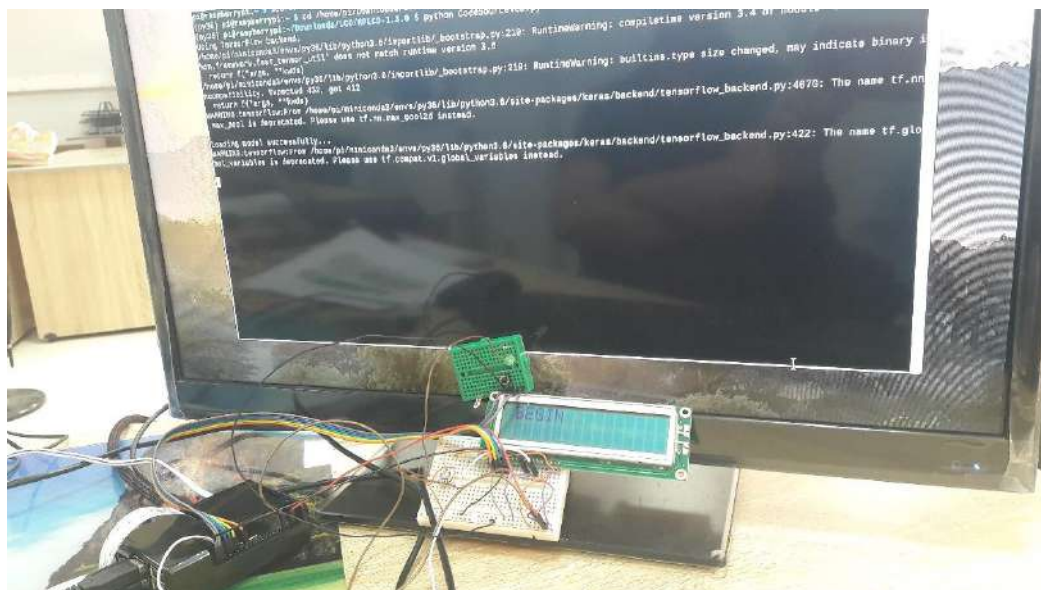
Python CodeSourceVGG.py

On obtient les résultats suivant :



Dans la première partie, le programme prend du temps à s'exécuter en affichant « Start Of Program » sur l'écran LCD.

Lorsque LCD s'affiche « BEGIN » cela signifie que le chargement du modèle est prêt :



En attente d'appuyer sur le bouton poussoir (ce bouton est utilisé pour simuler le fonctionnement du capteur PIR) pour lancer le traitement. La LED s'allumera. En ce moment l'écran LCD il s'affiche « START » et les tâches suivantes sont exécutées :

- 1- Acquisition de l'image : le module caméra capture l'image du véhicule.
- 2- Détection de la plaque.
- 3- Segmentation de la plaque.
- 4- Reconnaissance de la plaque et affichage du résultat sur LCD.



Figure III-24 Début de traitement

Finally, the LCD screen displays the number of the detected license plate, (Figure III-25) :

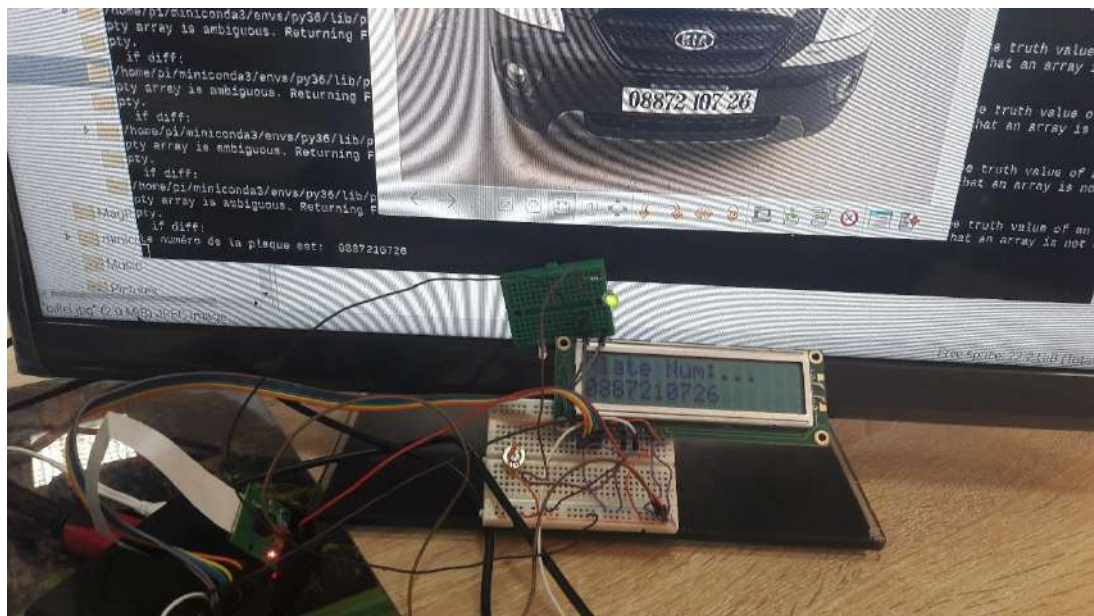


Figure III-25 Résultat de traitement

III.5 Conclusion

Dans ce chapitre nous avons présenté un aperçu sur le réseau VGG 16 et sa version modifiée que nous avons adoptée pour la reconnaissance des chiffres, nous devons impérativement comprendre son architecture dans les moindres détails avant de commencer la mise en œuvre de notre système de RAPI. Une description détaillée du hardware/software a été explorée dans ce chapitre. Les résultats d'implémentation montrent que le système proposé a réussi de détecter la plaque d'immatriculation et de lire son contenu à l'aide du modèle VGG16 adapté à la reconnaissance des chiffres. Cependant, un effort sur l'optimisation reste nécessaire pour réduire le temps de traitement et permettre l'intégration du système proposé dans divers applications telle que la sécurité routière et le contrôle des frontières.

CONCLUSION GENERALE

Le travail exposé dans ce mémoire représente le fruit d'une recherche dans le domaine de la reconnaissance des plaques d'immatriculation, suivi d'une implémentation de tel système sur une plateforme embarquée «Raspberry Pi 3», dans lequel nous nous sommes intéressés à extraire la zone de la plaque en utilisant le réseau Wpod-net et à identifier par la suite le matricule correspondant. Dans une première étape, nous avons exposé les notions de base sur le traitement d'image dont on a utilisé plusieurs techniques et concepts de bases dans ce domaine : acquisition d'image, filtrage et dé-bruitage, conversion de couleurs, détection des contours, etc. Globalement, le système RAPI utilise deux techniques de traitement d'image : la détection d'objet pour repérer les plaques d'immatriculation suivie de la reconnaissance des caractères.

Dans une deuxième étape, nous avons parlé sur l'apprentissage profond qui a prouvé son efficacité dans le domaine de traitement d'image pour identifier et traiter le contenu de l'image en basant sur des méthodes optimisées comme les réseaux de neurones convolutifs. L'apprentissage profond est devenu très populaire récemment car il a résolu efficacement certains problèmes qui ont défié l'intelligence artificielle. De point de vue implémentation, nous avons détaillé l'architecture du réseau VGG-16 et proposé une version modifiée dédiée à la reconnaissance des chiffres. Lors de la réalisation, nous implémenté le système en utilisant le langage de programmation Python avec l'environnement Anaconda et l'installateur Miniconda qui permet de gérer les bibliothèques Keras et Tensorflow et la bibliothèque de vision par ordinateur OpenCV, ce qui a facilité le procédé d'implémentation du système sur un Raspberry Pi 3.

Les résultats obtenus peuvent être jugé comme étant acceptables, pour embarquer sur un Raspberry, un système complexe traitant une multitude de données pour localiser et segmenter les caractères d'une plaque d'immatriculation.

Comme perspective, et pour arriver à améliorer ce travail nous pouvons citer qu'il est possible d'introduire des optimisations sur le software afin de réduire le temps d'exécution et faire fonctionner le système en temps réel. Une autre solution possible consiste à utiliser une clé d'Intel Neural Compute Stick 2 (NCS2) pour accélérer la prédiction et diminuer le temps de traitement.

BIBLIOGRAPHIE

- [1] AMEUR CHHAYDER ET IMENE BELHADJ MOHAMED IPEIS BP 3018 SFAX TUNISIE
« SYSTEME DE RECONNAISSANCE AUTOMATIQUE DES PLAQUES MINERALOGIQUES »,
PP.1-2, SETIT2009
- [2] LINGRAND DIANE, " COURS DE TRAITEMENT D'IMAGES", RAPPORT DE RECHERCHE,
LABORATOIRE I3S, UNIVERISTE DE NICE, JANVIER 2004.
- [3] MAÏTINE BERGOUNIOUX, " INTRODUCTION AU TRAITEMENT MATHEMATIQUE DES IMAGES
METHODES DETERMINISTES", EDITION SPRINGER, 2015.
- [4] PAR JEAN-MARIE MUNGUAKONKWA BIRINGANINE ISP (INSTITUT SUPERIEUR
PEDAGOGIQUE DE BUKAVU) - LICENCIE EN PEDAGOGIE, OPTION : INFORMATIQUE DE
GESTION 2008
- [5] M. HADALLAH, « CODAGE DES IMAGES FIXES PAR METHODES DES HYBRIDE BASEE SUR LA
QV ET LES APPROXIMATIONS :
- [6] MICROSOFT, « ENCYCLOPEDIE ENCARTA », 2005.
- [7] M. KUNT, « TRAITEMENT NUMERIQUE DES IMAGES », VOL.2, 1993
- [8] MOISE MWEZE, COMPRESSION DES IMAGES, TFC/ISP BUKAVU, 2003-2004
- [9] R.C. GONZALEZ ET P. WINTZ, « DIGITAL IMAGE PROCESSING », ADDISON WESLEY »
- [10] A. D'HARDANCOURT, « FOU DU MULTIMEDIA » SYBEX 1995
- [11] R.C. GONZALES ET P. WINTZ, « DIGITAL IMAGE PROCESSING », ADDITION WESSLEY »,
1997
- [12] M. KUNT, « TRAITEMENT NUMERIQUE DES IMAGES », VOL.2, 1993
- [13] SAMAMBA TONY, RECONNAISSANCE DES FORMES COMME OUTIL D'AIDE AUX
TRAITEMENTS D'IMAGE. CAS DES EMPREINTES DIGITALES, MEMOIRE ISP/BUKAVU,
2005-2006.
- [14] [HTTP://WWW.LIBRARY.CORNELL.EDU/PRESERVATION/TUTORIAL-](http://www.library.cornell.edu/preservation/tutorial-french/presentation/presentation-08.html)
FRENCH/PRESENTATION/PRESENTATION- 08.HTML, «DIDACTICIEL D'IMAGERIE
NUMERIQUE », DEPARTEMENT DE RECHERCHES, BIBLIOTHEQUE DE L'UNIVERSITE
CORNELL, 2000-2003.
- [15] [HTTP ://WWW.IPT34.COM/10-FORMATS-IMAGE-NUMERIQUE](http://www.ipt34.com/10-formats-image-numerique), 12/12/2015.
- [16] VINCENT MAZET, "TRAITEMENTS ET OUTILS DE BASE", COURS DE LA MATIERE OUTILS
FONDAMENTAUX EN TRAITEMENTS D'IMAGES, UNIVERSITE DE STRASBOURG, 2018-
2019.
- [17] NOR IMANE ET SIDHOUM SOUAD, "DEVELOPPEMENT D'UNE APPLICATION DEDETECTION
ET DE RECONNAISSANCE DE PLAQUES D'IMMATRICULATION(LAPIA)", MEMOIRE

- [18] NAGA SURYA SANDEEP ANGARA, « AUTOMATIC LICENSE PLATE RECOGNITION USING DEEP LEARNING TECHNIQUES » p.15-16, (2015). ELECTRICAL ENGINEERING THESES.
- [19] SIMON LUDOVIC, "SYSTEMES ET EQUIPEMENTS DE LECTURE AUTOMATIQUE DE PLAQUES D'IMMATRICULATION DES VEHICULES PRINCIPES ET APERÇU DES APPLICATIONS", RAPPORT D'ETUDES, SETRA, MINISTERE DE L'ECOLOGIE – DU DEVELOPPEMENT DURABLE ET DE L'ENERGIE – FRANCE, MAI 2013.
- [20] AMEUR CHHAYDER ET IMENE BELHADJ MOHAMED, "SYSTEME DE RECONNAISSANCE AUTOMATIQUE DES PLAQUES MINERALOGIQUES", COMMUNICATION, 5TH INTERNATIONAL CONFERENCE: SCIENCES OF ELECTRONIC, TECHNOLOGIES OF INFORMATION AND TELECOMMUNICATIONS, TUNISIE, MARS 2009.
- [21] J.F CANNY "A COMPUTATIONAL APPROACH TO EDGE DETECTION", IEEE TRANSACTION ON PATTERN ANALYSIS AND MACHINE INTELLIGENT, NOVEMBER 1986.
- [22] BAI, H LIU C. "A HYBRID LICENSE PLATE EXTRACTION METHOD BASED ON EDGE STATISTICS AND MORPHOLOGY IN: PROCEEDINGS OF THE 17TH INTERNATIONAL CONFERENCE ON PATTERN RECOGNITION" VOL 2, PP, 831, 23-26 AUG 2004.
- [23] BEATRIZ, A "EXPERIMENTS IN IMAGE SEGMENTATION FOR AUTOMATIC US LICENSE PLATE RECOGNITION." DISSERTATION, VIRGINIA POLYTECHNIC INSTITUTE AND STATE UNIVERSITY 2004.
- [24] BROUMANDNIA A, FATHY M. "APPLICATION OF PATTERN RECOGNITION FOR FARSI LICENSE PLATE RECOGNITION" IN VISION AND IMAGE PROCESSING (GVIP) DECEMBER 2005.
- [25] CHACON I, ZIMMERMAN A "LICENSE PLATE LOCATION BASED ON A DYNAMIC PCNN SCHEMA " IN : PROCEEDINGS OF THE INTERNATIONAL JOINT CONFERENCE ON NEURAL NETWORKS, VOL 2 PP 1195-1200, 20-24 JULY 2003.
- [26] DAI, Y., MA, H., LIU, J., LI, L: "A HIGH PERFORMANCE LICENSE PLATE RECOGNITION SYSTEM BASED ON THE WEB TECHNIQUE". IN: PROCEEDINGS OF IEEE INTELLIGENT TRANSPORTATION SYSTEMS, PP. 325–329 (2001).
- [27] DAVID, L., ANDRES, M., VICENTE, P., JUAN, V.: "CAR LICENSE PLATES EXTRACTION AND RECOGNITION BASED ON CONNECTED COMPONENTS ANALYSIS AND HMM DECODING". IN: IBPRIA, VOL. 1, PP. 571–578 (2005).
- [28] DLAGNEKOV, L., "VIDEO-BASED CAR SURVEILLANCE: LICENSE PLATE MAKES, AND MODEL RECOGNITION". DISSERTATION, U.C. SAN DIEGO. ADABOOST AND A DETECTOR CASCADE. IN: NIPS, VOL. 14 (2002).
- [29] ELLIMAN, G., LANXASTER, T : "A REVIEW OF SEGMENTATION AND CONTEXTUAL ANALYSIS TECHNIQUES FOR TEXT RECOGNITION". IN: PATTERN RECOGNITION, VOL. 23, No. 3337–346 (1990).
- [30] J.F CANNY "A COMPUTATIONAL APPROACH TO EDGE DETECTION", IEEE TRANSACTION ON PATTERN ANALYSIS AND MACHINE INTELLIGENT, NOVEMBER 1986
- [31] J.SERRA, "IMAGE ANALYSIS AND MATHEMATICAL MORPHOLOGY", ACADEMIC PRESS, LONDON, 1982.

- [32] JXIFAN, S., WEIZHONG, Z., YONGHANG, S., "AUTOMATIC LICENSE PLATE RECOGNITION SYSTEM BASED".
- [33] MATAS, J., ZIMMERMANN, K., "UNCONSTRAINED LICENCE PLATE DETECTION", IN: PROCEEDINGS OF THE 8TH INTERNATIONAL IEEE CONFERENCE ON INTELLIGENT TRANSPORTATION SYSTEMS.PP. 572–577, ISBN 07803-9216-7, WIEN AUSTRIA (2005)
- [34] NIJHUIS, J., BRUGGE, M., HELMHOLT, K., PLUIM, J., SPAANENBURG, L., VENEMA, R., WESTENBERG M., "CAR LICENSE PLATE RECOGNITION WITH NEURAL NETWORKS AND FUZZY LOGIC", IN: PROCEEDINGS OF IEEE INTER-NATIONAL CONFERENCE ON NEURAL NETWORKS, VOL. 5, PP. 2232–2236(1995).
- [35] OTSU, N: A THRESHOLD SELECTION METHOD FROM GRAY LEVEL HISTOGRAMS. IN: IEEE
- [36] P. A. MLSNA AND J. J. RODRIGUEZ, "GRADIENT AND LAPLACIAN TYPE EDGE DETECTION", IN HANDBOOK OF IMAGE AND VIDEO PROCESSING, ACADEMIC PRESS (2002)).
- [37] PARKER, J., FEDERL, P., "AN APPROACH TO LICENSE PLATE RECOGNITION". IN: COMPUTER SCIENCE TECHNICAL REPORTS, UNIVERSITY OF CALGARY, ALBERTA CANADA, 591–11, OCTOBER (1996).
- [38] SHYANG-LIH, C., LI-SHIEN, C., YUN-CHUNG, C., SEI-WAN, C. AUTOMATIC LICENSE PLATE RECOGNITION. IN: IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, VOL.5, NO.1, PP.42–53(2004). TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS, VOL.SMC-9, PP. 62–66 (1979).
- [39] H. ERDINC KOCER ET K. KURSAT CEVIK, « ARTIFICIAL NEURAL NETWORKS BASED VEHICLE LICENSE PLATE RECOGNITION », PROCEEDIA COMPUT. SCI., VOL. 3, P. 1033-1037, 2011, A STATE-OF-THE-ART REVIEW." IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY 23.2 (2012): 311-325.
- [40] L. ZHENG, X. HE, B. SAMALI, ET L. T. YANG, « AN ALGORITHM FOR ACCURACY ENHANCEMENT OF LICENSE PLATE RECOGNITION », J. COMPUT. SYST. SCI., VOL. 79, NO 2, P. 245-255, MARS 2013.
- [41] J. LIU, J. XIANG, ET F. LIAO, « LICENSE PLATE LOCATION BASED ON TEXTURE ANALYSIS AND PROJECTION APPROACH », IN ADVANCED MATERIALS RESEARCH, 2013, VOL. 790, P. 479-483.
- [42] PHIL KIM 2017 1 P. KIM, MATLAB DEEP LEARNING, DOI 10.1007/978-1-4842-2845-6_1.
- [43] SARAH GUIDO, ANDREAS MÜLLER « LE MACHING LEARNING AVEC PYTHON »PARIS – FRANCE, PUBLIE ET VENDU AVEC L'AUTORISATION DE O'REILLY MEDIA [2017-2018]
- [44] AMBROISE ET GOVAERT, 2000.
- [45] MACHINE LEARNING, CHAP. 13 REINFORCEMENT LEARNING, P. 367-390.
- [46] SINNO JIALIN PAN ET QIANG YANG, « A SURVEY ON TRANSFER LEARNING », IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, VOL. 22, NO 10, OCTOBRE 2010, P. 1345-1359 (ISSN 1041-4347, E-ISSN 1558-2191, DOI 10.1109/TKDE.2009.191, LIRE EN LIGNE [ARCHIVE]).
- [47] UNIVERSITE DE SIDI BEL ABBES - DJILLALI LIABES, DIF, NASSIMA, RESUME .

- [48] [HTTPS://MAKINA-CORPUS.COM/BLOG/METIER/2017/INITIATION-AU-MACHINE-LEARNING-AVEC-PYTHON-THEORIE](https://makina-corpus.com/blog/metier/2017/initiation-au-machine-learning-avec-python-theorie) .
- [49] THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN, BY F. ROSENBLATT, PSYCHOLOGICAL REVIEW, VOL. 65, PP. 386 - 408, 1958).
- [50] BACKPROPAGATION THROUGH TIME : WHAT IT DOES AND HOW TO DO IT, BY P. J. WERBOS, PROCEEDINGS OF THE IEEE, VOL. 78, PP. 1550 - 1560, 1990, AND A FAST LEARNING ALGORITHM FOR DEEP BELIEF NETS, BY G. E. HINTON, S. OSINDERO, AND Y. W. TEH, NEURAL COMPUTING, VOL. 18, PP. 1527 - 1554, 2006).
- [51] THE ROOTS OF BACKPROPAGATION: FROM ORDERED DERIVATIVES TO NEURAL NETWORKS AND POLITICAL FORECASTING, NEURAL NETWORKS, BY S. LEVEN, VOL. 9, 1996 AND LEARNING REPRESENTATIONS BY BACKPROPAGATING ERRORS, BY D. E. RUMELHART, G. E. HINTON, AND R. J. WILLIAMS, VOL. 323, 1986).
- [52] PRINCIPE DE FONCTIONNEMENT D'UN NEURONE FORMEL
[HTTPS://WWW.EDITIONSENI.FR/OPEN/MEDIABOOK.ASPX?IDR=2DAC0EC804E7E66899C47F486D228FF0](https://www.editionseni.fr/open/mediabook.aspx?idR=2DAC0EC804E7E66899C47F486D228FF0)
- [53] FRANÇOIS BLAYO ET MICHEL VERLEYSEN, LES RESEAUX DE NEURONES ARTIFICIELS, PUF, QUE SAIS-JE NO 3042 [ARCHIVE], 1RE ED., 1996.
- [54] ASU SCHOOL OF LIFE SCIENCES.
- [55] JEAN-PAUL HATON, MODELES CONNEXIONNISTES POUR L'INTELLIGENCE ARTIFICIELLE, 1989.
- [56] [HTTPS://WWW.RESEARCHGATE.NET/FIGURE/MODELE-DU-NEURONE-FORMEL-DE-MAC-CULLOCH-ET-PITTS-AVEC-BIAIS-IL-SAGIT-DE-REALISER_FIG1_328996656](https://www.researchgate.net/figure/MODELE-DU-NEURONE-FORMEL-DE-MAC-CULLOCH-ET-PITTS-AVEC-BIAIS-IL-SAGIT-DE-REALISER_FIG1_328996656). UPLOADED BY SOUFYANE CHEKROUN .
- [57] [HTTPS://WWW.LEBIGDATA.FR/RESEAU-DE-NEURONES-ARTIFICIELS-DEFINITION](https://www.lebigdata.fr/reseau-de-neurones-artificiels-definition).
- [58] [HTTPS://WWW.RESEARCHGATE.NET/FIGURE/STRUCTURE-DUN-NEURONE-ARTIFICIEL-LE-NEURONE-CALCULE-LA-SOMME-DE-SES-ENTREES-PUIS-CETTE_FIG14_29640044](https://www.researchgate.net/figure/STRUCTURE-DUN-NEURONE-ARTIFICIEL-LE-NEURONE-CALCULE-LA-SOMME-DE-SES-ENTREES-PUIS-CETTE_FIG14_29640044).
- [59] S.ZEGHLACHE, " COMMANDE INTELLIGENTE ", COURS, UNIVERSITE MOHAMED BOUDIAF-M'SILA.2008.
- [60] OUSSAR Y. [1998], RESEAUX D'ONDELETTES ET RESEAUX DE NEURONES POUR LA MODELISATION STATIQUE ET DYNAMIQUE DE PROCESSUS, THESE DE DOCTORAT DE L'UNIVERSITE PIERRE ET MARIE CURIE, PARIS.
- [61] SLIMI Wafa « COMMANDE D'UN FOUR ELECTRIQUE PAR RESEAU DE NEURONE ». UNIV-ANNABA P.14. 21.22.23. 24. [2018].
- [62] PETITJEAN, GERALD. COURS, RESEAUX DE NEURONES : " INTRODUCTION AUX RESEAUX DE NEURONES " 102P.
- [63] P. COMON, « CLASSIFICATION SUPERVISEE PAR RESEAUX MULTICOUCHES », TRAIT. SIGNAL, VOL. 8, NO 6, P. 387-407, 1991.
- [64] J. DE VILLIERS ET E. BARNARD, « BACKPROPAGATION NEURAL NETS WITH ONE AND TWO HIDDEN LAYERS », IEEE TRANS. NEURAL NETW., VOL. 4, NO 1, P. 136-141, JANV. 1993.

- [65] APPRENDRE LES CONTRAINTES TOPOLOGIQUES DANS LES CARTES AUTO-ORGANISATRICES
GUENAËL CABANES*, YOUNES BENNANI* *LIPN-CNRS, UMR 7030 99 AVENUE J-B.
CLEMENT, 93430 VILLETANEUSE, FRANCE CABANES@LIPN.UNIV-PARIS13.FR
- [66] [HTTPS://FR.WIKIVERSITY.ORG/WIKI/RESEAUX_DE_NEURONES/APPLICATIONS_DES_RESEAU
X_DE_NEURONES#CITE_NOTE-1](https://fr.wikiversity.org/wiki/RESEAUX_DE_NEURONES/APPLICATIONS_DES_RESEAU_X_DE_NEURONES#CITE_NOTE-1).
- [67] H. BHAVSAR ET A. GANATRA, « A COMPARATIVE STUDY OF TRAINING ALGORITHMS FOR
SUPERVISED MACHINE LEARNING », INT. J. SOFT COMPUT. ENG. IJSCE, VOL. 2, NO 4, P.
2231-2307, 2012.
- [68] A BRIEF INTRODUCTION TO NEURAL NETWORKS DAVID KRIESEL UNIVERSITY OF BONN IN
GERMANY 2007.
- [69] « CLASSIFICATION DES IMAGES AVEC LES RESEAUX DE NEURONES CONVOLUTIONNELS »
MOKRI MOHAMMED ZAKARIA, UNIVERSITE ABOU BAKR BELKAID TLEMSEN 2017.
- [70] GRADIENTBASED LEARNING APPLIED TO DOCUMENT RECOGNITION YANN LECUN –LEON
BOTTOU- YOSHUA BENGIO - PATRICK HANER NEWYORK UNIVERSITY 1998.
- [71] IMAGENET CLASSIFICATION WITH DEEP CONVOLUTIONAL NEURAL NETWORKS A.
KRIZHEVSKY, I. SUTSKEVER - G. E. HINTON IN ADVANCES IN NEURAL INFORMATION
PROCESSING SYSTEMS, PP. 1097–1105 - UNIVERSITY OF TORONTO 2012.
- [72] VISUALIZING AND UNDERSTANDING CONVOLUTIONAL NETWORKS M. D. ZEILER - R.
FERGUS IN EUROPEAN CONFERENCE ON COMPUTER VISION, PP. 818–833- 2014.
- [73] XCEPTION: DEEP LEARNING WITH DEPTHWISE SEPARABLE CONVOLUTIONS FRANCOIS
CHOLLET; THE IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION
(CVPR) PP. 1251-1258 – 2017.
- [74] REAL-TIME CONVOLUTIONAL NEURAL NETWORKS FOR EMOTION AND GENDER
CLASSIFICATION OCTAVIO ARRIAGA UNIVERSITY OF BREMEN GERMANY 2017.
- [75] DEEP RESIDUAL LEARNING FOR IMAGE RECOGNITION K. HE, X. ZHANG, S. REN, J. SUN IN
PROCEEDINGS OF THE IEEE CONFERENCE ON COMPUTER VISION AND PATTERN
RECOGNITION, PP. 770–778- 2016.
- [76] DEEP LEARNING POUR LA CLASSIFICATION DES IMAGES MOUALEK DJALOUL YUCEF
UNIVERSITÉ ABOU BAKR BELKAID TLEMSEN 2017.
- [77] VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION KAREN
SIMONYAN - ANDREW ZISSERMAN + VISUAL GEOMETRY GROUP, DEPARTMENT OF
ENGINEERING SCIENCE, UNIVERSITY OF OXFORD 2015.
- [78] [HTTPS://OPENCLASSROOMS.COM/FR/COURSES/4470531-CLASSEZ-ET-SEGMENTEZ-DES-
DONNEES-VISUELLES/5097666-TP-IMPLEMENTEZ-VOTRE-PREMIER-RESEAU-DE-
NEURONES-AVEC-KERAS](https://openclassrooms.com/fr/courses/4470531-classez-et-segmentez-des-donnees-visuelles/5097666-tp-implementez-votre-premier-reseau-de-neurones-avec-keras)
- [79][HTTP://DEEPLEARNING.STANFORD.EDU/TUTORIAL/SUPERVISED/CONVOLUTIONALNEURALN
ETWORK/](http://deeplearning.stanford.edu/tutorial/supervised/convolutionalneuralnetwork/)
- [80][HTTPS://FR.M.WIKIPEDIA.ORG/WIKI/RASPBERRY_PI?IORG_SERVICE_ID_INTERNAL=8034784
43041409%3BAfpPP1KHdHC1h2_w#cite_note-site_RASPBERRY-3](https://fr.m.wikipedia.org/wiki/Raspberry_Pi?iorg_service_id_internal=803478443041409%3BAfpPP1KHdHC1h2_w#cite_note-site_Raspberry-3)
- [81] [HTTPS://WWW.GRAFIKART.FR/BLOG/RASPBERRY-PI-UTILISATION](https://www.grafikart.fr/blog/raspberry-pi-utilisation)

- [82] BROWN, G. D., YAMADA, S., & SEJNOWSKI, T. J. (2001). INDEPENDENT COMPONENT ANALYSIS AT THE NEURAL COCKTAIL PARTY. TRENDS IN NEUROSCIENCES, 24(1), 54-63.
- [83] [HTTPS://BLOG.AXOPEN.COM/2019/09/OPEN-CV-CEST-QUOI/](https://blog.axopen.com/2019/09/open-cv-cest-quoi/)
- [84] [HTTPS://RIPTUTORIAL.COM/FR/KERAS/EXAMPLE/27132/INSTALLATION-ET-CONFIGURATION](https://riptutorial.com/fr/keras/example/27132/installation-et-configuration)
- [85] [HTTPS://WWW.GOOGLE.COM/URL?SA=T&RCT=J&Q=&ESRC=S&SOURCE=WEB&CD=&CAD=RJA&UACT=8&VED=2AHUKEwibZYNH8CXQAHWf3YUKHRUJABWQFjACEGQIAHAB&URL=HTTPS%3A%2F%2FDOC.UBUNTU-FR.ORG%2FMINICONDA&USG=AOvVaw0Xpa7rdY-X1nnPPZw8yE_H](https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&cad=rja&uact=8&ved=2AHUKEwibZYNH8CXQAHWf3YUKHRUJABWQFjACEGQIAHAB&url=https%3A%2F%2Fdoc.ubuntu-fr.org%2Fminiconda&usg=AOvVaw0Xpa7rdY-X1nnPPZw8yE_H)
- [86] [HTTPS://MAKER.PRO/RASPBERRY-PI/TUTORIAL/HOW-TO-INTERFACE-A-PIR-MOTION-SENSOR-WITH-RASPBERRY-PI-GPIO](https://maker.pro/raspberry-pi/tutorial/how-to-interface-a-pir-motion-sensor-with-raspberry-pi-gpio)
- [87] [HTTPS://WWW.ELPROCUS.COM/LCD-16X2-PIN-CONFIGURATION-AND-ITS-WORKING/](https://www.elprocus.com/lcd-16x2-pin-configuration-and-its-working/)